



**MS860**

Les fondamentaux du  
développement .Net en  
C# sous Visual Studio



# Plan de la formation

- 1 - Introduction à C# et .Net**
- 2 - Structure de programmation C#**
- 3 - Déclaration et appel de méthodes**
- 4 - Gestion d'exceptions**
- 5 - Lire et écrire dans des fichiers**
- 6 - Création de nouveaux types de données**
- 7 - Encapsulation de données et de méthodes**
- 8 - Héritage de classes et implémentation d'interfaces**
- 9 - Gestion de la durée de vie des objets et contrôle des ressources**
- 10 - Encapsulation avancée**
- 11 - Découplage de méthodes et gestion d'évènements**
- 12 - Utilisation des collections et construction de types génériques**
- 13 - Programmation asynchrone et personnalisation du code**
- 14 - Utilisation de LINQ pour interroger des données**
- 15 - Développement dirigé par les Tests**



## Environnement de travail

Pour réaliser les ateliers nous allons utiliser

- Visual Studio 2022 Community Edition
- .NET 6

Il serait également possible de travailler avec Visual Studio Code et l'utilitaire *dotnet* fourni avec .NET 6

# Chapitre 1 - Introduction à C# et .Net

# Plan de la formation

- 1 - Introduction à C# et .Net
- 2 - Structure de programmation C#
- 3 - Déclaration et appel de méthodes
- 4 - Gestion d'exceptions
- 5 - Lire et écrire dans des fichiers
- 6 - Création de nouveaux types de données
- 7 - Encapsulation de données et de méthodes
- 8 - Héritage de classes et implémentation
- 9 - Gestion de la durée de vie des objets et contrôle des ressources
- 10 - Encapsulation avancée
- 11 - Découplage de méthodes et gestion d'évènements
- 12 - Utilisation des collections et construction de types génériques
- 13 - Programmation asynchrone et personnalisation du code
- 14 - Utilisation de LINQ pour interroger des données
- 15 - Développement dirigé par les Tests

## Objectifs :

- Introduire les participants au cadre et à la philosophie de .NET.
- Familiariser les participants avec l'environnement de développement Visual Studio 2022.
- Initier les participants à la syntaxe de base de C# et à la structure d'une application C#.
- Mettre en évidence l'importance de la documentation dans le développement d'applications.
- Fournir une compréhension de base des techniques de débogage avec Visual Studio 2022.



# Commençons par échanger

## **Expériences précédentes avec .NET et C# :**

"Parmi vous, qui a déjà eu l'occasion de travailler ou d'expérimenter avec la plateforme .NET ou le langage C# ? Si oui, dans quel contexte ou projet l'avez-vous utilisé ?"

## **Comparaison avec d'autres langages :**

"Certains d'entre vous ont de l'expérience en programmation avec d'autres langages comme Java, C++ ou Python. Comment imaginez-vous que C# se compare à ces langages en termes de syntaxe, de fonctionnalités et d'écosystème ?"

## **Attentes envers Visual Studio 2022 :**

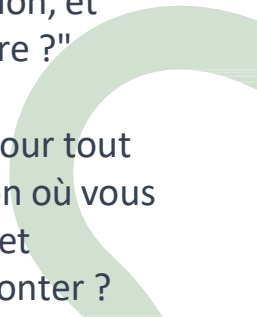
"Quelle est votre expérience avec les environnements de développement intégrés (IDE) en général ? Que recherchez-vous dans un bon IDE et quelles sont vos attentes envers Visual Studio 2022 en particulier ?"

## **Importance de la documentation :**

"Quelle est votre approche personnelle concernant la documentation de votre code ? Pourquoi pensez-vous qu'il est important de bien documenter une application, et quelles sont vos méthodes préférées pour le faire ?"

## **Expérience avec le débogage :**

"Le débogage est une compétence essentielle pour tout développeur. Pouvez-vous partager une situation où vous avez été confronté à un bug difficile à résoudre et comment vous avez finalement réussi à le surmonter ? Quels outils ou techniques avez-vous trouvés les plus utiles dans ce processus ?"





# Leçon 1: Introduction au .NET Framework



- Qu'est-ce que .NET?
- Historique
- .NET
- C#
- Notion d'Assembly
- Comment le CLR charge, compile et exécute les assemblies
  - Outils .NET

# Qu'est-ce que .NET?

- Plateforme de développement gratuite et open-source
- Support d'un grand nombre de types d'application
  - Web, Web API, Cloud, Mobile, Desktop...
- Multi-plateforme
  - Windows, macOS, Linux, Android, iOS...
- Choix du langage de programmation
  - C#, F# ou Visual Basic
- Basé sur un environnement d'exécution « managé » : CLR (Common Language Runtime)



# Historique

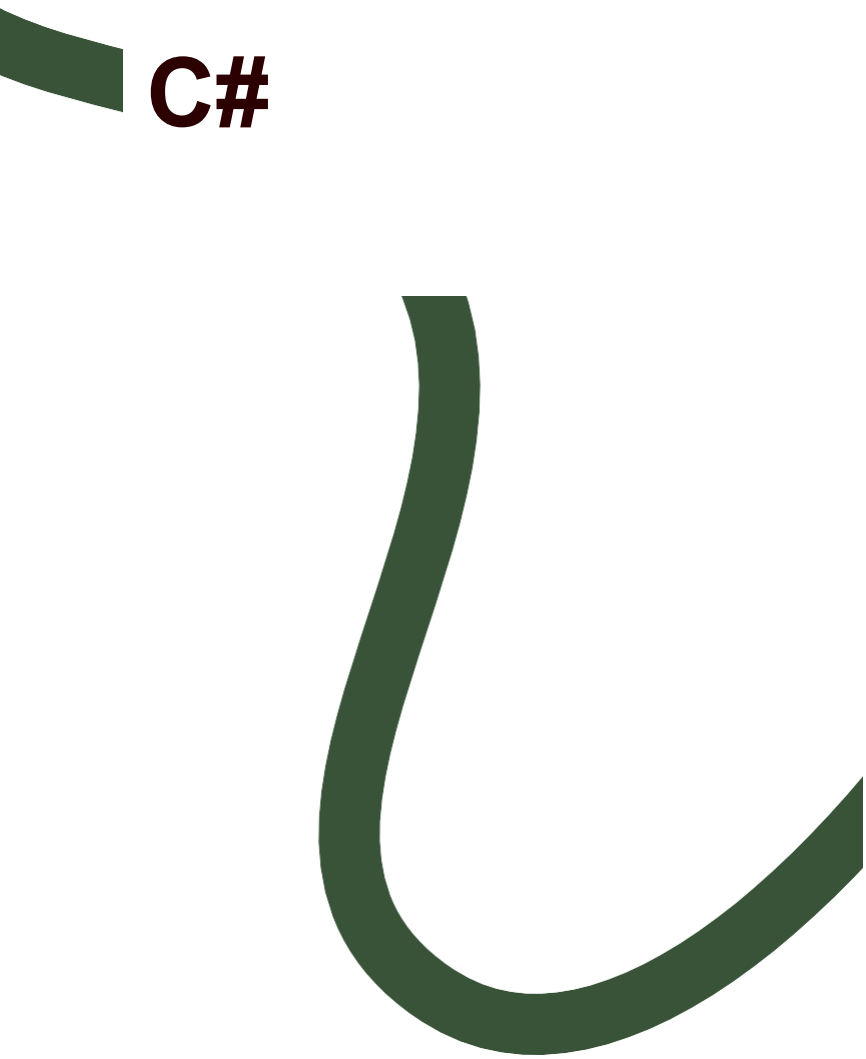
| Windows |                                              | Multi-OS      |
|---------|----------------------------------------------|---------------|
| 2002    | .NET Framework 1.0                           |               |
| 2005    | .NET Framework 2.0                           |               |
| 2006    | .NET Framework 3.0<br>(WPF – WCF – WF)       |               |
| 2007    | .NET Framework 3.5<br>(LINQ)                 |               |
| 2008    | .NET Framework 3.5 SP1<br>(Entity Framework) |               |
| 2010    | .NET Framework 4.0                           |               |
| 2012    | .NET Framework 4.5                           |               |
| 2015    | .NET Framework 4.6                           |               |
| 2016    |                                              | .NET Core 1.0 |
| 2017    | .NET Framework 4.7                           | .NET Core 2.0 |
| 2019    | .NET Framework 4.8                           | .NET Core 3.0 |
| 2020    |                                              | .NET 5        |
| 2021    |                                              | .NET 6        |



# .NET 6

## — Dernière version de .NET Core

- ▶ .NET 5 ne s'appelle pas .NET Core 4.0 pour deux raisons:
  - Eviter la confusion avec .NET Framework 4.x
  - Ne plus utiliser le terme Core pour mettre en avant qu'il s'agit de l'implémentation principale de .NET à l'avenir
- ▶ Ne signifie pas la mort de .NET Framework
  - Web Forms et WF ne sont pas supportés en .NET 5

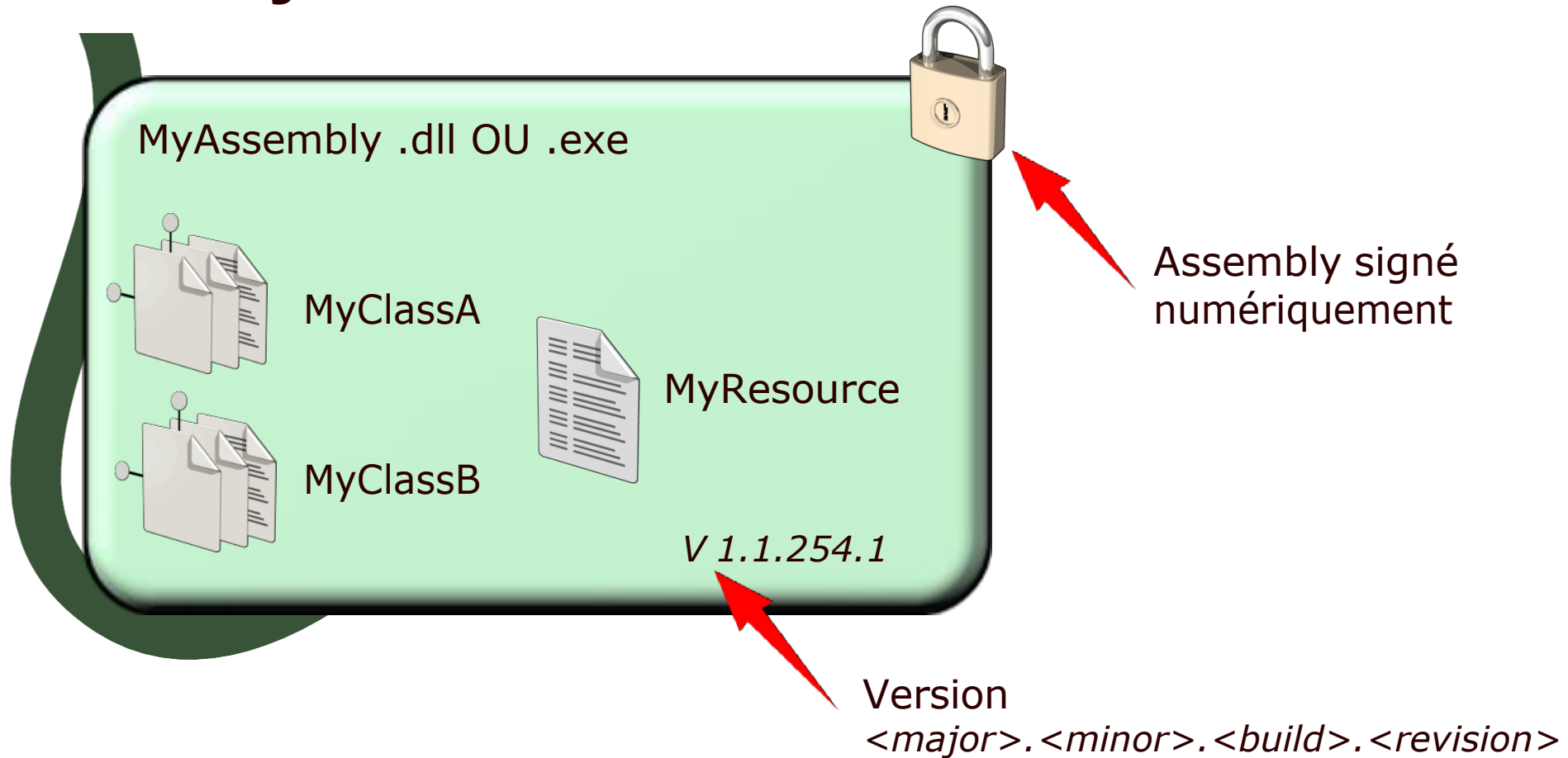


# C#

- C# est devenu le langage de prédilection des développeurs .NET
- C# est syntaxiquement très proche des langages C, C++ ou encore Java
- C# a été normalisé par des spécifications ECMA et ISO/IEC
- Les dernières évolutions sont liées à .NET Core
  - ▶ C# 7.3 avec .NET Framework 4.8
  - ▶ C# 8 avec .NET Core 3.x
  - ▶ C# 9 avec .NET 5
  - ▶ C# 10 avec .NET 6

# Notion d'Assembly

- Blocs élémentaires des applications .NET
- Collection de types et de ressources

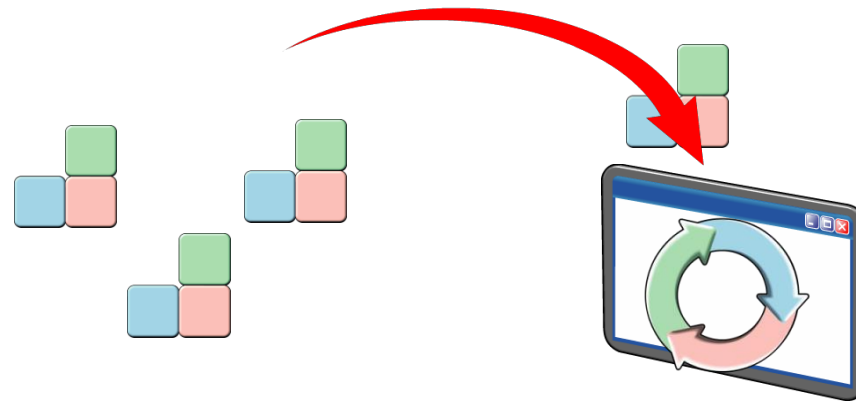


# Comment le CLR charge, compile et exécute les Assemblies

Les Assemblies contiennent du code MSIL non exécutable

Le CLR charge le code MSIL et le compile en code natif en fonction de la machine physique

- 1 Charge les Assemblies que l'application référence
- 2 Vérifie et compile le code MSIL en langage machine
- 3 Exécute le code





# Outils .NET

## — .NET Core SDK

- ▶ .NET Core CLI: dotnet
- ▶ Bibliothèques et composants d'exécution

## — Les environnements de développement

- ▶ Visual Studio
- ▶ Visual Studio Code
- ▶ Visual Studio for Mac
- ▶ GitHub Codespaces



## **Leçon 2: Création de projets avec Visual Studio 2022**



- Visual Studio Code
- Visual Studio
- Gestion de projet
- Modèles de projet
- Démonstration :Création d'une application .NET, compilation et exécution

# Visual Studio Code

- Editeur de code source
  - ▶ Gratuit, léger, puissant et multi-plateforme
- Support de la plupart des langages (natif ou via extensions)
  - ▶ Natif: JavaScript, TypeScript, Node.js
  - ▶ Par extensions: C++, C#, Java, Python, PHP...







# Visual Studio

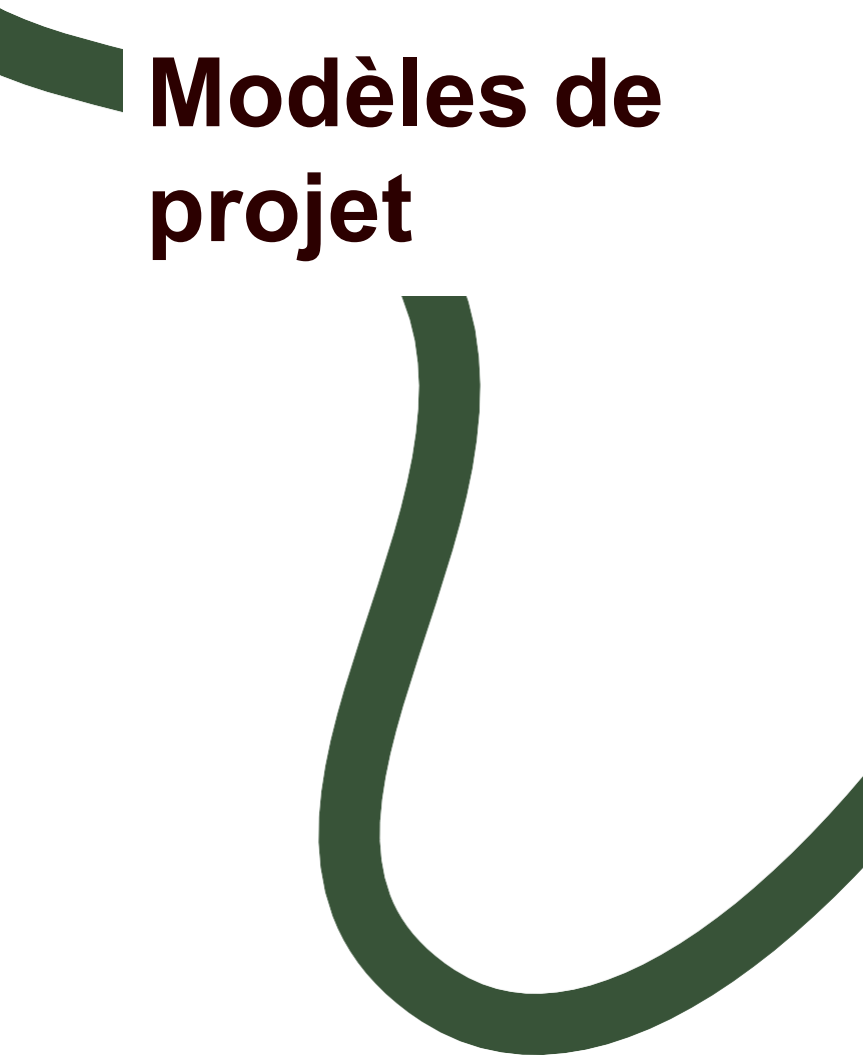


- Environnement de développement intégré
  - ▶ Plusieurs éditions: Community, Professional, Enterprise
- Support d'un grand nombre de fonctionnalité
  - ▶ Rapid Application Development
  - ▶ Gestion de données locales ou distantes
  - ▶ Débogage et Gestion d'erreur
  - ▶ Aide et documentation
  - ▶ Extensions par NuGet



# Gestion de projet

- Visual Studio gère toute l'arborescence physique d'un projet
  - ▶ Fichier Solution \*.sln, Fichier Projet \*.csproj
- Visual Studio Code nécessite plus de travail:
  - ▶ Création du dossier de projet
  - ▶ Création du projet via dotnet `new template`
    - `dotnet new --list`
  - ▶ Ouverture dans Visual Studio Code via `code .`
  - ▶ Workspace: ensemble de projets/dossiers associés dans VS Code, référencé par un fichier `.code-workspace` (json)



# Modèles de projet

- Bibliothèque de classe (*classlib*)
  - Application de bureau:
    - ▶ Console (*console*), Windows Forms (*winforms*), WPF (*wpf*)
  - Application Web:
    - ▶ Razor Pages (*webapp*), MVC (*mvc*), Angular (*angular*), Reast.js (*react*)
  - Architecture SOA:
    - ▶ Web API (*webapi*)
  - Tests Unitaires (*mstest*)



# Démonstration

- Création d'une application .NET, compilation et exécution





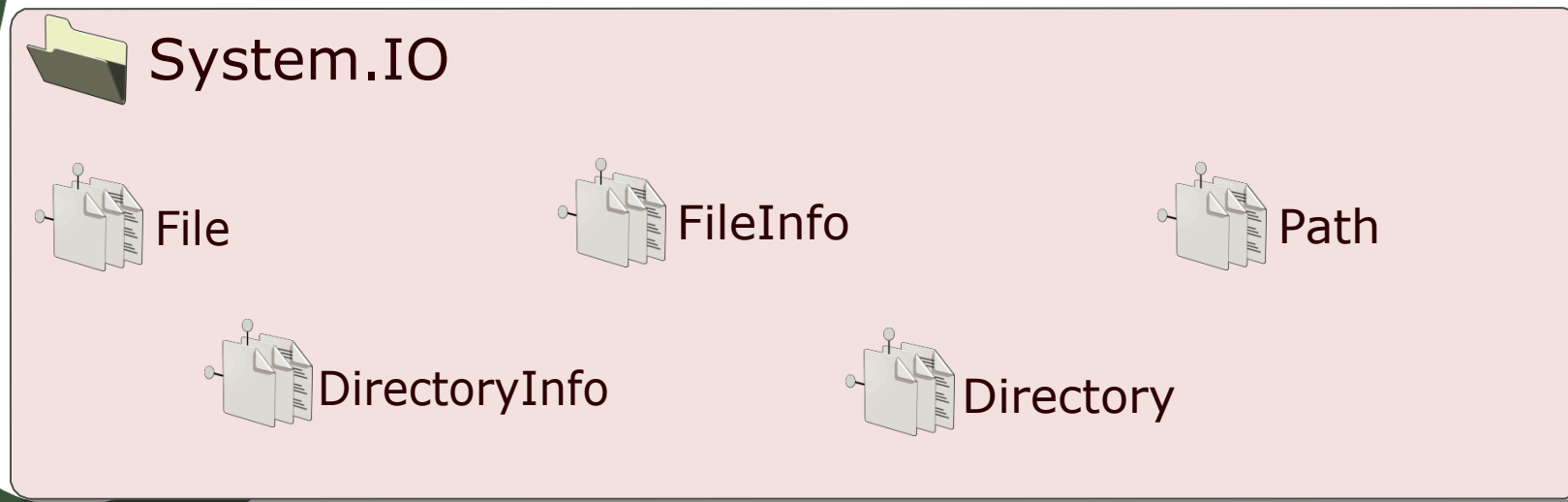
## **Leçon 3: Ecrire une application C#**



- Classes et Espaces de Noms
- Structure d'une application Console
- Entrées/Sorties dans une application Console
- Bonnes pratiques de commentaires

# Classes et Espaces de Noms

- Une classe est un plan permettant de définir les caractéristiques d'une entité
- Un Espace de Noms est une organisation logique de classes



# Structure d'une application Console

```
using System;  
namespace MyFirstApplication  
{  
    class Program  
    {  
        static void Main(string[] args)  
        {  
        }  
    }  
}
```

Création de l'alias System

Espace de noms

Classe Program

Point d'entrée du programme

# Entrées/Sorties dans une application Console

System.Console

Clear()

Read()

ReadKey()

ReadLine()

Write()

WriteLine()

C:/

```
using System;  
...  
Console.WriteLine("Hello there!");
```



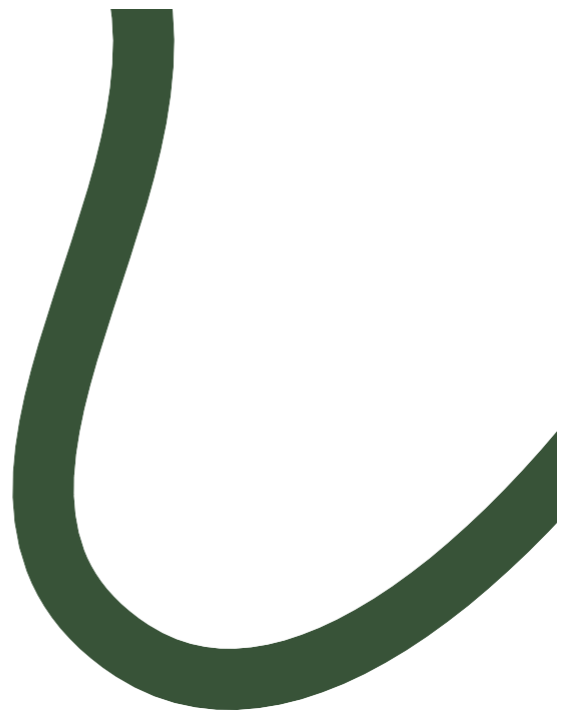
# Bonnes pratiques de commentaires

- Faire précéder les déclarations par un commentaire
- Dans une méthode, ajouter des commentaires intermédiaires pour aérer le code ou documenter les structures de décision
- Respecter les conventions de nommage:
  - ▶ **PascalCasing**: Espaces de noms, Classes, Méthodes
  - ▶ **camelCasing**: variables, paramètres
- Utiliser un préfixe de type pour les variables et paramètres

```
/* Ceci est un bloc  
de commentaires */  
string strMessage = "Hello there!"; // Commentaire de ligne
```



# **Leçon 4: Documenter une Application**



- Commentaires XML
- Génération de la documentation

# Commentaires XML

```
/// <summary> The Hello class prints a greeting on the screen
/// </summary>
public class Hello
{
    /// <summary> We use console-based I/O. For more information
    /// about
    /// WriteLine, see <seealso cref="System.Console.WriteLine"/>
    /// </summary>
    public static void Main( )
    {
        Console.WriteLine("Hello World");
    }
}
```

Ces commentaires peuvent être générés en XML à la compilation pour être ensuite dissociés du code.



## Génération de la documentation

- Sandcastle Help File Builder

- ▶ solution graphique basée sur l'outil Sandcastle initialement fourni et développé par Microsoft

- DocFX

- ▶ solution mise en avant par Microsoft depuis 2017, basée sur un outil ligne de commande (proche de javadoc)

- Différents produits tiers existent également

- ▶ GhostDoc, Atomineer Pro Documentation, Doxygen, Wyam...



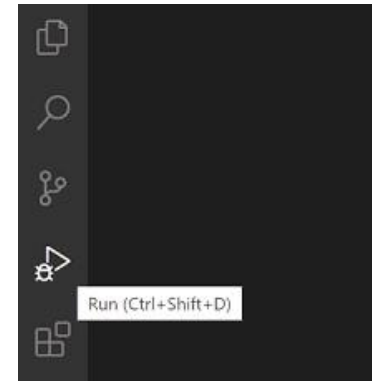
## **Leçon 6: Exécuter et débuguer des applications avec Visual Studio 2022**



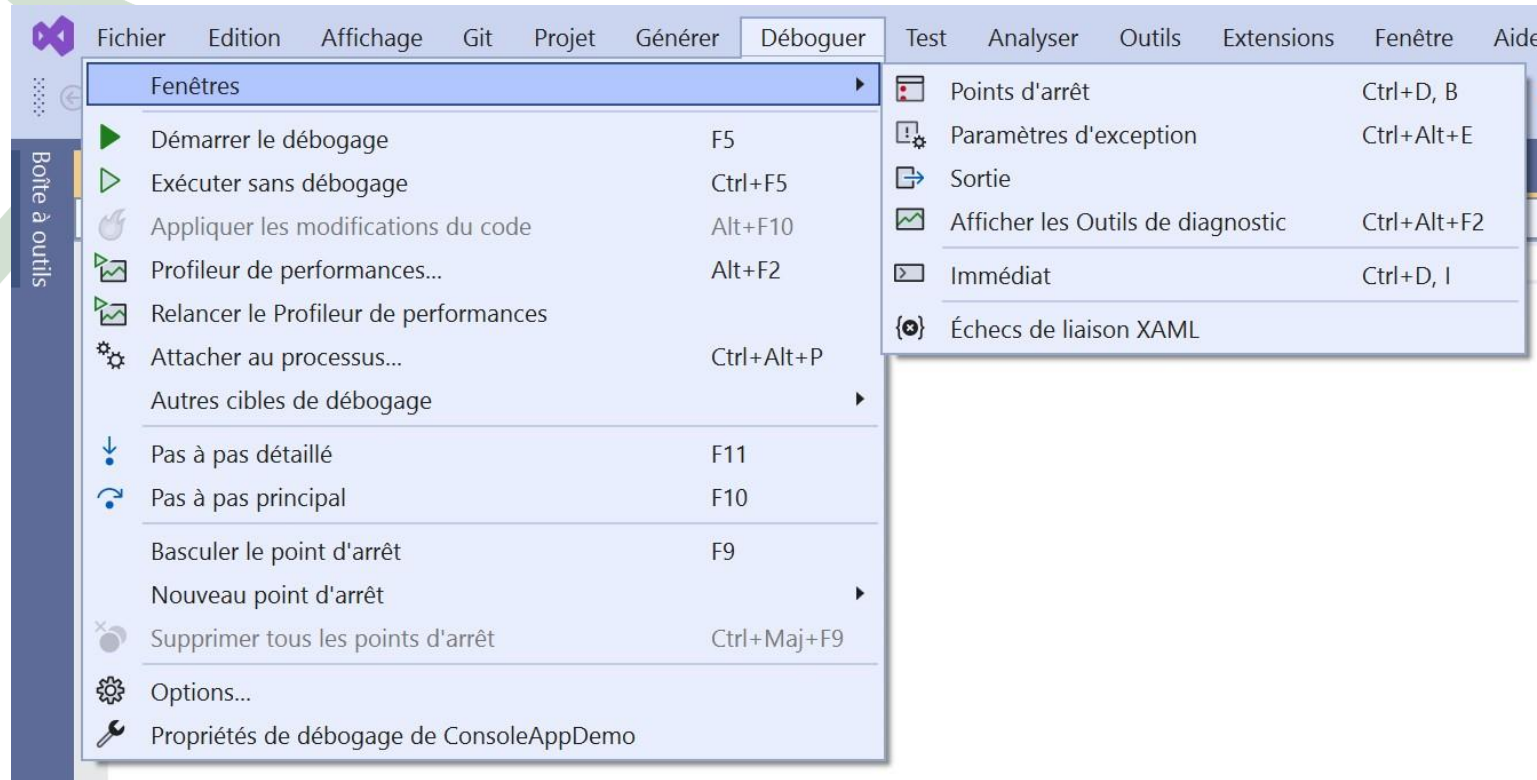
- Débogage
- Points d'arrêt
- Pas à pas
- Panneaux de débogage

# Débogage

- Le débogage consiste habituellement à corriger les erreurs de logique du code
- Composante importante de tout éditeur de code
- Visual Studio Code propose une vue d'exécution:
  - ▶ Gestion de points d'arrêt
  - ▶ Gestion du pas à pas
  - ▶ Visualisation et modification des variables pendant le débogage
  - ▶ Pile des appels
- Visual Studio offre une gestion complète du débogage



# Débogage et Visual Studio





# Points d'arrêt

- Mécanisme de mise en pause de l'exécution
  - ▶ Sur une ligne précise
  - ▶ Sur une ligne précise, en fonction d'une condition
  - ▶ Lors de l'appel d'une méthode (l'interruption se fait à la première ligne de cette méthode)
  - ▶ Sur une ligne précise, sur une colonne précise (utile pour du code minifié)
- Désactivation/Réactivation
  - ▶ Conserver les points d'arrêts sans qu'ils soient actifs
- Persistance après un redémarrage de Visual Studio/Visual Studio Code



# Pas à Pas

- Permet d'avancer dans l'exécution ligne par ligne durant une session de débogage

- 3 modes:

- Pas à Pas Principal (*Step Over* – F10)

- Avance ligne par ligne
    - Ne rentre pas dans les méthodes appelées

- Pas à Pas Détaillé (*Step Into* – F11)

- Avance ligne par ligne
    - Parcours tout le code

- Pas à Pas Sortant (*Step Out* – Shift+F11)

- Finis l'exécution de la méthode en cours
    - Interromps de nouveau l'exécution à la ligne d'appel de la méthode



# Panneaux de débogage

## — Variables

- ▶ Visualisation des variables actuellement utilisées
- ▶ Possibilité d'édition du contenu
- ▶ Accès en profondeur au contenu d'un objet

## — Espions (Watch)

- ▶ Visualisation des expressions sélectionnées

## — Pile des appels (Call Stack)

- ▶ Visualisation de l'imbrication des appels de méthodes

## — Points d'arrêts (Breakpoints)

- ▶ Gestion et édition des points d'arrêts

# Atelier 01

- Exercice 1: Création d'un programme simple en C#
- Exercice 2: Utilisation du débogueur

**Activité**  
**20 min**





## A retenir

1. **.NET Framework et sa Philosophie** : ".NET est une plateforme de développement polyvalente et puissante qui prend en charge de nombreux langages, C# étant l'un des plus populaires."
2. **Historique et Évolution** : "Le .NET Framework a considérablement évolué au fil des ans, avec .NET 6 représentant la dernière version unifiée, regroupant les meilleures fonctionnalités de ses prédécesseurs."
3. **Comprendre les Assemblies** : "Un Assembly est l'unité de base du déploiement, de la version, des autorisations et de la sécurité dans .NET. Comprendre son fonctionnement est essentiel pour travailler avec .NET."
4. **Rôle du CLR** : "Le Common Language Runtime (CLR) est le moteur au cœur de .NET, responsable de la compilation Just-In-Time (JIT) et de l'exécution des applications .NET."
5. **Pouvoir des Outils .NET** : "Les outils fournis avec .NET, tels que Visual Studio, simplifient et accélèrent le processus de développement, de débogage et de déploiement."
6. **Visual Studio 2022 vs Visual Studio Code** : "Visual Studio 2022 est un IDE complet, tandis que Visual Studio Code est un éditeur de code source léger mais puissant. Chacun a ses propres forces et est adapté à des scénarios d'utilisation spécifiques."
7. **Structuration en C#** : "Les classes et les espaces de noms sont essentiels pour organiser et structurer votre code en C#. Ils aident à encapsuler la logique et à favoriser la réutilisabilité."
8. **Importance de la Documentation** : "Documenter votre code, en particulier avec des commentaires XML, n'est pas seulement pour les autres développeurs. C'est un investissement pour vous-même, pour l'avenir, facilitant la maintenance et la compréhension du code."
9. **Débogage dans Visual Studio 2022** : "Le débogueur de Visual Studio 2022 est un outil puissant qui permet de traquer et de résoudre les problèmes dans le code. Maîtriser les points d'arrêt, le pas à pas et les panneaux de débogage est essentiel pour tout développeur .NET."
10. **Bonnes Pratiques** : "Outre l'écriture et le débogage de code, adopter de bonnes pratiques, comme commenter correctement son code et comprendre la structure d'une application, est essentiel pour la qualité et la pérennité du logiciel."

# Chapitre 2 - Introduction à C# et .Net

# Plan de la formation

- 1 - Introduction à C# et .Net
- 2 - **Structure de programmation C#**
- 3 - Déclaration et appel de méthodes
- 4 - Gestion d'exceptions
- 5 - Lire et écrire dans des fichiers
- 6 - Création de nouveaux types de données
- 7 - Encapsulation de données et de méthodes
- 8 - Héritage de classes et implémentation d'interfaces
- 9 - Gestion de la durée de vie des objets et contrôle des ressources
- 10 - Encapsulation avancée
- 11 - Découplage de méthodes et gestion d'évènements
- 12 - Utilisation des collections et construction de types génériques
- 13 - Programmation asynchrone et personnalisation du code
- 14 - Utilisation de LINQ pour interroger des données
- 15 - Développement dirigé par les Tests

## Objectifs :

- Compréhension des Bases
- Manipulation de Données
- Contrôle du Flux d'Exécution
- Introduction à l'Optimisation



# Commençons par échanger

**Expérience antérieure avec les langages de programmation :**  
"Pouvez-vous me donner des exemples de langages de programmation que vous avez utilisés auparavant et comment vous avez structuré votre code dans ces langages ?"

**Concept de Variables :**

"Dans votre expérience de programmation, comment définiriez-vous une variable et pourquoi est-elle si essentielle dans le développement d'une application ?"

**Instructions Conditionnelles :**

"Pouvez-vous décrire une situation où vous avez dû utiliser des conditions pour décider de la suite des événements dans une application ou un programme ? Comment avez-vous géré cela ?"

**Boucles et Itérations :**

"Avez-vous déjà rencontré une situation où vous deviez répéter une tâche plusieurs fois dans votre code ? Comment avez-vous abordé cette répétition et quel type de boucle avez-vous utilisé ?"

**Optimisation et Bonnes Pratiques :**

"Dans vos projets précédents, comment avez-vous veillé à ce que votre code soit à la fois efficace et lisible ? Avez-vous des exemples de bonnes pratiques que vous avez adoptées ou rencontrées ?"



# **Vue d'ensemble**

- Déclaration de variables et affectation de valeurs
- Utilisation d'expression et d'opérateurs
- Création et utilisation des tableaux
  - Instructions de décision
  - Instructions d'itérations
- Extraits de code de Visual Studio 2022





# **Leçon 1: Déclaration de variables et affectation de valeurs**



- Définition
- Types de données
- Déclaration et affectation
  - Suffixes et Littéraux
  - Portée
  - Conversion
- Variables lecture seule et constantes
  - Typage implicite

# Définition

Les variables stockent des valeurs requises par l'application dans des emplacements mémoires temporaires

Les variables sont caractérisées par:

Nom

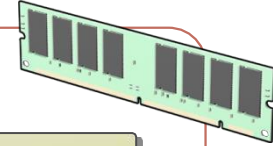
Adresse

Type de données

Valeur

Portée

Durée de vie



# Types de données

C# est un langage typé fort

Le compilateur garantit qu'une valeur stockée dans une variable est du type approprié

Quelques exemples de type de données:

int

long

float

double

decimal

char

bool

DateTime

string



# Déclaration et Affectation

Avant de pouvoir utiliser une variable, il faut la déclarer

```
DataType variableName;  
...  
DataType variableName1, variableName2;  
...  
DataType variableName = new DataType();
```

Après avoir déclaré une variable, il est possible de lui affecter une valeur

```
DataType variableName = value;  
...  
variableName = anotherValue;
```

NOTE: Le nom d'une variable est un identifiant et doit respecter:

- Ne peut contenir que des lettres, chiffres et le caractère \_
- Doit débuter par une lettre ou \_
- Ne doit pas être un des mots-clés réservés du C# (sauf si préfixé par @)

## ○ Possibilité de typer une valeur numérique par un suffixe

Type : unsigned int  
Caractère : U  
Exemple : uint x = 100U;

Type : long  
Caractère : L  
Exemple : long x = 100L;

Type : unsigned long  
Caractère : UL  
Exemple : ulong x = 100UL;

Type : float  
Caractère : F  
Exemple : float x = 100F;

Type : double  
Caractère : D  
Exemple : double x = 100D;

Type : decimal  
Caractère : M  
Exemple : decimal x = 100M;

- Possibilité d'écrire des littéraux numériques de façon plus lisible

```
byte binaire = 0b01010101;  
byte binary = 0b0001_1000;
```

```
int hex = 0xAB_BA;
```

```
int dec = 1_234_567_890;
```

# Portée

## Bloc

```
if (length > 10)
{
    int area = length * length;
}
```

## Méthode

```
void ShowName()
{
    string name = "Bob";
}
```

## Classe

```
private string message;

void SetString()
{
    message = "Hello World!";
}
```

## Espace de noms

```
public class CreateMessage
{
    public string message
        = "Hello";
}

public class DisplayMessage
{
    ...
}

public void ShowMessage()
{
    CreateMessage newMessage
        = new CreateMessage();
    MessageBox.Show(
        newMessage.message);
}
```

# Conversion

## Conversion Implicite

Réalisée automatiquement par le CLR

```
int a = 4;  
long b;  
b = a;           // Conversion implicit int vers long
```

## Conversion Explicite

Nécessite d'écrire le code de conversion

```
DataType variableName1 = (castDataType) variableName2;  
...  
int count = Convert.ToInt32("1234");  
...  
int number = int.Parse("1234");  
...  
int number = 0;  
if (int.TryParse("1234", out number)) { // Conversion réussie }
```

# Variables lecture seule et constantes

## Champs en lecture seule

Déclarés avec le mot clé readonly  
Initialisés à l'exécution

```
readonly string currentDateTime = DateTime.Now.ToString();
```

## Constantes

Déclarées avec le mot clé const  
Initialisées à la compilation

```
const double PI = 3.14159;  
int radius = 5;  
double area = PI * radius * radius;  
double circumference = 2 * PI * radius;
```



- Grâce au mot-clé `var`, il est possible de ne pas préciser le type d'une variable lors de sa déclaration
- Obligation d'effectuer une assignation de valeur
- Le type de la valeur affectée est utilisé par le compilateur pour déterminer le type de la variable

```
var myString = "Hello World!"; // myString est typé String  
var data = 5;                  // data est typé int
```

## Leçon 2: Utilisation d'expression et d'opérateurs

---

- Expressions
- Opérateurs
- Bonnes pratiques de concaténation de chaînes

# Expressions

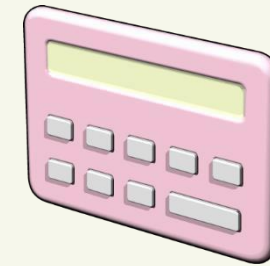
Les expressions sont les structures fondamentales permettant d'évaluer et de manipuler des données

```
a + 1
```

```
(a + b) / 2
```

```
"Answer: " + c.ToString()
```

```
b * System.Math.Tan(theta)
```



# Opérateurs

Les opérateurs sont des séquences de caractères utilisés pour définir des opérations réalisées sur des opérandes

Arithmétique +, -, \*, /, %

Assignation =, +=, -=, \*=, /=

Incrément, décrément ++, --

Décalage de bit <<, >>

Comparaison ==, !=, <, >, <=, >=, is

Information sizeof, typeof, nameof

Concaténation +

Délégué +, -

Logique/binaire &, |, ^, !, ~, &&, ||

Dépassement checked, unchecked

Tableaux [ ], ^, ..

Indirection, Adresse \*, ->, [ ], &

Cast ( ), as

Conditionnel ?:, ?? (??=), ?.

# Bonnes pratiques de concaténation de chaînes

La concaténation de chaînes de caractères peut être effectuée très simplement à l'aide de l'opérateur +

```
string address = "23";  
address = address + ", Oxford Street";  
address = address + ", Thornbury";
```

Cela est toutefois considéré comme une mauvaise pratique car les chaînes de caractères sont immuables (problème de performance)

## ○ Première solution: la classe StringBuilder

```
StringBuilder address = new StringBuilder();  
  
address.Append(adresse.Numero);  
address.Append(adresse.Rue);  
address.Append(adresse.Ville);  
  
string concatenatedAddress = address.ToString();
```

Deuxième solution: la méthode String.Format  
(on retrouve la même implementation dans Console.WriteLine)

```
string concatenatedAddress = String.Format(  
    "{0}, {1}, {2}",  
    adresse.Numero, adresse.Rue, adresse.Ville);
```

Troisième solution: l'interpolation de chaînes

```
string concatenatedAddress =  
    $"{adresse.Numero}, {adresse.Rue}, {adresse.Ville}";
```

# Préfixes de chaînes de caractères

- Le caractère \ est un caractère d'échappement dans une chaîne
  - \" => ", \t => tabulation, \n => nouvelle ligne, \r => retour, ...
  - Obligation de doubler \ dans un chemin d'accès

- Utilisation du préfixe @ pour que \ ne soit plus interprété

```
string path = "C:\\Users\\Admin\\Documents\\File.txt";
```

- Il est possible de cumuler @ et \$ (@\$"..." ou \$"@"...")

```
string path = @"C:\Users\Admin\Documents\File.txt";
```

## Leçon 3: Création et utilisation de tableaux

---

- Définition
- Création et Initialisation
- Quelques propriétés et méthodes
- Accès aux données dans un tableau
- Indices et Plages



- Un tableau est une séquence d'éléments de même type, groupés ensemble
  - Chaque élément du tableau contient une valeur
  - Les indices sont numérotés à partir de zéro
  - La longueur d'un tableau est le nombre maximum d'éléments qu'il peut contenir
  - Le rang d'un tableau correspond à son nombre de dimensions
  - Les tableaux peuvent être unidimensionnels, multidimensionnels ou imbriqués
  - Les tableaux ont une taille fixe en C#

# Création et Initialisation

## Unidimensionnel

```
Type[] arrayName = new Type[ Size ];  
  
int[] monTableau = new int[10];
```

## Multidimensionnel

```
Type[ , ] arrayName = new Type[ Size1, Size2];  
  
int[,] maMatrice = new int[10,10];
```

## Imbriqué

```
Type[][] JaggedArray = new Type[size][];  
  
int[][] mesCinqTableaux = new int[5][];
```

# Quelques propriétés et méthodes

## Length

```
int[] oldNumbers = new int[5]{ 1, 2, 3, 4, 5 };  
int numberCount = oldNumbers.Length;  
int numberRank = oldNumbers.GetLength(0);
```

## Rank

```
int[] oldNumbers = { 1, 2, 3, 4, 5 };  
int rank = oldNumbers.Rank;
```

## CopyTo()

```
int[] oldNumbers = { 1, 2, 3, 4, 5 };  
int[] newNumbers = new int[oldNumbers.Length];  
oldNumbers.CopyTo(newNumbers, 0);
```

## Sort()

```
int[] oldNumbers = { 5, 2, 1, 3, 4 };  
Array.Sort(oldNumbers);
```

# Accès aux données dans un tableau

Les indices d'un tableau de N éléments vont de 0 à N-1

## Accès à un élément spécifique

```
int[] oldNumbers = { 1, 2, 3, 4, 5 };  
int number = oldNumbers[2];
```

## Parcourir tous les éléments

```
int[] oldNumbers = { 1, 2, 3, 4, 5 };  
...  
for (int i = 0; i < oldNumbers.Length; i++)  
{  
    int number = oldNumbers[i];  
    Console.WriteLine(number);  
}  
// OU  
foreach (int number in oldNumbers) {  
    Console.WriteLine(number);  
}
```

- System.Index: Parcourir le tableau depuis sa fin à l'aide de l'opérateur ^ (décompte à partir de 1)

```
int[] oldNumbers = { 1, 2, 3, 4, 5 };  
int number = oldNumbers[4];  
int sameNumber = oldNumbers[^1];
```

- System.Range: Considérer une plage de valeurs dans le tableau à l'aide de l'opérateur .. (du type [x,y[ )

```
int[] oldNumbers = { 1, 2, 3, 4, 5 };  
int number = oldNumbers[1..2]; // 2  
int[] extract = oldNumbers[..3]; // {1, 2, 3}  
int[] otherExtract = oldNumbers[2..]; // {3, 4, 5}  
...  
Range plage = 1..4;  
int[] morceau = oldNumbers[plage];
```

## Leçon 4: Instructions de décision

- Instruction if
- Instruction if else
- Imbrication d'instructions if
- Instruction switch
- Critères Spéciaux

# Instructions if

## syntaxe

```
if ([condition]) [code à executer]

// OU

if ([condition])
{
    [code à executer si la condition est vraie]
}
```

Opérateurs Conditionnels:

OU est représenté par ||

ET est représenté par &&

## Exemple

```
if ((percent >= 0) && (percent <= 100))
{
    // Ajouter le code à exécuter si percent est entre 0 et 100.
}
```

# Instructions if else

Permet de traiter le cas où la condition du Si est fausse

## Exemple

```
if (a > 50)
{
    // a est supérieur à 50
}
else
{
    // a est inférieur ou égal à 50
}

// OU

string carColor = "green";

string response = (carColor == "red") ?
    "You have a red car" :
    "You do not have a red car";
```



# Imbrication d'instructions if

Il est tout à fait possible de combiner plusieurs instructions Si-Sinon en une seule

## Exemple

```
if (a > 50)
{
    // a est plus grand que 50
}
else if (a > 10)
{
    // a est plus grand que 10 et inférieur ou égal à 50
}
else
{
    // a est inférieur ou égal à 10
}
```

# Instruction switch

L'instruction switch permet de simplifier l'écriture de tests multiples sur la même variable

## Exemple

```
switch (a)
{
    case 0:
        // a vaut 0
        break;

    case 1:
    case 2:
    case 3:
        // a vaut 1, 2 ou 3
        break;

    default:
        // a est différent de 0, 1, 2 et 3
        break;
}
```

Le *break* est obligatoire  
si du code est écrit dans  
le *case*



## Critères Spéciaux (Pattern Matching)

- Permet de déterminer si une valeur correspond à des cas prédéterminés
- Implémenté par l'expression `is` et une extension de l'instruction `switch`
- Expression `is`: étend les tests de type en ajoutant une déclaration de variable typée
- Expression `switch`: étend les cas du `switch` en validant le type. L'ordre des cas doit être contrôlé avec précaution.

## Critères Spéciaux: expression is

```
public static int DiceSum2(object[] values)
{
    int sum = 0;
    foreach(object item in values)
    {
        if (item is int val)
            sum += val;
        else if (item is object[] subList)
            sum += DiceSum2(subList);
    }
    return sum;
}
```

## Critères Spéciaux: expression switch

```
public static int DiceSum4(object[] values)
{
    int sum = 0;
    foreach (object item in values)
    {
        switch (item)
        {
            case 0:
                break;
            case int val:
                sum += val;
                break;
            case object[] subList when subList.Length >= 1:
                sum += DiceSum4(subList);
                break;
            case object[] subList:
                break;
            case null:
                break;
            default:
                throw new InvalidOperationException("unknown item type");
        }
    }
    return sum;
}
```

## Critères Spéciaux

- D'autres modes d'utilisation sont possibles:
- Réécriture plus concise des switch à retour de valeur
- Modèles de propriétés
  - switch sur un objet avec filtre sur une propriété
- Modèles de tuples
  - Passage d'un tuple à un switch
- Modèles positionnels
  - switch sur un objet par déconstruction en tuple
- Combinaison de modèles
  - Utilisation des mots-clés and et or, notations entre parenthèses

## Lesson 5: Instructions d'itérations

---

- Les différentes instructions d'itération
- Instruction while
- Instruction do
- Instruction for
- Instructions break et continue

# Les différentes instructions d'itération

`while`

Représente une itération conditionnée de type Tant Que

`do`

Réprésente une itération conditionnée de type Faire/Tant Que

`for`

Représente une itération dont le nombre d'itérations est connu



# Instruction while

- Boucle de type 0..N
- La condition est testée en début d'itération
- Le code d'itération est entre accolade s'il est défini de plusieurs instructions

## Exemple

```
double balance = 100D;  
double rate = 2.5D;  
double targetBalance = 1000D;  
int years = 0;  
while (balance <= targetBalance)  
{  
    balance *= (rate / 100) + 1;  
    years += 1;  
}
```

# Instruction do

- Boucle de type 1..N
- La condition est testée en fin d'itération
- L'instruction se termine par un point-virgule

## Exemple

```
string userInput = "";  
do  
{  
    userInput = GetUserInput();  
    if (userInput.Length < 5)  
    {  
        // Traitement si la chaîne fait moins de 5 caractères  
    }  
} while (userInput.Length < 5);
```

# Instruction for

La boucle est définie à l'aide de trois parties:

*for* ( initialisation; condition; fin d'itération) { ... }

- L'initialisation est exécutée une seule fois en début d'instruction
- La condition est testée à chaque itération (de type tant que)
- La fin d'itération est exécutée en fin de chaque itération

## Example

```
for (int i = 0; i < 10; i++)  
{  
    // code à exécuter, pouvant utiliser i  
}
```

# Instructions break et continue

## Break

```
while (oldNumbers.Length > count)
{
    if (oldNumbers[count] == 5)
    {
        break;
    }
    count++;
}
```

Permet de sortir de la boucle

## Continue

```
while (oldNumbers.Length > count)
{
    if (oldNumbers[count]%2==1)
    {
        count++; continue;
    }
    // code pour valeurs paires
    count++;
}
```

Permet d'interrompre la boucle et de reprendre l'exécution à l'itération suivante

- Définition
- Exemples
- Création

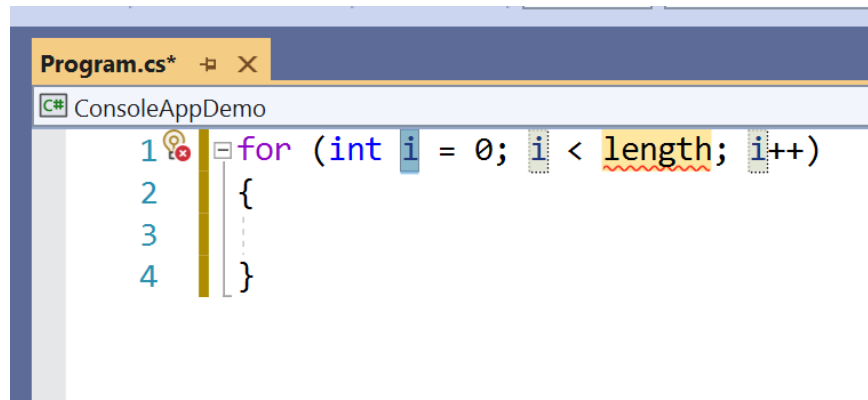
## Définition

---

- Les extraits de code (Code Snippets) sont des fragments de code prêt à l'emploi
- Peuvent être appelés par un mot-clé suivi d'une double tabulation ou par un clic-droit
- Certains extraits sont conçus pour encadrer un code existant
- Peuvent être paramétrés, la navigation entre chaque paramètre se faisant par tabulation
- Les extraits de code (\*.snippet) sont stockés dans  
C:\Program Files\Microsoft Visual Studio\2022\Community

# Exemples

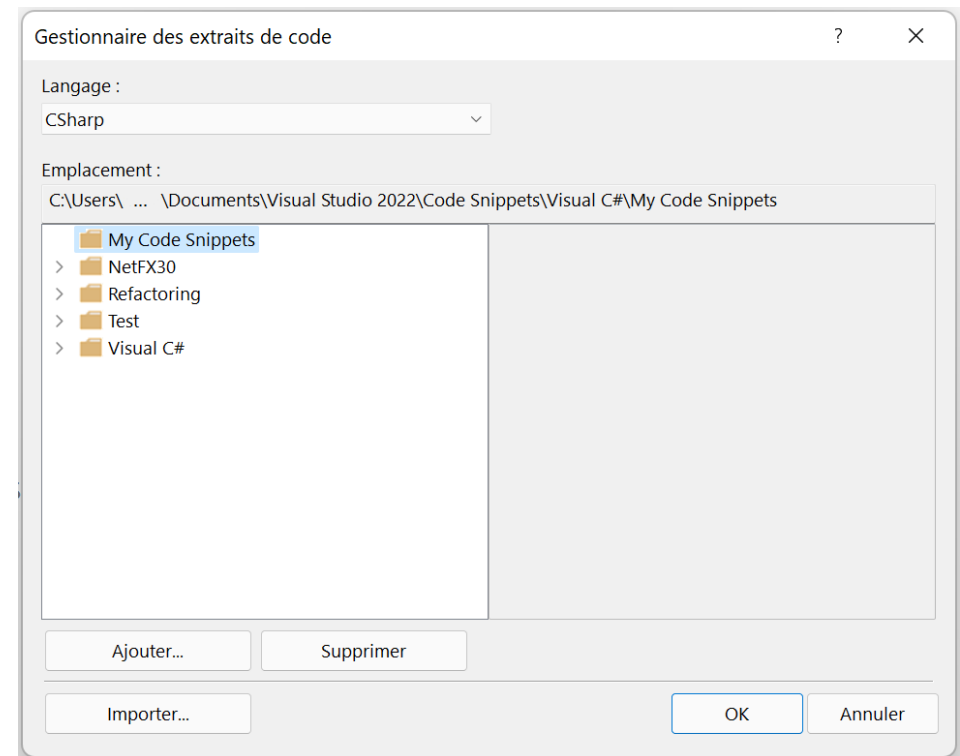
- Structures itératives: for, for, foreach, while, ...
- Structures conditionnelles: if, switch, ...



The screenshot shows a code editor window titled "Program.cs\*" with a sub-header "ConsoleAppDemo". The code is a C# for loop. Line 1: `1 for (int i = 0; i < length; i++)`. Line 2: `2 {`. Line 3: `3 ...`. Line 4: `4 }`. The variable `length` is underlined with a red squiggly line, indicating it is not defined. The variable `i` is highlighted in blue. The loop body is enclosed in curly braces.

```
1 for (int i = 0; i < length; i++)
2 {
3     ...
4 }
```

- Les extraits personnalisés sont stockés dans Mes Documents\Visual Studio 2022\Code Snippets
- Pour créer un nouvel extrait de code, il suffit de dupliquer un fichier .snippet existant et de l'importer dans Visual Studio via le Gestionnaire d'extrait de code





# Création (suite)

```
<?xml version="1.0" encoding="utf-8"?>
<CodeSnippets xmlns="http://schemas.microsoft.com/VisualStudio/2005/CodeSnippet">
  <CodeSnippet Format="1.0.0">
    <Header>
      <Title>for</Title>
      <Shortcut>for</Shortcut>
      <Description>Extrait de code pour une boucle 'for'</Description>
      <Author>Microsoft Corporation</Author>
      <SnippetTypes>
        <SnippetType>Expansion</SnippetType>
        <SnippetType>SurroundsWith</SnippetType>
      </SnippetTypes>
    </Header>
    <Snippet>
      <Declarations>
        <Literal>
          <ID>index</ID>
          <Default>i</Default>
          <ToolTip>Index</ToolTip>
        </Literal>
        <Literal>
          <ID>max</ID>
          <Default>length</Default>
          <ToolTip>Longueur max</ToolTip>
        </Literal>
      </Declarations>
      <Code Language="csharp"><! [CDATA [for (int $index$ 0; $index$ < $max$; $index$++)
{
    $selected$ $end$
}]]>
    </Code>
  </Snippet>
</CodeSnippet>
</CodeSnippets>
```

# Atelier 01

**Activité**  
**20 min**



- Exercice 1: Reconnaître des erreurs de syntaxe en C#
- Exercice 2: Le jeu du plus ou moins



## A retenir

### **Déclaration de Variables :**

"En C#, chaque variable a un type, définissant la nature des données qu'elle peut contenir. Bien déclarer une variable est la première étape pour garantir la fiabilité de votre code."

### **Affectation de Valeurs :**

"L'affectation correcte des valeurs aux variables est cruciale pour prévenir les erreurs inattendues à l'exécution."

### **Expression et Opérateurs :**

"Les opérateurs permettent de manipuler les valeurs des variables. Comprendre leur priorité et leur fonctionnement est essentiel pour éviter les erreurs logiques."

### **Tableaux :**

"Un tableau est une collection ordonnée. En C#, ils sont à taille fixe, et il est crucial de savoir les initialiser et y accéder correctement."

### **Instructions Conditionnelles :**

"Les structures if, else if et else sont le cœur du contrôle de flux. Elles dirigent l'exécution du code selon les conditions données."

### **Boucles et Itérations :**

"Les boucles comme for, while et foreach permettent d'exécuter du code de manière répétée. Elles sont fondamentales pour traiter des collections de données."

### **Optimisation du Code :**

"Bien que C# offre plusieurs façons d'aborder un problème, il est essentiel de choisir la structure la plus adaptée pour une exécution optimale et une lisibilité du code."

### **Extraits de Code avec Visual Studio 2022 :**

"Visual Studio 2022 est un allié précieux. Il facilite la rédaction, le débogage, et l'optimisation du code C#. Utilisez-le à son plein potentiel."

### **Éviter les Erreurs Courantes :**

"Comprendre ces concepts fondamentaux est la clé pour prévenir des erreurs courantes, comme les dépassements de tableau ou les boucles infinies."

### **Importance des Bonnes Pratiques :**

"Au-delà de la syntaxe, adopter de bonnes pratiques dès le départ permet de garantir un code propre, maintenable, et efficace."-

# **Chapitre 3 - Déclaration et appel de Méthodes**

# Plan de la formation

- 1 - Introduction à C# et .Net
- 2 - Structure de programmation C#
- 3 - **Déclaration et appel de méthodes**
- 4 - Gestion d'exceptions
- 5 - Lire et écrire dans des fichiers
- 6 - Création de nouveaux types de données
- 7 - Encapsulation de données et de méthodes
- 8 - Héritage de classes et implémentation d'interfaces
- 9 - Gestion de la durée de vie des objets et contrôle des ressources
- 10 - Encapsulation avancée
- 11 - Découplage de méthodes et gestion d'évènements
- 12 - Utilisation des collections et construction de types génériques
- 13 - Programmation asynchrone et personnalisation du code
- 14 - Utilisation de LINQ pour interroger des données
- 15 - Développement dirigé par les Tests

## Objectifs :

- Compréhension des Méthodes
- Maîtrise de la Syntaxe
- Gestion des Paramètres



# Commençons par échanger

Pourquoi utilise-t-on des méthodes en programmation ?

Quelle est la différence, selon vous, entre une fonction et une procédure ?

Pouvez-vous me donner un exemple d'une situation où il serait bénéfique de passer des paramètres à une méthode ?

Avez-vous déjà rencontré une situation où vous avez dû utiliser le même nom de méthode pour plusieurs tâches, mais avec des paramètres différents ?

Comment distinguez-vous le passage de paramètres "par valeur" et "par référence" dans vos expériences précédentes ? Et quelle a été votre préférence ?



## Vue d'ensemble

---

- Définir et appeler des méthodes
- Passage de paramètres

# Leçon 1: Définir et appeler des Méthodes

---

- Définition
- Création de méthodes
- Invocation de méthodes
- Surcharge
- Paramètres de type tableau
- Refactorisation
- Démonstration: Refactorisation



# Définition

- Tout code exécutable doit appartenir à une méthode
- Une application C# doit contenir au minimum une méthode (le point d'entrée du programme Main)
- Une méthode peut retourner un résultat (une fonction) ou simplement effectuer une action (une procédure)
- Une méthode ne peut pas exister en dehors d'une classe mais peut être une méthode d'instance ou statique

## Méthode d'instance

```
int count = 99;  
string strCount =  
    count.ToString();
```

## Méthode statique

```
string strCount = "99";  
count =  
    Convert.ToInt32(strCount);
```

# Création de méthodes

Une méthode contient:

- Une spécification (accès, type de retour et signature)
- Un corps (code)

Signature de méthode:

1

Nom (Pascal case)

2

paramètres (camel case)

La signature de la méthode doit être unique dans la portée où elle est définie

## Exemple

```
bool LockReport(string userName)
{
    bool success = false;
    // Traitement
    return success;
}
```

# Invocation de méthodes

Pour invoquer une méthode:

- Spécifier le nom de la méthode
- Fournir un argument pour chaque paramètre
- Gérer la valeur de retour

## Exemple de méthode

```
bool LockReport(string reportName, string userName)
{
    ...;
}
```

## Invocation

```
bool isReportLocked =
    LockReport("Medical Report", "Don Hall");
```

Note: Les arguments sont évalués de gauche à droite. Il s'agit du passage par position

# Surcharge

Une méthode surchargée:

- a le même nom qu'une méthode existante
- *devrait* réaliser le même type d'opération
- utilise des paramètres différents

Le compilateur invoque la méthode qui correspond au nombre et au type des paramètres fournis

## Exemple

```
void Deposit(decimal amount)
{
    _balance += amount;
}

void Deposit(int dollars, int cents)
{
    _balance += dollars + (cents / 100.0m);
}
```

La signature est  
différente pour  
chaque  
implémentation  
de la méthode

# Paramètres de type tableau

La surcharge n'est pas toujours appropriée pour traiter des listes d'arguments variables.

Un seul paramètre de type params est autorisé (le dernier).

## Exemple

```
int Add(params int[] data)
{
    int sum = 0;
    for (int i = 0; i < data.Length; i++)
    {
        sum += data[i];
    }
    return sum;
}
```

Note: Il n'est pas obligatoire de passer un argument de type tableau même si cela est également possible

## Method call

```
int sum = Add(99, 2, 55, -26);
```

# Refactorisation

La refactorisation permet d'améliorer la lisibilité et la modularité de son code en réalisant l'extraction de blocs de code répétés sous forme de méthodes.

Visual Studio et Visual Studio Code permettent de sélectionner le code à refactoriser et via la petite ampoule (Ctrl+;), d'Extraire la méthode tout en détectant les potentiels paramètres.

## Exemple de résultat de refactorisation

```
LogMessage(messageContents, filePath);  
...  
private void LogMessage(string messageContents, string filePath)  
{  
    ...  
}
```

- Refactorisation de méthodes

## Leçon 2: Passage de paramètres

---

- Paramètres optionnels
- Passage de paramètres nommés
- Paramètres de sortie
- Variable de sortie
- Tuples
- Discards



# Paramètres Optionnels

Les paramètres optionnels permettent de fournir des valeurs par défaut pour certains paramètres d'une méthode:

- Ils assurent une meilleure interopérabilité avec des langages supportant les paramètres facultatifs
- Ils offrent une alternative élégante à la problématique des surcharges multiples

## Exemple

```
void MyMethod(  
    int intData, float floatData, int moreIntData = 99)  
{  
    ...  
}
```

```
// Un argument pour chaque paramètre  
MyMethod(1, 123.45, 66);
```

```
// Seulement deux arguments pour les deux premiers paramètres  
MyMethod(100, 54.321);
```

# Passage de paramètres nommés

- Alternative au passage par position
- Les arguments sont fournis dans n'importe quel ordre
- Les valeurs manquantes sont fournies par les valeurs par défaut de la méthode
- Les deux types de passage sont cumulables à condition de commencer avec les arguments par position

## Exemple

```
void MyMethod(  
    int intData, float floatData = 101.1F, int moreIntData = 99)  
{  
    ...  
}
```

```
MyMethod(moreIntData: 100, intData: 10);  
// floatData recevra la valeur par défaut 101.1F
```

# Modification de paramètres

---

- Par défaut le passage d'un paramètre se fait **par valeur**:
  - Une copie de la variable est passée au paramètre
  - Les modifications apportées au paramètre ne sont pas reportées sur la variable
- Il est possible d'utiliser le mot-clé **ref** pour effectuer un passage **par référence**:
  - La variable est référencée par le paramètre
  - Toute modification du paramètre est appliquée à la variable
  - Le paramètre est considéré comme entrée-sortie
  - La variable doit être initialisée avant son utilisation

# Passage de paramètres par référence

```
static void Main(string[] args)
{
    int myInt = 1005;
    ChangeInput(myInt);
}
```

myInt vaut 1005



```
static void ChangeInput(int input)
{
    input = 29910;
}
```

```
static void Main(string[] args)
{
    int myInt = 1005;
    ChangeInput(ref myInt);
}
```

myInt vaut 29910



```
static void ChangeInput(ref int input)
{
    input = 29910;
}
```

# Paramètres de sortie

- Les paramètres de sortie permettent de retourner des valeurs par le biais des arguments en entrée par le mot-clé out
- Les paramètres ainsi désignés sont considérés comme des paramètres de sortie
- Il n'est pas obligatoire d'initialiser l'argument passé en paramètre (contrairement à l'utilisation du mot-clé ref)

```
void MyMethod(int first, double second, out int data)
{
    ...
    data = 99;
}
```



```
int value;
MyMethod(10, 101.1F, out value);
// value = 99
```

- L'utilisation du mot clé **out** nécessite de déclarer la variable au préalable.
- Une variable de sortie est déclarée lors de l'invocation de la méthode

```
int numericResult;  
if (int.TryParse(input, out numericResult))  
    WriteLine(numericResult);  
else  
    WriteLine("Could not parse input");
```

```
if (int.TryParse(input, out int numericResult))  
    WriteLine(numericResult);  
else  
    WriteLine("Could not parse input");
```



- C# permet de développer ses propres types (structures, classes) mais cela peut s'avérer lourd pour une problématique ponctuelle
- Les Tuples permettent de gérer un jeu de valeurs ensemble, en particulier en retour d'une méthode

```
var letters = ("a", "b");  
Console.WriteLine(letters.Item1);  
...  
var alphabetStart = (Alpha: "a", Beta: "b");  
Console.WriteLine(alphabetStart.Alpha);  
...  
(string Alpha, string Beta) namedLetters = ("a", "b");  
Console.WriteLine(namedLetters.Alpha);
```

## Tuples (suite)

```
private static (int Max, int Min) Range(int[] numbers)
{
    int min = int.MaxValue;
    int max = int.MinValue;
    foreach(var n in numbers)
    {
        min = (n < min) ? n : min;
        max = (n > max) ? n : max;
    }
    return (max, min);
}
```

```
var range = Range(numbers);
// ou bien
(int max, int min) = Range(numbers)
```



Deconstruction du tuple



- Utilisation du symbole `_`
- Permet d'ignorer un membre d'un tuple, ou une variable de sortie

```
private static (string, double, int, int, int, int)
GetCityDetails(string name, int year1, int year2)
{
    /* Récupère nom, superficie, première année, population
    pour année 1, deuxième année, population pour année 2 */
    return (name, area, year1, pop1, year2, pop2);
}
...
var (_, _, _, p1, _, p2)=GetCityDetails("Paris",1960,2010);
```

```
if (int.TryParse(s, out _))
{
    Console.WriteLine("La chaine représente un entier");
}
```

# Atelier 01

**Activité**  
**20 min**



- Exercice 1: Modularisation d'un code existant



- **Modularité** : Les méthodes permettent de découper un programme en blocs logiques, facilitant sa lecture, sa maintenance et sa réutilisabilité.
- **Signature Unique** : Chaque méthode a une signature unique, définie par son nom et la liste des types de ses paramètres, permettant une surcharge efficace.
- **Valeurs de Retour** : Une méthode peut retourner une valeur. Si elle ne retourne rien, elle est définie comme "void" en C#.
- **Paramètres** : Les méthodes peuvent prendre des paramètres qui permettent de transmettre des informations à la méthode lors de son appel.
- **Par Valeur vs Par Référence** : En C#, les paramètres peuvent être passés "par valeur" (où une copie est transmise) ou "par référence" (où l'adresse réelle en mémoire est transmise).
- **Paramètres Par Défaut** : C# permet la définition de valeurs par défaut pour les paramètres, rendant certains arguments optionnels lors de l'appel de la méthode.
- **Réutilisabilité** : En encapsulant des fonctionnalités spécifiques dans des méthodes, on peut facilement réutiliser ces fonctions dans d'autres parties du programme ou dans d'autres programmes.
- **Surcharge de Méthodes** : C# permet la définition de plusieurs méthodes avec le même nom mais des listes de paramètres différentes, offrant une flexibilité dans la façon dont une méthode peut être utilisée.
- **Clarté et Lisibilité** : Nommer correctement les méthodes et utiliser des commentaires appropriés rend le code plus compréhensible pour les autres développeurs.
- **Gestion d'Erreurs** : Grâce au mécanisme d'exceptions de C#, les méthodes peuvent signaler des erreurs d'exécution de manière robuste et contrôlée.

# Chapitre 4 - Gestion d'exceptions

# Plan de la formation

- 1 - Introduction à C# et .Net
- 2 - Structure de programmation C#
- 3 - Déclaration et appel de méthodes
- 4 - Gestion d'exceptions**
- 5 - Lire et écrire dans des fichiers
- 6 - Création de nouveaux types de données
- 7 - Encapsulation de données et de méthodes
- 8 - Héritage de classes et implémentation d'interfaces
- 9 - Gestion de la durée de vie des objets et contrôle des ressources
- 10 - Encapsulation avancée
- 11 - Découplage de méthodes et gestion d'évènements
- 12 - Utilisation des collections et construction de types génériques
- 13 - Programmation asynchrone et personnalisation du code
- 14 - Utilisation de LINQ pour interroger des données
- 15 - Développement dirigé par les Tests

## Objectifs :

- Comprendre l'importance et le rôle de la gestion des exceptions dans le développement logiciel.
- Identifier et gérer les erreurs courantes qui peuvent survenir lors de l'exécution d'un programme.
- Utiliser efficacement les outils fournis par C# pour la gestion des exceptions.
- Créer du code robuste et résilient qui peut gérer les scénarios d'erreur et reprendre l'exécution.



# Commençons par échanger

"Pouvez-vous partager une expérience où une erreur non gérée dans une application vous a causé des problèmes, soit en tant que développeur, soit en tant qu'utilisateur ? Qu'est-ce qui aurait pu être fait différemment ?"

"Comment définiriez-vous une 'exception' en programmation à partir de vos connaissances antérieures ? Et pourquoi pensez-vous que la gestion de ces exceptions est importante ?"

"Avez-vous déjà rencontré une situation où vous auriez souhaité avoir mis en place une gestion d'exceptions mais ne l'avez pas fait ? Quelles ont été les conséquences ?"

"Selon vous, est-il préférable de prévoir toutes les exceptions possibles avant qu'elles ne se produisent ou de réagir aux exceptions lorsqu'elles surviennent ? Pourquoi ?"

"Comment pensez-vous qu'un utilisateur moyen réagit lorsqu'il rencontre une erreur non gérée dans une application ? Quel impact cela pourrait-il avoir sur la perception qu'il a de cette application ?"

## Vue d'ensemble

---

- Gestion des exceptions
- Soulever des exceptions

# Leçon 1: Gestion des Exceptions

- Définition
- Le bloc try/catch
- Propriétés des exceptions
- Le bloc finally et le catch conditionné
- Les mots clés checked et unchecked



- Une exception est le résultat d'une erreur d'exécution rencontrée par l'application
- Si l'exception n'est pas gérée par le code l'ayant rencontrée, celle-ci remonte la pile des appels jusqu'à être capturée ou affichée à l'utilisateur
- Il est possible, dans certains Framework de développement, de mettre en place une procédure de gestion d'exceptions non gérées
- Les exceptions sont classées à travers une hiérarchie d'exception de la plus générique (Exception) à la plus spécifique (par exemple FileNotFoundException ou encore DivideByZeroException)

# Le bloc try/catch

```
try  
{  
    // bloc try  
}  
catch (DivideByZeroException ex)  
{  
    // bloc catch, accède au détail de l'exception  
    // par la variable ex  
}  
catch (Exception ex)  
{  
    // bloc catch générique  
}
```

Le code que l'on veut exécuter et qui peut déclencher une erreur

Logique de traitement de l'exception  
DivideByZeroException

Logique de traitement des autres exceptions

- Il est possible d'imbriquer les blocs try/catch
- Toujours ordonner les blocs catch du plus spécifique au plus générique

# Propriétés des exceptions

Quelques propriétés:

Message

Source

StackTrace

TargetSite

InnerException

HelpLink

Data

```
try
{
    // Try block.
}
catch (DivideByZeroException ex)
{
    Console.WriteLine(ex.Message);
}
```

Affichage du  
message associé à  
cette exception



# Le bloc finally et le catch conditionné

- Le bloc finally est toujours exécuté
- Un catch soumis à condition permet d'affiner la gestion d'exception

```
try
{
    OpenFile("MyFile");
    WriteToFile(...);
}
catch (IOException ex) when (myCondition)
{
    MessageBox.Show(ex.Message);
}
finally
{
    CloseFile("MyFile"); // Clos le fichier
}
```



Gère l'exception  
IOException seulement si  
myCondition est vrai



Sera toujours exécuté

# Les mots clés checked et unchecked

- Par défaut, les applications C# ne vérifient pas les dépassement de capacités sur les numériques entiers. Vous pouvez changer ce comportement dans les paramètres de votre projet.
- Vous pouvez également contrôler ce comportement localement dans votre code grâce aux mots clés checked et unchecked

```
checked
{
    int x = ...;
    int y = ...;
    int z = ...;
    ...
}
```

Blocs checked et unchecked

```
unchecked
{
    int x = ...;
    int y = ...;
    int z = ...;
    ...
}
```

```
...
int z = checked(x * y);
...
```

Opérateurs checked et unchecked

```
...
int z = unchecked(x * y);
...
```

## Leçon 2: Soulever des Exceptions

---

- Création d'objets de type Exception
- Lancer une exception
- Bonnes pratiques

# Création d'objets de type Exception

System.Exception

System.SystemException

System.FormatException

System.ArgumentException

System.NotSupportedException

+ beaucoup d'autres

```
catch (Exception e)
{
    FormatException ex =
        new FormatException("Argument has the wrong format", e);
}
```

# Lancer une exception

Mot clé throw

`throw [exception object];`

Déclaration de l'objet Exception

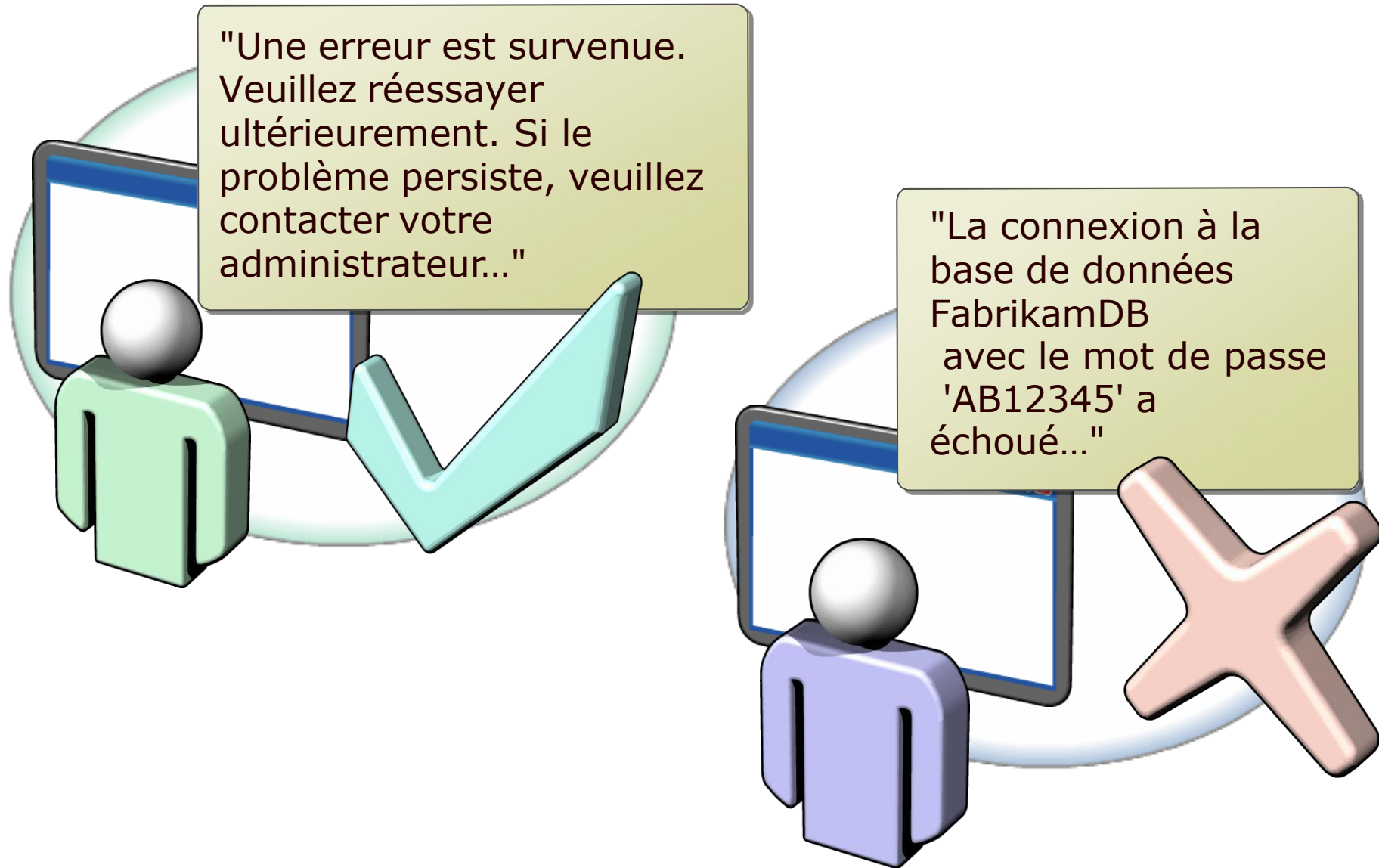
## Exemple

```
public int GetIntegerRoot(int operand)
{
    double root = Math.Sqrt(operand);
    if (root != (int)root)
    {
        throw new ArgumentException("No integer root found.");
    }
    return (int)root;
}
```

```
name = value ?? throw new ArgumentNullException("Name must not be null");
```



# Bonnes pratiques



# Atelier 01

**Activité**  
**20 min**



- Exercice 1: Les dépassements de capacité en C#
- Exercice 2: Levée d'exception
- Exercice 3: Capture d'exception



## A retenir

**Proactivité** : "La gestion des exceptions ne consiste pas simplement à réagir aux erreurs, mais à les anticiper pour garantir une exécution fluide du programme.«

**Différenciation** : "Il est crucial de différencier une 'erreur' d'une 'exception'. Une erreur est souvent irrécupérable, tandis qu'une exception est une condition que vous pouvez prévoir et gérer.«

**Structuration** : "Utilisez les blocs try, catch, et finally pour structurer votre code autour de la gestion des exceptions de manière propre et efficace.«

**Spécificité** : "Il est préférable de gérer les exceptions spécifiques avant les exceptions générales afin d'offrir des retours d'information plus précis et pertinents.«

**Personnalisation** : "Soulever des exceptions personnalisées peut aider à clarifier l'intention du code et à gérer des situations spécifiques à votre application.«

**Messages informatifs** : "Lorsque vous soulevez une exception, fournissez toujours un message clair et informatif. Cela facilite le débogage et améliore l'expérience utilisateur.«

**Responsabilité** : "Ne vous fiez pas uniquement aux exceptions fournies par le framework ; sentez-vous libre de soulever les vôtres pour gérer les cas spécifiques à votre application.«

**Évolutivité** : "Bien que le code puisse fonctionner sans gestion d'exceptions, une bonne gestion des exceptions rend votre application plus robuste, évolutives, et amicale pour l'utilisateur.«

**Documentation** : "Documentez toujours les exceptions que votre méthode peut soulever, afin d'informer les autres développeurs et de permettre une meilleure intégration.«

**Principe de précaution** : "Il vaut mieux prévoir et gérer une exception même si elle semble peu probable. Cela montre que vous avez pris en compte toutes les éventualités, ce qui renforce la robustesse de votre application."

# **Chapitre 5 - Lire et écrire dans des fichiers**

# Plan de la formation

- 1 - Introduction à C# et .Net
- 2 - Structure de programmation C#
- 3 - Déclaration et appel de méthodes
- 4 - Gestion d'exceptions
- 5 - Lire et écrire dans des fichiers**
- 6 - Création de nouveaux types de données
- 7 - Encapsulation de données et de méthodes
- 8 - Héritage de classes et implémentation d'interfaces
- 9 - Gestion de la durée de vie des objets et contrôle des ressources
- 10 - Encapsulation avancée
- 11 - Découplage de méthodes et gestion d'évènements
- 12 - Utilisation des collections et construction de types génériques
- 13 - Programmation asynchrone et personnalisation du code
- 14 - Utilisation de LINQ pour interroger des données
- 15 - Développement dirigé par les Tests

## Objectifs :

- Comprendre les Fondamentaux : Maîtriser les bases de la lecture et de l'écriture de fichiers en C#.
- Gérer Efficacement les Fichiers : Apprendre à gérer de manière sécurisée et efficace l'accès aux fichiers et les opérations d'entrée/sortie.
- Appliquer les Bonnes Pratiques : S'assurer que les fichiers sont traités de manière optimale, tout en gérant les éventuelles erreurs ou exceptions.
- Maîtriser les Formats : Se familiariser avec la lecture et l'écriture de différents types de fichiers, des simples fichiers texte aux formats complexes comme le XML.



# Commençons par échanger

"Pouvez-vous partager une expérience où vous avez dû lire ou écrire des données dans un fichier, que ce soit dans un autre langage ou contexte ? Quels étaient les défis auxquels vous avez été confrontés ?"

"Pourquoi pensez-vous que la capacité de lire et d'écrire dans des fichiers est une compétence si importante pour un développeur ? Dans quelles situations l'avez-vous trouvé utile dans le passé ?"

"Quels types de formats de fichiers avez-vous déjà rencontrés ou utilisés dans vos projets précédents ? Y a-t-il des formats que vous trouvez particulièrement pratiques ou complexes ?"

"Quelle importance accordez-vous à la sécurité lors de la manipulation des fichiers ? Avez-vous des anecdotes ou des préoccupations spécifiques à ce sujet ?"

"Quelle est votre approche lorsque vous rencontrez une erreur ou une exception lors de la lecture ou de l'écriture dans un fichier ? Avez-vous des exemples d'erreurs courantes que vous avez rencontrées et comment les avez-vous résolues ?"

## Vue d'ensemble

---

- Accéder au système de fichier
- Lecture et écriture dans des fichiers en utilisant les flux

# Leçon 1: Accéder au système de fichier

---

- Manipulation des fichiers
- Lecture et écriture de fichiers
- Manipulation des répertoires
- Manipulation des chemins



# Manipulation des fichiers

- L'espace de noms System.IO contient de nombreuses classes qui simplifient les interactions avec le système de fichier, notamment les classes File et FileInfo
- La classe File est une classe statique offrant des méthodes simples
- La classe FileInfo est une classe permettant d'obtenir des informations détaillées sur un fichier en particulier

## Méthodes de la classe File:

Copy()

Create()

Delete()

Exists()

## Méthode de la classe FileInfo:

CopyTo()

Delete()

Length

Open()

# Lecture et écriture de fichiers

## Lecture en bloc depuis un fichier

```
string filePath = "myFile.txt";  
...  
byte[] data = File.ReadAllBytes(filePath); // données binaires  
string[] lines = File.ReadAllLines(filePath); // lignes  
string data = File.ReadAllText(filePath); // tout le texte
```

## Ecriture en bloc vers un fichier

```
string filePath = "myFile.txt";  
...  
string[] fileLines = {"Line 1", "Line 2", "Line 3"};  
File.AppendAllLines(filePath, fileLines); // ajout de lignes  
File.WriteAllLines(filePath, fileLines); // nouveau fichier  
...  
string fileContents = "I am writing this text to a file ...";  
File.AppendAllText(filePath, fileContents); // ajout de texte  
File.WriteAllText(filePath, fileContents); // nouveau fichier
```

# Manipulation des répertoires

Les classes `Directory` et `DirectoryInfo` de `System.IO` permettent de gérer les répertoires du système de fichier

## Classe `Directory`

```
string dirPath = @"C:\Users\Student\MyDirectory";  
...  
Directory.CreateDirectory(dirPath);  
Directory.Delete(dirPath);  
string[] dirs = Directory.GetDirectories(dirPath);  
string[] files = Directory.GetFiles(dirPath);
```

## Classe `DirectoryInfo`

```
string dirPath = @"C:\Users\Student\MyDirectory";  
DirectoryInfo dir = new DirectoryInfo(dirPath);  
...  
bool exists = dir.Exists;  
DirectoryInfo[] dirs = dir.GetDirectories();  
FileInfo[] files = dir.GetFiles();  
string fullName = dir.FullName;
```

# Manipulation des chemins

La classe Path de System.IO permet de créer et gérer les noms d'accès au fichier

La classe Path inclut notamment les méthodes:

GetDirectoryName()

GetExtension()

GetFileName()

GetFileNameWithoutExtension()

GetRandomFileName()

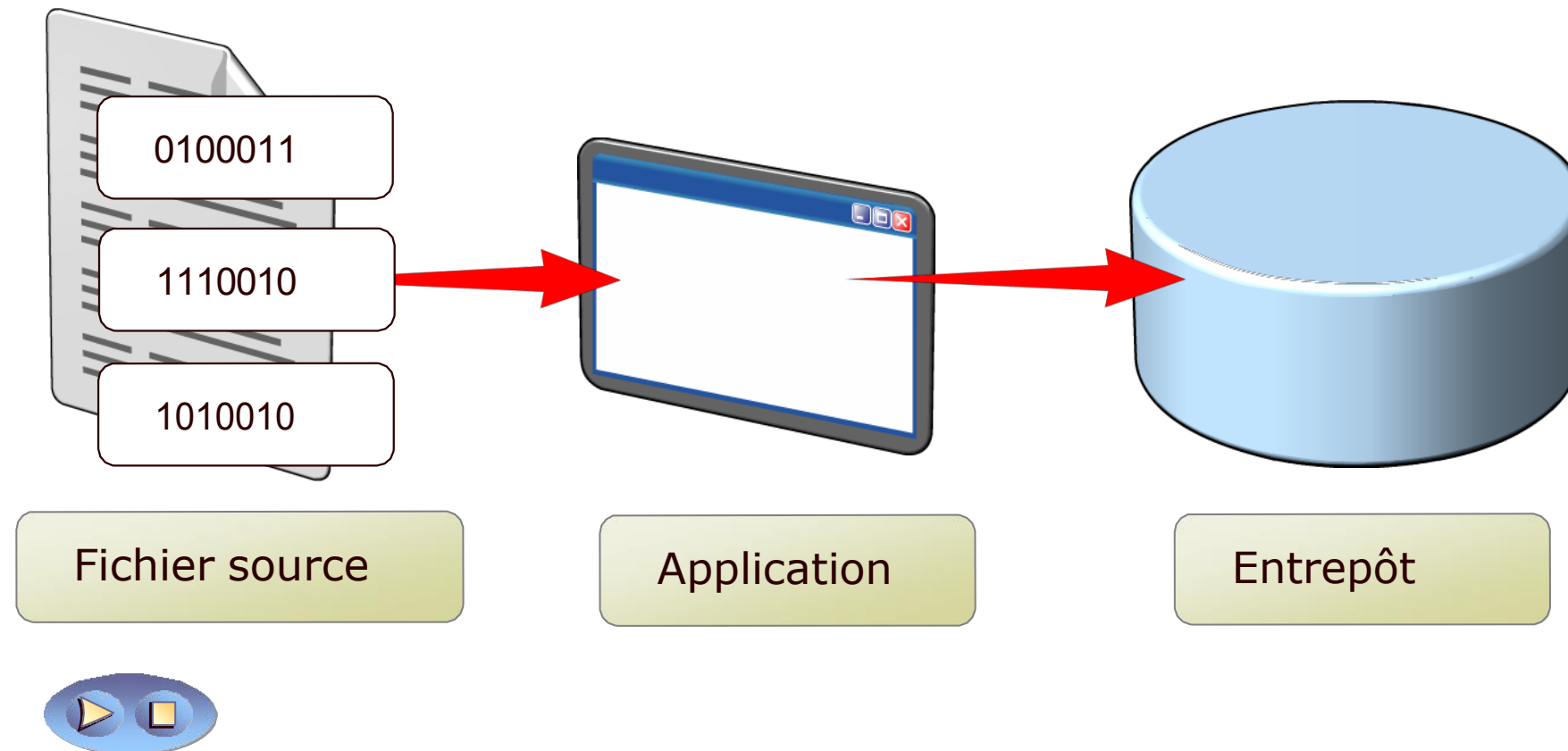
## Leçon 2: Lecture et écriture dans des fichiers en utilisant les flux

---

- Notion de flux
- Lecture et écriture de données binaires
- Lecture et écriture des types de données primitifs
- Lecture et écriture de texte

# Notion de flux

Un flux est un mécanisme qui permet de manipuler des données sous forme de morceaux maniables



# Lecture et écriture de données binaires

```
FileStream sourceFile = new FileStream(sourceFilePath,...);
BinaryReader reader = new BinaryReader(sourceFile);
int position = 0;
int length = (int)reader.BaseStream.Length;
byte[] dataCollection = new byte[length];
int returnedByte;
while ((returnedByte = reader.Read()) != -1)
{
    dataCollection[position] = (byte)returnedByte;
    position += sizeof(byte);
}
reader.Close();
sourceFile.Close();
```

Classe BinaryReader



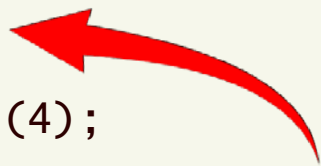
```
byte[] dataCollection = { 1, 4, 6, 7, 12, 33, 26, 98, 82, 101 };
FileStream destFile = new FileStream(destinationFilePath,...);
BinaryWriter writer = new BinaryWriter(destFile);
foreach (byte data in dataCollection)
{
    writer.Write(data);
}
writer.Close();
destFile.Close();
```

Classe BinaryWriter



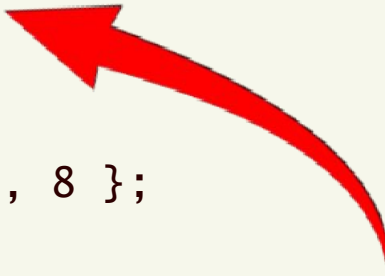
# Lecture et écriture des types de données primitifs

```
bool boolValue = reader.ReadBoolean();  
byte byteValue = reader.ReadByte();  
byte[] byteArrayValue = reader.ReadBytes(4);  
char charValue = reader.ReadChar();  
...
```



Classe BinaryReader

```
bool boolValue = true;  
writer.Write(boolValue);  
  
byte byteValue = 1;  
writer.Write(byteValue);  
  
byte[] byteArrayValue = { 1, 4, 6, 8 };  
writer.Write(byteArrayValue);  
  
char charValue = 'a';  
writer.Write(charValue);  
...
```



Classe BinaryWriter



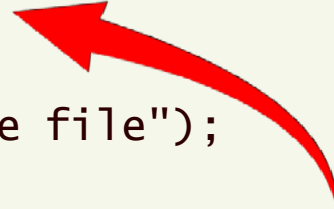
# Lecture et écriture de texte

```
FileStream sourceFile = new FileStream(sourceFilePath,...);  
StreamReader reader = new StreamReader(sourceFile);  
StringBuilder fileContents = new StringBuilder();  
while (reader.Peek() != -1)  
{  
    fileContents.Append((char)reader.Read());  
}  
string data = fileContents.ToString();  
reader.Close();  
sourceFile.Close();
```



Classe StreamReader

```
FileStream destFile = new FileStream(destFilePath,...);  
StreamWriter writer = new StreamWriter(destFile);  
  
writer.WriteLine("Hello, this will be written to the file");  
  
writer.Close();  
destFile.Close();
```



Classe StreamWriter

# Atelier 01

**Activité**  
**20 min**



- Exercice 1: Persistance d'informations dans un fichier
- Exercice 2: Persistance du meilleur score
- Exercice 3: Lecture du meilleur score



## A retenir

1. **Accès Essentiel** : Accéder au système de fichiers est une compétence fondamentale pour tout développeur, permettant de lire, écrire et manipuler des données sur le disque.
2. **Flux en .NET** : En C#, les flux (Stream) sont des abstractions essentielles pour traiter les séquences d'octets, que ce soit pour les fichiers, les réseaux ou d'autres sources.
3. **Manipulation Prudente** : Manipuler directement le système de fichiers requiert une attention particulière pour éviter la perte de données, les erreurs d'accès et les problèmes de concurrence.
4. **StreamReader & StreamWriter** : Pour lire et écrire du texte dans des fichiers, les classes StreamReader et StreamWriter sont couramment utilisées et fournissent des méthodes pratiques pour ces opérations.
5. **Gestion des Chemins** : Utilisez les méthodes fournies par la classe Path pour gérer et manipuler les chemins de fichiers de manière sécurisée et compatible.
6. **Fermer les Flux** : Il est crucial de toujours fermer les flux après leur utilisation pour libérer les ressources associées et éviter des comportements indésirables.
7. **Exceptions & Fichiers** : La manipulation des fichiers peut générer diverses exceptions (par exemple, FileNotFoundException). Il est essentiel de les gérer correctement pour assurer la robustesse de votre application.
8. **Accès Concurrent** : Si plusieurs opérations tentent d'accéder au même fichier simultanément, cela peut causer des conflits. Il est important d'être conscient de cela et de gérer l'accès concurrentiel approprié.
9. **Encodage** : Soyez toujours attentif à l'encodage utilisé lors de la lecture ou de l'écriture de texte dans des fichiers, surtout lorsque vous travaillez avec des données internationalisées.
10. **Pratiques de Sécurité** : Lors de l'accès au système de fichiers, gardez toujours à l'esprit les meilleures pratiques de sécurité, en évitant d'exposer des informations sensibles et en utilisant des permissions appropriées.

# **Chapitre 6 - Création de nouveaux types de données**

# Plan de la formation

- 1 - Introduction à C# et .Net
- 2 - Structure de programmation C#
- 3 - Déclaration et appel de méthodes
- 4 - Gestion d'exceptions
- 5 - Lire et écrire dans des fichiers
- 6 - **Création de nouveaux types de données**
- 7 - Encapsulation de données et de méthodes
- 8 - Héritage de classes et implémentation d'interfaces
- 9 - Gestion de la durée de vie des objets et collections
- 10 - Encapsulation avancée
- 11 - Découplage de méthodes et gestion d'événements
- 12 - Utilisation des collections et construction de types génériques
- 13 - Programmation asynchrone et personnalisation du code
- 14 - Utilisation de LINQ pour interroger des données
- 15 - Développement dirigé par les Tests

## Objectifs :

- Compréhension Approfondie
- Polyvalence dans le Code
- Principes d'Organisation



# Commençons par échanger

"Pouvez-vous me donner un exemple d'une situation où vous auriez besoin de définir un ensemble précis de valeurs possibles pour une variable, comme les jours de la semaine ou les couleurs d'un feu de circulation?"

"Avez-vous déjà travaillé avec des concepts d'orienté objet dans un autre langage de programmation ? Si oui, comment définiriez-vous une 'classe' et un 'objet' d'après votre expérience?"

"Dans quels scénarios pourriez-vous imaginer qu'une structure serait préférable à une classe, étant donné que les deux peuvent être utilisées pour définir des types?"

Types Référence vs. Types Valeur : "Quelle est, d'après vous, la différence principale entre stocker une donnée en tant que référence et la stocker en tant que valeur directe?"

Bibliothèques de Classes : "Avez-vous déjà créé ou utilisé des bibliothèques de fonctions ou de classes dans d'autres projets ou langages de programmation ? Comment avez-vous organisé ou intégré ces bibliothèques ?"

- Création et utilisation d'énumérations
- Création et utilisation de classes
- Création et utilisation de structures
- Comparaison des types références et types valeurs
- Gestion des références aux bibliothèques de classes sous Visual Studio 2022

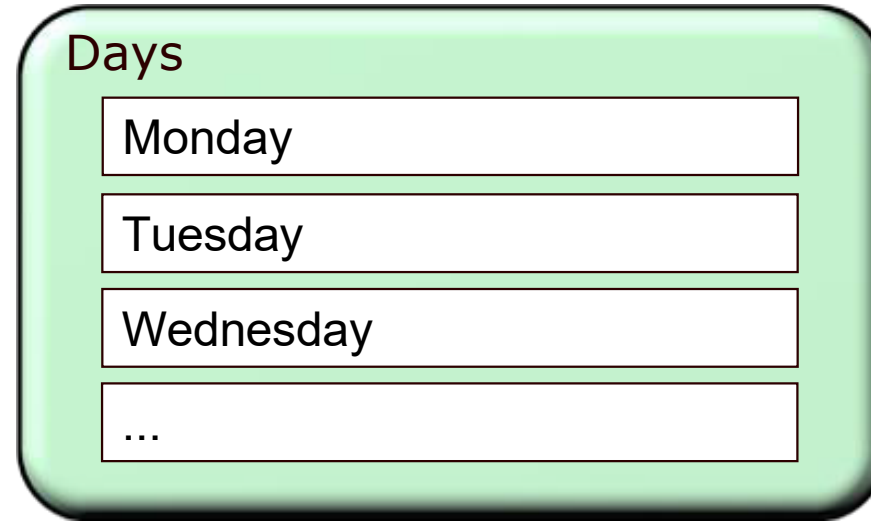
# Leçon 1: Création et utilisation d'énumérations

---

- Définition
- Création
- Utilisation



Une énumération est un ensemble de constantes nommées



## Avantages:

- Le code est plus facile à maintenir car vous affectez des valeurs connues et bien définies aux variables
- Le code est plus facile à lire car vous affectez des valeurs facilement identifiables aux variables
- Les code est plus facile à écrire car l'IntelliSense vous affiche la liste des valeurs utilisables

- Une énumération se définit à l'aide du mot clé enum

```
enum Name : DataType { Value1, Value2 . . . };
```

- Il est possible de déclarer des énumérations dans des espaces de noms ou dans des classes, mais pas dans des méthodes
- Sauf indication explicite, les valeurs d'énumérations débutent à 0

```
namespace Fabrikam.CustomEnumerations  
{  
    enum Days : int { Monday = 1, Tuesday = 2, Wednesday = 3 };  
    ...  
}
```

Une variable de type énumération est déclarée comme les autres

```
[EnumType] variableName = [EnumValue]
```

Seules les valeurs définies peuvent être utilisées

```
enum Days : int
{
    Monday = 1, Tuesday = 2, Wednesday = 3, Thursday = 4,
    Friday = 5, Saturday = 6, Sunday = 7
};
static void Main(string[] args)
{
    Days myDayOff = Days.Sunday;
}
```

*Ou myDayOff = (Days) 7*

Certaines opérations simples sont possibles

```
for (Days dayOfWeek = Days.Monday; dayOfWeek <= Days.Sunday;
    dayOfWeek++)
{
    ...
}
```

## Leçon 2: Création et utilisation de classes

---

- Définition
- Ajout de membres
- Constructeurs et initialisation
- Constructeur par défaut
- Création d'objets
- Accès aux membres
- Classes et méthodes partielles

# Définition

- Une classe est un plan à partir duquel sont créés les objets
- Une classe définit les caractéristiques des objets

```
class House  
{  
  ...  
}
```

Définition de classe avec le mot  
clé class



- Un objet est une instance de classe

myHouse

yourHouse

# Ajout de membres

Les membres définissent les données et comportements de la classe

```
public class Residence  
{
```

```
    public ResidenceType type;  
    public int numberOfBedrooms;  
    public bool hasGarage;  
    public bool hasGarden;
```

Champs

```
    public int CalculateSalePrice()  
    {  
        // Code de calcul du prix de  
        // la résidence.  
    }
```

```
    public int CalculateRebuildingCost()  
    {  
        // Code de calcul du coût de rénovation  
        // de la résidence.  
    }
```

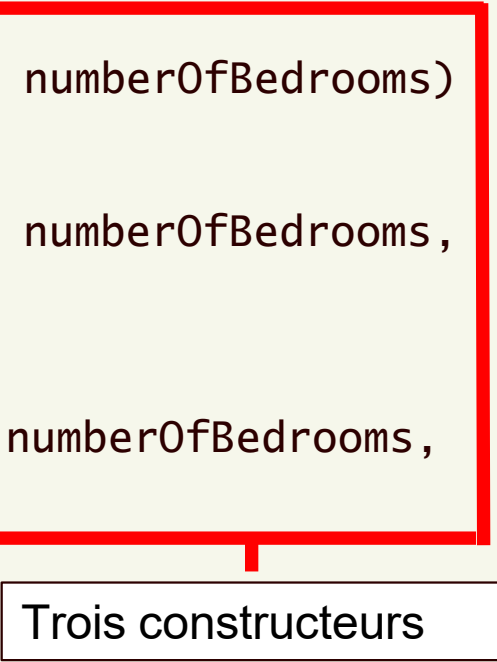
```
}
```

Méthodes

# Constructeurs et initialisation

Un constructeur est une méthode spéciale exécutée implicitement par le CLR

```
public class Residence
{
    public Residence(ResidenceType type, int numberOfBedrooms)
    {
    }
    public Residence(ResidenceType type, int numberOfBedrooms,
        bool hasGarage)
    {
    }
    public Residence(ResidenceType type, int numberOfBedrooms,
        bool hasGarage, bool hasGarden)
    {
    }
}
```



Trois constructeurs

Sans initialisation explicite, les champs seront affectés de la valeur par défaut de leur type

## Constructeur par défaut

---

- Constructeur vide (sans paramètre)
- Généré par le compilateur
- Uniquement créé si aucun constructeur n'est déclaré explicitement
- Assure qu'il soit toujours possible d'instancier les objets



# Création d'objets

- Les objets sont initialement non assignés (*null*)
- Pour utiliser une classe, il faut d'abord créer une instance de la classe
- La création d'une instance implique l'utilisation de l'opérateur new

L'opérateur new permet:

- L'allocation mémoire de l'objet par le CLR
- L'invocation du constructeur pour initialiser l'objet

```
// Appartement avec 2 chambres
Residence myFlat = new Residence(ResidenceType.Flat, 2);

// Maison avec 3 chambres et garage
Residence myHouse = new Residence(ResidenceType.House, 3, true);

// Bungalow avec 2 chambres, garage et jardin
Residence myBungalow =
    new Residence(ResidenceType.Bungalow, 2, true, true);
```

# Accès aux membres

L'accès aux membres d'un objet se fait par le nom de l'instance suivi d'un point

*InstanceName.MemberName*

## Exemple

```
// Création d'une maison avec 3 chambres
Residence myHouse = new Residence(ResidenceType.House, 3);

// La maison a un jardin
myHouse.hasGarden = true;

// Calcul du prix de vente
int salePrice = myHouse.CalculateSalePrice();

// Calcul du prix de rénovation
int rebuildCost = myHouse.CalculateRebuildingCost();
```

# Classes et méthodes partielles

- Permet de répartir la définition de la classe sur plusieurs fichiers
- La classe et les méthodes doivent être préfixées du mot clé partial

```
public partial class Residence
{
    partial void SaleResidence(string name);
}
```

Définition de la méthode  
SaleResidence

```
public partial class Residence
{
    partial void SaleResidence(string name)
    {
        //Logic goes here.
    }
}
```

Implémentation de la  
méthode SaleResidence

- Compilés en une seule et unique entité
- Généralement mises en oeuvre par des outils de generation de code

## Leçon 3: Création et utilisation de structures

---

- Définition
- Création et utilisation
- Initialisation

# Définition

Les structures sont des types valeurs

System.Byte  
System.Int16  
System.Int32

System.Int64  
System.Single  
System.Double

System.Decimal  
System.Boolean  
System.Char

- Les données d'une structure sont stockées sur la pile
- Les structures sont utilisées pour des modèles avec peu de données
- Les structures peuvent contenir des champs et des méthodes

```
int x = 99;  
string xAsString = x.ToString();
```

# Création et utilisation

- La déclaration d'une structure utilise le mot clé struct
- Les membres sont définis comme dans une classe

```
struct Currency
{
    public string currencyCode;
    public string currencySymbol;
    public int fractionDigits
}
```

- Les structures sont automatiquement allouées en mémoire sur la pile

```
Currency unitedStatesCurrency;

unitedStatesCurrency.currencyCode = "USD";
unitedStatesCurrency.currencySymbol = "$";
unitedStatesCurrency.fractionDigits = 2;
```

- Il n'est pas nécessaire d'utiliser l'opérateur new pour créer une instance de structure

# Initialisation

- Il est possible de définir des constructeurs
- Il y a toujours un constructeur vide
- Un constructeur doit initialiser tous les champs

```
struct Currency
{
    public string currencyCode;
    public string currencySymbol;
    public int fractionDigits
    public Currency(string code, string symbol)
    {
        this.currencyCode = code;
        this.currencySymbol = symbol;
        this.fractionDigits = 2;
    }
};

...
Currency unitedKindgdomCurrency = new Currency("GBP", "£");
```

## Leçon 4: Comparaison des types références et types valeurs

---

- Comparer types références et types valeurs
- Passage de paramètres par référence
- Boxing et Unboxing
- Types nullables
- Retour de fonctions par référence
- Déconstruction



# Comparer types références et types valeurs

Types références (référence sur la pile, données dans le tas)

- Une variable de type référence contient la référence vers un objet en mémoire
- Si deux variables contiennent la même référence, elles pointent vers le même objet
- Si l'objet est modifié, les deux variables permettront de voir cette modification

Types valeurs (valeur sur la pile)

- En copiant un type valeur, les variables ne représentent pas la même donnée
- La modification d'une variable ne sera pas visible par l'autre

## Passage de paramètres par référence (rappel)

```
static void Main(string[] args)
{
    int myInt = 1005;
    ChangeInput(myInt);
}
```

myInt vaut 1005



```
static void ChangeInput(int input)
{
    input = 29910;
}
```

```
static void Main(string[] args)
{
    int myInt = 1005;
    ChangeInput(ref myInt);
}
```

myInt vaut 29910



```
static void ChangeInput(ref int input)
{
    input = 29910;
}
```

# Boxing et Unboxing

## Boxing (type valeur vers type référence)

1. Le CLR alloue la mémoire dans le tas
2. Le CLR copie la valeur de la pile vers le tas

```
Currency myCurrency = new Currency(); // Type valeur  
object o = myCurrency; // Référence sur la valeur
```

## Unboxing (type référence vers type valeur)

1. Le CLR vérifie le type de la valeur référencée par la variable
2. Si cela correspond, le CLR copie la valeur depuis le tas

```
Currency myCurrency = new Currency(...);  
object o = myCurrency; // boxing  
...  
Currency anotherCurrency = (Currency)o; // ok à la compilation
```

# Types nullables

- Il est possible d'affecter la valeur *null* à une variable de type référence pour indiquer qu'elle n'est pas initialisée
- La valeur *null* est en soi une référence et il n'existe pas de correspondance pour indiquer qu'une variable de type valeur n'est pas initialisée

```
Currency myCurrency = null; // illégal
```

- Le point d'interrogation (?) permet d'indiquer un type valeur nullable

```
Currency? myCurrency = null; // légal  
if (myCurrency == null){...}
```

- Les types nullables exposent les propriétés HasValue et Value

```
Currency? myCurrency = null;  
if (myCurrency.HasValue)  
{  
    Console.WriteLine(myCurrency.Value);  
}
```

# Retour de fonctions par référence

```
struct MyBigStruct{
    public int Id;
    public string Name;
}

static MyBigStruct[] _items;
static ref MyBigStruct FindItem(int id){
    for (int i = 0; i < _items.Length; i++)
    {
        if (_items[i].Id == id)
        {
            return ref _items[i];
        }
    }
    throw new Exception("Item not found");
}

static void Test(){
    ref MyBigStruct item = ref FindItem(42);
    item.Name = "test";
    FindItem(42) = new MyBigStruct();
}
```

Utile dans des  
scénarios  
d'optimisation

# Déconstruction

- A ne pas confondre avec la notion de Destructeur
- Permet de déconstruire un type (classe ou structure) vers un Tuple

```
public class Point
{
    ...
    public void Deconstruct(out double x, out double y)
    {
        x = this.X;
        y = this.Y;
    }
}
```

```
var p = new Point(3.14, 2.71);
(double X, double Y) = p;
...
(double horizontalDistance, double verticalDistance) = p;
```

- Rôle des références
- Ajout d'une référence

- Permet de spécifier au programme où trouver les définitions des types utilisés
- Peut-être précoce (liaison dès le développement) ou tardive (liaison dynamique lors de l'exécution)
- A ne pas confondre avec la clause *using* qui permet de définir des alias pour simplifier la syntaxe



- Types de références:
  - Assembly (.NET Framework uniquement)
  - Projet
  - Projet partagé
  - COM
- Dans Visual Studio, via l'explorateur de solutions:
  - Clic droit sur le projet puis Ajouter...
  - Clic droit sur le noeud Références (.NET Framework) ou Dépendances (.NET)

# Atelier 01

**Activité**  
**20 min**



- Exercice 1: Création d'une classe
- Exercice 2: Mise en oeuvre d'une bibliothèque de classes



## A retenir

1. **Énumérations** : Les énumérations offrent un moyen efficace de représenter un ensemble fixe de valeurs connexes, augmentant la lisibilité et réduisant les erreurs dans le code.
2. **Polyvalence des Classes** : Les classes sont au cœur de la programmation orientée objet en C# et permettent de modéliser des objets du monde réel avec des propriétés et des comportements.
3. **Structures vs Classes** : Contrairement aux classes, les structures sont des types valeur. Elles sont plus légères, idéales pour des objets simples et de petite taille, mais ne supportent pas l'héritage.
4. **Choix de Type** : Le choix entre type référence (comme les classes) et type valeur (comme les structures) doit être basé sur les besoins de l'application et la manière dont les données sont utilisées.
5. **Avantage des Types Valeur** : Les types valeur, comme les structures, sont généralement plus rapides à accéder et ne nécessitent pas de gestion de la mémoire via le garbage collector, contrairement aux types référence.
6. **Héritage** : Seules les classes permettent l'héritage, offrant des mécanismes pour créer de nouveaux types basés sur des types existants.
7. **Bibliothèques de Classes** : Les bibliothèques de classes facilitent la modularité et la réutilisabilité, permettant de regrouper des types et des fonctionnalités couramment utilisés.
8. **Référencement dans Visual Studio** : Visual Studio 2022 facilite la gestion des références aux bibliothèques de classes, assurant que les bonnes versions des bibliothèques sont utilisées.
9. **Cohérence** : L'utilisation cohérente des types, que ce soit des énumérations, des classes ou des structures, améliore la lisibilité du code et réduit les chances d'erreurs.
10. **Adaptabilité** : Grâce à la richesse des types de données en C#, les développeurs ont la flexibilité de choisir le type le plus approprié pour chaque situation, assurant ainsi une performance et une maintenance optimales du code.

# **Chapitre 7 - Encapsulation de données et de méthodes**

# Plan de la formation

- 1 - Introduction à C# et .Net
- 2 - Structure de programmation C#
- 3 - Déclaration et appel de méthodes
- 4 - Gestion d'exceptions
- 5 - Lire et écrire dans des fichiers
- 6 - Création de nouveaux types de données
- 7 - **Encapsulation de données et de méthodes**
- 8 - Héritage de classes et implémentation d'interfaces
- 9 - Gestion de la durée de vie des objets
- 10 - Encapsulation avancée
- 11 - Découplage de méthodes et gestion
- 12 - Utilisation des collections et constr
- 13 - Programmation asynchrone et pers
- 14 - Utilisation de LINQ pour interroger d
- 15 - Développement dirigé par les Tests

## Objectifs :

- Comprendre le concept d'encapsulation et son importance dans la programmation orientée objet (POO).
- Maîtriser les mécanismes offerts par C# pour implémenter l'encapsulation de manière efficace.
- Savoir comment protéger l'intégrité des données d'une classe tout en fournissant un accès contrôlé à ces données.
- Évaluer l'importance de l'encapsulation pour assurer la modularité, la maintenance, et la sécurité du code.



# Commençons par échanger

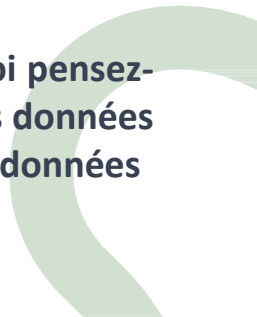
"Pouvez-vous partager une expérience où vous avez senti le besoin de protéger ou de cacher certaines informations à l'intérieur d'une classe ou d'un objet ? Comment avez-vous géré cela ?"

"Quels sont, selon vous, les principaux avantages de l'utilisation des principes de la programmation orientée objet, en particulier l'encapsulation, dans la conception de logiciels ?"

"Si vous avez déjà travaillé avec d'autres langages orientés objet, comment l'encapsulation y est-elle traitée par rapport à ce que vous savez de C# ?"

"Dans quelles situations pourrait-il être essentiel de limiter l'accès direct aux données d'un objet ? Pouvez-vous donner un exemple concret ?"

"Selon votre compréhension actuelle, pourquoi pensez-vous qu'il est crucial d'associer étroitement les données d'un objet et les méthodes qui opèrent sur ces données ?"



- Contrôler la visibilité des membres
- Partager méthodes et données

# Leçon 1: Contrôler la visibilité des membres

---

- Définition de l'encapsulation
- Membres publics et membres privés
- Types publics et types internes



# Définition de l'encapsulation

## Définition

- Principe essentiel de la programmation orientée-objet
- Masque les données et traitements internes
- Assure des opérations publics bien définies

## Avantages

- Code externe plus simple et plus consistant
- Evolution interne du code possible sans modification du code externe

# Membres publics et membres privés

private

Niveau d'accès le plus strict

```
class Sales
{
    private double monthlyProfit;
    private void SetMonthlyProfit(double monthlyProfit)
    {
        this.monthlyProfit = monthlyProfit;
    }
}
```

Uniquement  
accessibles  
dans la classe  
Sales

public

Niveau d'accès le moins strict

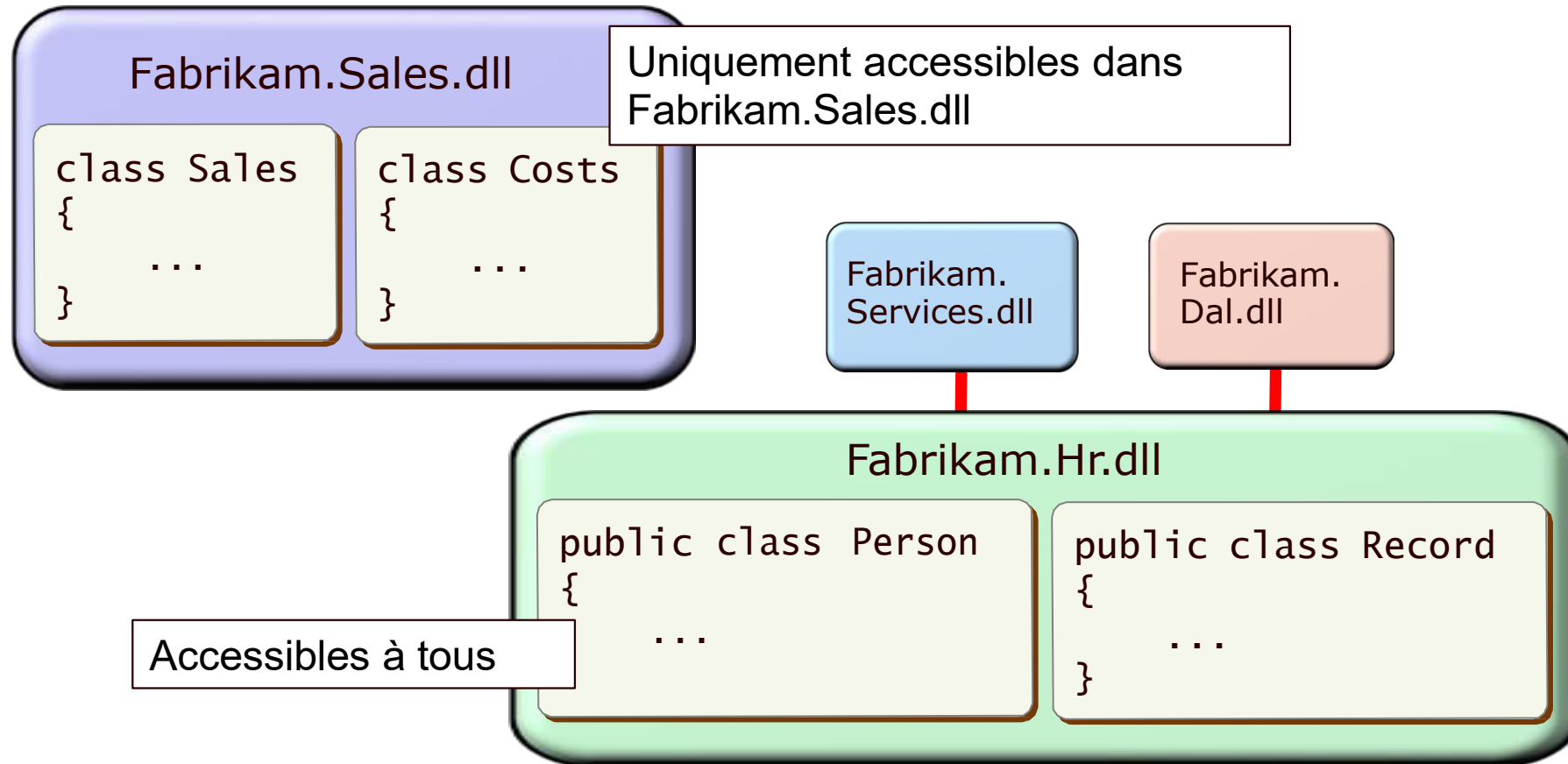
```
class Sales
{
    private double monthlyProfit;
    public void SetMonthlyProfit(double monthlyProfit)
    {
        this.monthlyProfit = monthlyProfit;
    }
}
```

Accessible  
depuis  
n'importe quel  
type

# Types publics et types internes

Par défaut les types sont Internes

- Internal: n'est accessible qu'aux autres types du même assembly
- Public: est accessible par des types d'autres assemblies



## Leçon 2: Partager méthodes et données

---

- Création et utilisation de champs statiques
- Création et utilisation de méthodes statiques
- Création de types statiques et constructeur statique
- Création et utilisation de méthodes d'extension
- Import statique

# Création et utilisation de champs statiques

- Les champs d'instance contiennent des données spécifiques à une instance d'un type donné
- Les champs statiques contiennent des données spécifiques à un type

```
class Sales
{
    public static double salesTaxPercentage = 20;
}
```

- Un champ statique est accédé en le préfixant du nom du type dans lequel il est défini

```
Sales.salesTaxPercentage = 32;
```

# Création et utilisation de méthodes statiques

- Les méthodes statiques offrent des fonctionnalités pertinentes pour un type et non pour une instance en particulier
- Les méthodes statiques ne peuvent accéder qu'aux champs statiques de la classe

```
class Sales
{
    public static double GetMonthlySalesTax(double monthProfit)
    {
        ...
    }
}
```

- Il n'est pas nécessaire de créer une instance pour pouvoir utiliser des méthodes statiques

```
double monthlySalesTax = Sales.GetMonthlySalesTax(34267);
```

# Création de types statiques et constructeur statique

- Un type statique est déclaré en utilisant le modificateur static
- Les types statiques ne peuvent contenir que des membres statiques (sorte de boîte à outils, ex. System.Math)

```
static class Sales
{
}
```

- Les types statiques peuvent avoir un constructeur
- Les constructeurs statiques sont invoqués implicitement par le CLR

```
static class Sales
{
    static Sales()
    {
    }
}
```

Défini avec le modificateur static

Défini sans paramètres

Défini sans modificateur d'accès

# Création et utilisation de méthodes d'extension

Ajouter des méthodes à une classe dans les cas suivants:

- Aucun accès au code source de la classe
- Impossible de recompiler le code de la classe
- Impossible d'hériter de la classe

```
namespace Fabrikam.Extens
{
    static class IntExtension
    {
        internal static int NextRand(this int seed, int
            maxValue)
        {
            Random randomNumberGenerator = new Random(seed);
            return randomNumberGenerator.Next(maxValue);
        }
    }
}
```

Diagramme illustrant la structure du code :

- Classe statique** : pointe vers `static class IntExtension`
- Méthode statique** : pointe vers `internal static int NextRand`
- Type à étendre** : pointe vers `this int seed`

```
using Fabrikam.Extensions;
...
int i = 8;
int j = i.NextRand(20);
```

Aucune référence à la classe `IntExtension`



# Import statique

En plus de pouvoir définir des alias d'espace de noms, la clause *using* peut également permettre de se passer de préfixer les méthodes statiques

```
// Définition d'alias  
using MyAlias = Society.Package.Tools;  
...  
MyAlias.MyType t = new MyAlias.MyType();
```

```
// Import statique  
using static System.Math;  
...  
double value = Sqrt(data);
```

# Atelier 01

**Activité**  
**20 min**



- Exercice 1: Encapsulation de données
- Exercice 2: Les membres statiques
- Exercice 3: Méthodes d'extension



## A retenir

1. **Définition fondamentale** : L'encapsulation est un pilier de la programmation orientée objet, permettant de regrouper les données d'un objet et les méthodes qui opèrent sur ces données.
2. **Contrôle d'accès** : En C#, les modificateurs d'accès (comme `private`, `public`, `protected` et `internal`) déterminent comment les membres d'une classe peuvent être accessibles depuis d'autres classes ou assemblages.
3. **Protection des données** : L'encapsulation aide à protéger l'intégrité des données en empêchant un accès direct ou non autorisé à certaines parties d'un objet.
4. **Simplicité et flexibilité** : En cachant les détails d'implémentation, l'encapsulation rend le code plus lisible et facilite la maintenance.
5. **Principe de moindre privilège** : Par défaut, il est recommandé de rendre les membres `private` sauf si une bonne raison justifie une autre visibilité.
6. **Réutilisabilité** : Grâce à l'encapsulation, les classes peuvent être plus facilement réutilisées dans d'autres contextes sans craindre que leur fonctionnement interne ne soit perturbé.
7. **Interdépendance** : En partageant méthodes et données à l'intérieur d'une classe, nous pouvons garantir que toutes les opérations sur ces données sont cohérentes et sécurisées.
8. **Évolution du code** : L'encapsulation permet d'apporter des modifications à l'implémentation d'une classe sans affecter les parties du code qui utilisent cette classe.
9. **Conception robuste** : Une bonne encapsulation contribue à créer des systèmes robustes et évolutifs en isolant les différentes parties du code les unes des autres.
10. **Cohésion et couplage** : L'encapsulation favorise une forte cohésion à l'intérieur des classes et un faible couplage entre elles, ce qui est une pratique souhaitable en conception logicielle.

# **Chapitre 8 - Héritage de classes et implémentation d'interfaces**

# Plan de la formation

- 1 - Introduction à C# et .Net
- 2 - Structure de programmation C#
- 3 - Déclaration et appel de méthodes
- 4 - Gestion d'exceptions
- 5 - Lire et écrire dans des fichiers
- 6 - Création de nouveaux types de données
- 7 - Encapsulation de données et de méthodes
- 8 - Héritage de classes et implémentation d'interfaces**
- 9 - Gestion de la durée de vie des objets et contrôle des ressources
- 10 - Encapsulation avancée
- 11 - Découplage de méthodes et gestion d'événements
- 12 - Utilisation des collections et constructions génériques
- 13 - Programmation asynchrone et personnalisation
- 14 - Utilisation de LINQ pour interroger des données
- 15 - Développement dirigé par les Tests

## Objectifs :

- Comprendre l'héritage
- Maîtriser les mécanismes d'héritage
- Découvrir les interfaces



# Commençons par échanger

Quelle est votre expérience avec la notion d'héritage en programmation?

Pouvez-vous citer un exemple, dans la vie réelle ou dans la programmation, où la relation "est un" est évidente ?

Avez-vous déjà utilisé des interfaces dans votre code? Si oui, dans quel contexte?

Quels avantages voyez-vous à définir des comportements à l'aide d'interfaces plutôt que d'hériter directement d'une classe?

Pouvez-vous décrire une situation où le polymorphisme pourrait être utile dans une application ?



- Utiliser l'héritage pour définir de nouveaux types références
- Définir et implémenter des interfaces
- Définir des classes abstraites

# Leçon 1: Utiliser l'héritage pour définir de nouveaux types références

- Définition
- Hiérarchie d'héritage dans le .NET Framework
- Transtypage et héritage
- Polymorphisme et masquage
- Comprendre le polymorphisme
- Appel de méthode et de constructeur de la classe de base
- Classes et méthodes scellées
- Récapitulatif des niveaux d'accessibilité



# Définition

L'héritage permet de créer de nouveaux types à partir de types existants:

- Par exemple, les classes Manager et ManualWorker peuvent hériter de la classe Employee
- Les champs et les méthodes de la classe Employee sont hérités par Manager et ManualWorker
- Manager et ManualWorker peuvent définir leur propre champs et comportements
- protected: public pour les classes héritantes, privées pour les autres

```
// Classe de base
class Employee
{
    public string empNum;
    public string empName;
    public void DoWork() { ... }
}

// Classes héritantes
class Manager : Employee
{
    public void DoManagementWork()
    { ... }
}

class ManualWorker : Employee
{
    public void DoManualWork()
    { ... }
}
```



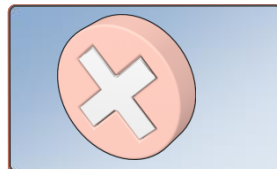
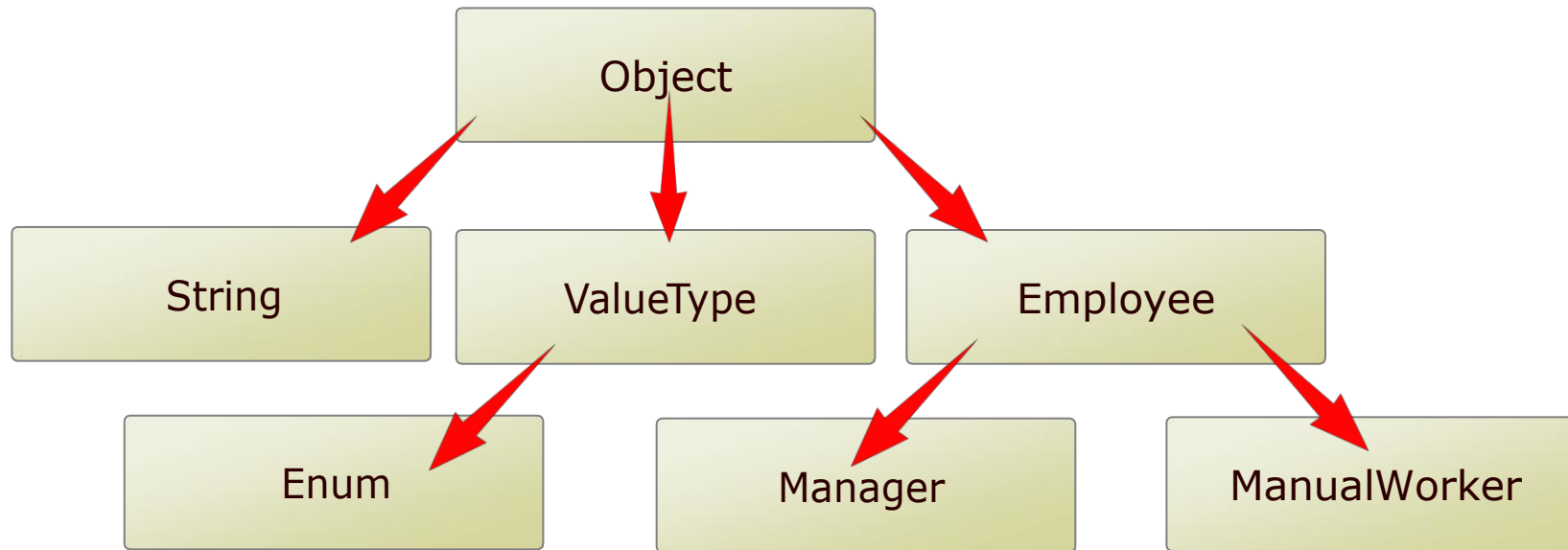
C# ne permet d'hériter que d'une seule classe



# Hiérarchie d'héritage dans le .NET Framework

Tous les types héritent directement ou non de la classe System.Object

- Inutile de spécifier :Object dans la définition
- Structures et Enumérations héritent de ValueType



Il n'est pas possible de définir sa propre hiérarchie d'héritage par rapport aux types valeurs



# Transtypage et héritage

Le typage fort de C# vous empêche de référencer un type dans une variable d'un autre type ...



```
Manager myManager = new  
Manager(...);  
ManualWorker myWorker =  
myManager;
```

... mais vous pouvez affectez une référence à un type différent plus haut dans la hiérarchie d'héritage



```
Manager myManager = new  
Manager(...);  
Employee myEmployee =  
myManager;
```

Utiliser les opérateurs `is` et `as` pour assurer la sécurité des conversions inverses



```
Manager myManagerAgain =  
myEmployee as Manager;  
// OU  
if(myEmployee is Manager)  
    Manager myManagerAgain =  
    (Manager) myEmployee;
```



# Polymorphisme et masquage



Polymorphisme: Remplacer ou étendre des fonctionnalités de la classe de base avec un comportement sémantiquement équivalent (intentionnel)



```
class Employee
{
    protected virtual void
        DoWork()
    { ... }
}

class Manager : Employee
{
    protected override void
        DoWork()
    { ... }
}
```

```
class Employee
{
    protected void DoWork()
    { ... }
}

class Manager : Employee
{
    public new void DoWork()
    { ... }
}
```



Masquage: Remplacer des fonctionnalités de la classe de base par un nouveau comportement (potentiellement une erreur)



# Comprendre le polymorphisme

Dans une hiérarchie d'héritage, la même instruction peut appeler différentes implémentations de la même méthode



```
Employee myEmployee;  
Manager myManager =  
    new Manager(...);  
ManualWorker myWorker =  
    new ManualWorker(...);  
  
myEmployee = myManager;  
Console.WriteLine(  
    myEmployee.GetTypeName());  
  
myEmployee = myWorker;  
Console.WriteLine(  
    myEmployee.GetTypeName());
```



```
class Employee  
{  
    public virtual string  
    GetTypeName()  
    {  
        return "This is an  
        Employee";  
    }  
}  
  
class Manager : Employee  
{  
    public override string  
    GetTypeName()  
    {  
        return "This is a Manager";  
    }  
}  
  
class ManualWorker : Employee  
{  
    // Pas de polymorphisme  
}
```

# Appel de méthode et de constructeur de la classe de base

Utilisation du mot clé base

```
class Employee
{
    protected string empName;
    public Employee(string name)
    { this.empName = name; }
}

class Manager : Employee
{
    protected string empGrade;
    public Manager(string name,
                    string grade)
        : base(name)
    {
        this.empGrade = grade;
    }
}
```

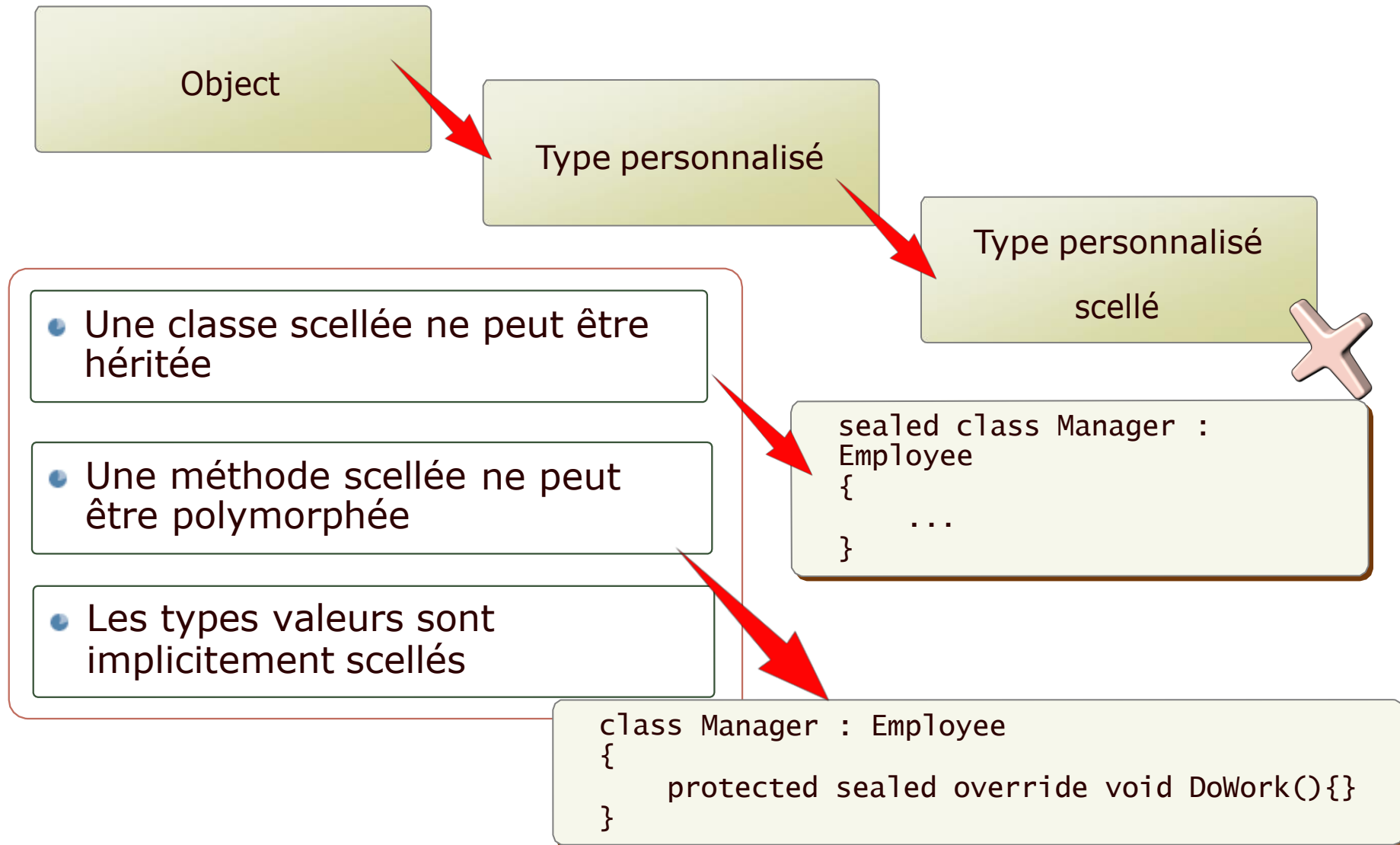
```
class Employee
{
    protected virtual void
        DoWork()
    { ... }
}

class Manager : Employee
{
    protected override void
        DoWork()
    {
        ...
        base.DoWork();
    }
}
```

Les constructeurs invoquent  
automatiquement le constructeur  
par défaut de la classe mère sauf  
indication explicite



# Classes et méthodes scellées



# Récapitulatif des niveaux d'accessibilité

---

- Une classe peut être
  - public
  - internal (par défaut)
- Les membres d'une classe peuvent être
  - public (partout)
  - protected (par héritage)
  - internal (même assembly)
  - private (par défaut – uniquement dans la classe)
  - protected internal (même assembly ou par héritage)
  - private protected (par héritage d'une classe du même assembly)



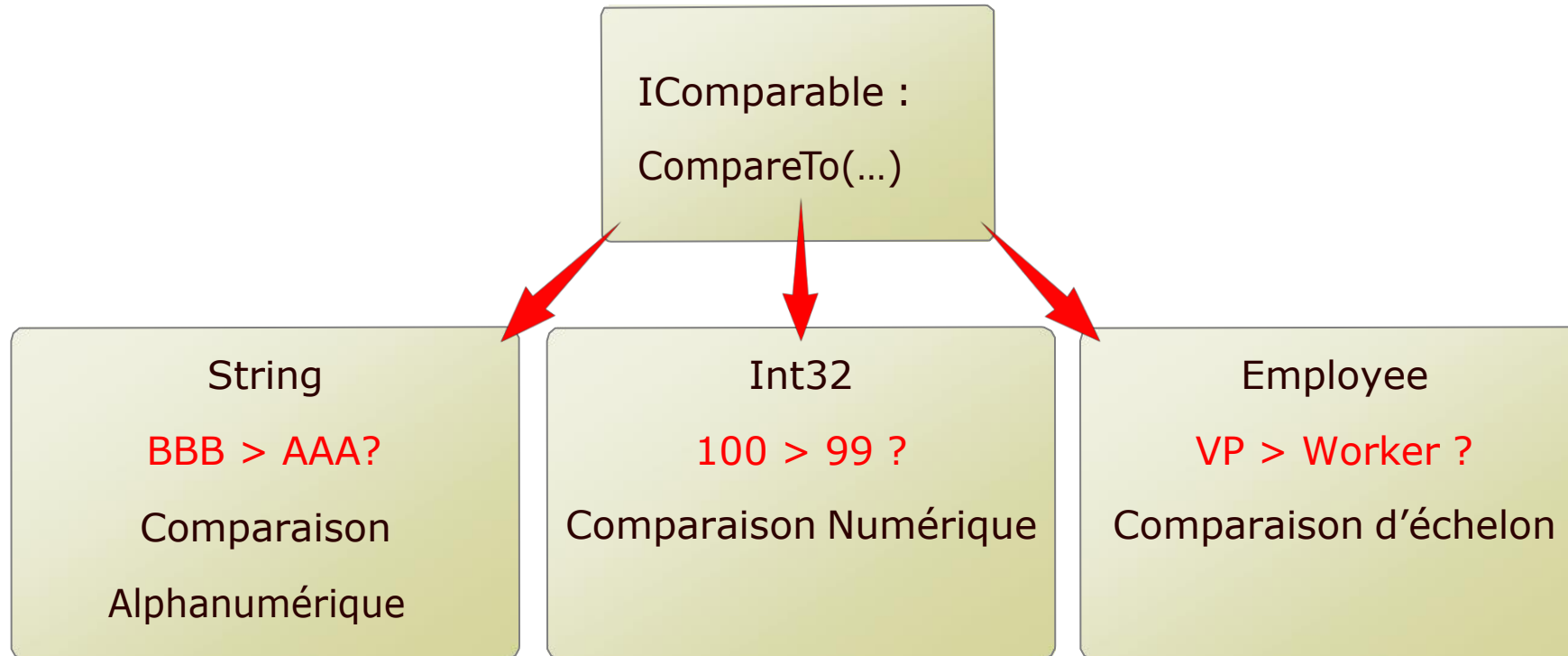
## Leçon 2: Définir et implémenter des interfaces

---

- Définition
- Création et implémentation d'interface
- Transtypage et interface
- Implémentation explicite et implicite
- Méthodes d'interface par défaut

# Définition

- Un contrat qui spécifie les méthodes que doit exposer la classe qui l'implémente (impose un certain comportement)
- Indépendante de l'implémentation
- Facilite l'injection de dépendance / inversion de contrôle



# Création et implémentation d'interface

Utilisation du mot clé interface

```
interface ICalculator
{
    double Add();
    double Subtract();
    double Multiply();
    double Divide();
}
```

Modificateur d'accès facultatif  
(si défini, plus délicat à gérer)

```
class Calculator : ICalculator
{
    public double Add(){}
    public double Subtract(){}
    public double Multiply(){}
    public double Divide(){}
}
```

Même syntaxe que l'héritage

Attention à l'accessibilité des méthodes

 C# autorise l'implémentation de plusieurs interfaces



# Transtypage et interface

Vous pouvez définir une classe qui implémente l'interface ...

```
class Calculator : ICalculator  
{  
    ...  
}
```

... et utiliser l'interface pour référencer une instance de cette classe

```
Calculator myCalculator =  
    new Calculator();  
ICalculator iMyCalculator =  
    myCalculator;
```

You pouvez également utiliser les interfaces comme paramètres ...

```
public int PerformAnalysis(ICalculator c)  
{  
    ...  
}
```

... , vérifier qu'une instance implémente une interface, ou encore qu'une variable de type interface contient le bon objet

```
Calculator calc =  
    iMyCalculator as Calculator;  
bool isCalc =  
    iMyCalculator is Calculator;
```

```
ICalculator iCalc =  
    myCalculator as ICalculator;
```



# Implémentation explicite et implicite

## Implémentation implicite:

- Scénarios simples
- Peut engendrer des ambiguïtés
- Méthodes visibles pour l'objet en tant qu'instance de classe

```
class Calculator : ICalculator
{
    double Add()
    {
    }

    double Subtract()
    {
    }

    ...
}
```

## Implémentation explicite:

- Scénarios complexes
- Evite les ambiguïtés
- Solution recommandée pour les interfaces
- Impose de référencer l'objet en tant qu'interface pour voir les méthodes

```
class Calculator : ICalculator
{
    double ICalculator.Add()
    {
    }

    double ICalculator.Subtract()
    {
    }

    ...
}
```



## Méthodes d'interface par défaut

---

- Proposer une implémentation de méthode dans une interface
- Faciliter l'ajout de méthodes à une interface sans rupture avec le code existant
  - La nouvelle méthode proposera une implémentation par défaut en cas d'absence dans la classe
- Peut référencer tout membre de l'interface

## Leçon 3: Définir des classes abstraites

---

- Classes Abstraites
- Méthodes abstraites

# Classes Abstraites

- Contiennent du code comme toute classe
- Doivent être héritées
- Ne peuvent pas être instanciées
- Peuvent contenir des champs, des méthodes...
- Utilise le mot clé abstract

```
abstract class SalariedEmployee : Employee, ISalaried
{
    void ISalaried.PaySalary()
    {
        Console.WriteLine("Pay salary: {0}", currentSalary);
        // Logique métier
    }
    int currentSalary;
}

class ManualWorker : SalariedEmployee, ISalaried
{
    ...
}

class Manager : SalariedEmployee, ISalaried
{
    ...
}
```



# Méthodes abstraites

Ajoutées à une classe abstraite

```
abstract class SalariedEmployee : Employee, ISalariedEmployee
{
    abstract void PayBonus();
    ...
}
```

Définie avec le modificateur abstract

Pas de corps de méthode



Les méthodes abstraites sont utiles lors du développement d'une classe abstraite qui implémente une interface ou dépend d'une méthode dont l'implémentation par défaut ne serait pas appropriée



Les classes qui héritent d'une classe abstraite doivent polymorpher toutes les méthodes abstraites ou être également abstraites



# Atelier 01

- Exercice 1: Syntaxe d'héritage et d'interface en C#
- Exercice 2: Héritage de classe

**Activité**  
**20 min**





## A retenir

1. **Héritage en C#** : L'héritage est un pilier de la programmation orientée objet. Il permet à une classe (classe fille) d'hériter des membres d'une autre classe (classe mère).
2. **"Est-un" vs. "A-un"** : L'héritage établit une relation "est-un" entre la classe fille et la classe mère. C'est différent d'une composition où l'on a une relation "a-un".
3. **Les classes de base** : En C#, toutes les classes héritent implicitement de la classe Object, qui est la classe de base ultime du .NET Framework.
4. **L'héritage simple** : C# ne supporte que l'héritage simple, ce qui signifie qu'une classe ne peut hériter que d'une seule autre classe. Cependant, elle peut implémenter plusieurs interfaces.
5. **Les interfaces** : Une interface définit un contrat que les classes implémentantes doivent respecter. Une interface peut être considérée comme une classe entièrement abstraite qui ne contient que des déclarations de méthodes, propriétés, indexeurs et événements.
6. **Avantages des interfaces** : Les interfaces offrent une grande flexibilité en permettant à une classe d'hériter de plusieurs comportements sans les contraintes de l'héritage multiple direct.
7. **Classes abstraites** : Une classe abstraite est une classe qui ne peut pas être instanciée directement. Elle sert de base pour d'autres classes et peut contenir des méthodes abstraites (sans implémentation) que les classes dérivées doivent implémenter.
8. **Différence entre interfaces et classes abstraites** : Alors que les interfaces définissent un contrat sans fournir d'implémentation, les classes abstraites peuvent fournir une implémentation partielle tout en imposant un certain contrat à leurs classes dérivées.
9. **Override et New** : Lors de l'héritage, si une classe dérivée souhaite modifier le comportement d'une méthode héritée, elle peut utiliser le mot-clé override. Si elle souhaite masquer la méthode d'origine, elle utilise new.
10. **Protection contre la modification** : L'utilisation du mot-clé sealed sur une classe empêche d'autres classes d'hériter de celle-ci, garantissant ainsi que ses comportements et propriétés ne seront pas modifiés de manière inattendue.

# **Chapitre 9 - Gestion de la durée de vie des objets et contrôle des ressources**

# Plan de la formation

- 1 - Introduction à C# et .Net
- 2 - Structure de programmation C#
- 3 - Déclaration et appel de méthodes
- 4 - Gestion d'exceptions
- 5 - Lire et écrire dans des fichiers
- 6 - Création de nouveaux types de données
- 7 - Encapsulation de données et de méthodes
- 8 - Héritage de classes et implémentation d'interfaces
- 9 - **Gestion de la durée de vie des objets et contrôle des ressources**
- 10 - Encapsulation avancée
- 11 - Découplage de méthodes et gestion
- 12 - Utilisation des collections et collections
- 13 - Programmation asynchrone et parallèle
- 14 - Utilisation de LINQ pour interroger des données
- 15 - Développement dirigé par les Tests

## Objectifs :

- Comprendre la gestion de la mémoire en C# et comment elle impacte la durée de vie des objets.
- Apprendre à gérer explicitement la mémoire et à libérer les ressources.
- Acquérir des connaissances sur les bonnes pratiques pour éviter les fuites de mémoire et assurer une utilisation efficace des ressources.



# Commençons par échanger

Pouvez-vous expliquer comment fonctionne le ramasse-miettes en C# et pourquoi il est essentiel pour la gestion de la mémoire ?

Avez-vous déjà rencontré les méthodes Dispose ou Finalize lors de votre expérience de codage ? Si oui, pouvez-vous nous dire dans quel contexte ?

Quelle est l'importance de l'interface IDisposable et dans quelles situations l'avez-vous trouvée utile dans le développement d'applications ?

Avez-vous déjà utilisé le mot-clé 'using' dans vos codes précédents ? Comment cela a-t-il influencé la gestion des ressources de votre application ?

Avez-vous déjà rencontré des problèmes de fuites de mémoire dans une application ? Comment avez-vous identifié et résolu ces problèmes ?



- Introduction au Garbage Collection
- Gestion des Ressources

# Leçon 1: Introduction au Garbage Collection

---

- Cycle de vie des objets
- Ressources managés dans .NET
- Fonctionnement du Garbage Collector
- Définir un destructeur
- La classe GC



# Cycle de vie des objets

- Bloc de mémoire alloué

1

2

• Mémoire convertie en objet (constructeur)

3

• Objet utilisé dans l'application

4

• Objet nettoyé (destructeur)

5

• Mémoire réclamée

# Ressources managés dans .NET

## Types Valeurs

- Créés sur la pile
- Détruits automatiquement quand hors de portée
- Jamais fragmentés
- Rapides à allouer et désallouer

```
private void MyMethod(...)
{
    MyStruct a = ...;
    DateTime z = ...;
    double y = ...;
    int x = ...;
    ...

    // x, y, z et a disparaissent ici
}
```

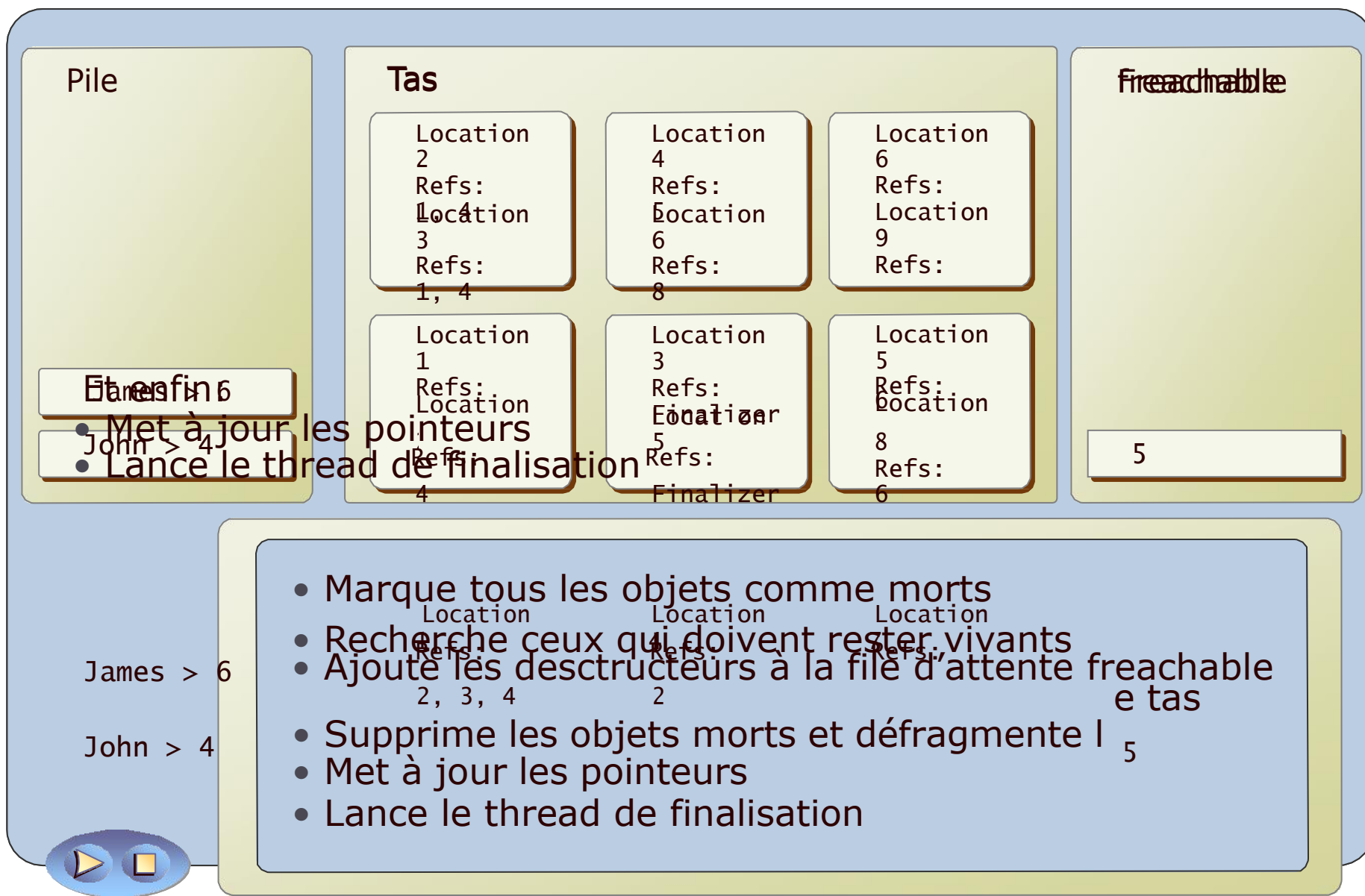
## Types références

- Créés dans le tas
- Supportent plusieurs références au même objet
- Détruits après la disparition de toutes les références
- Fragmentation possible
- Gérés par le garbage collector
- Plus coûteux à gérer

```
string s = ...;
MyClass c = ...;

private void MyMethod2(...)
{
    string t = s;
    MyClass d = c;
    ...
    // les références t et d
    disparaissent
    // les objets s et c existent encore
}
```

# Fonctionnement du Garbage Collector



# Définir un destructeur

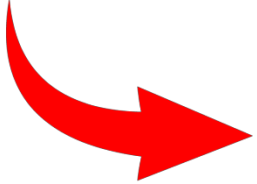
```
class Employee
{
    ...
    ~Employee()
    {
        // Code Nettoyage
    }
}
```

- Le destructeur est défini par un tilde (~) suivi du nom de la classe

- Les types valeurs ne peuvent pas avoir de destructeur

- Les destructeurs n'ont pas de paramètres

- Le compilateur convertit le destructeur en méthode polymorphe Finalize



```
class Employee
{
    ...
    protected override void Finalize()
    {
        try
        {
            // Code Nettoyage
        }
        finally
        {
            base.Finalize();
        }
    }
}
```

# La classe GC

| Méthode                         | Description                                                                          |
|---------------------------------|--------------------------------------------------------------------------------------|
| <b>Collect</b>                  | Force le garbage collection                                                          |
| <b>WaitForPendingFinalizers</b> | Suspend le thread en cours jusqu'à la fin de traitement de la file <b>freachable</b> |
| <b>SuppressFinalize</b>         | Désactive l'invocation du destructeur sur l'objet passé en paramètre                 |
| <b>ReRegisterForFinalize</b>    | Réactive l'invocation du destructeur pour l'objet passé en paramètre                 |
| <b>AddMemoryPressure</b>        | Informe le système de l'allocation d'un bloc volumineux de mémoire non managé        |
| <b>RemoveMemoryPressure</b>     | Informe le système de la libération d'un bloc volumineux de mémoire non managé       |

## Leçon 2: Gestion des Ressources

---

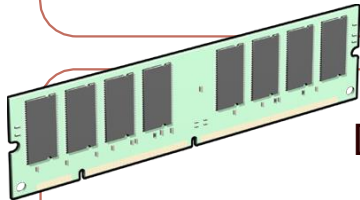
- Pourquoi gérer les ressources dans un environnement managé?
- Modèle Dispose
- Gestion des ressources dans une application
- Déclaration using

# Pourquoi gérer les ressources dans un environnement managé?

Les ressources non managées ne sont pas du ressort du garbage collection:

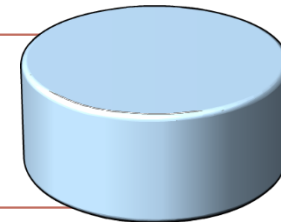
- Vous devez vous charger de libérer manuellement ces ressources lorsqu'un objet est détruit
- Ne pas libérer proprement des objets non managés peut être source de problèmes (perte de données, fuite mémoire...)

Fichiers verrouillés empêchant d'autres applications d'y accéder



Données mises en cache mémoire, perdues suite à une mauvaise gestion des objets

Les connexions aux bases de données sont souvent limitées



# Modèle Dispose

```
class LogFileWriter : ..., IDisposable
{
    private bool isDisposed = false;

    ~LogFileWriter()
    {
        Dispose(false);
    }

    public void Dispose()
    {
        Dispose(true);
        GC.SuppressFinalize(this);
    }

    protected virtual void
        Dispose(bool isDisposing)
    {
        if (!isDisposed)
        {
            if (isDisposing)
            {
                // Nettoyage ressources managées;
            }
            // Nettoyage ressources non managées;
            isDisposed = true;
            base.Dispose(isDisposing);
        }
    }
}
```

Implémenter l'interface IDisposable

Garder une trace de l'état de l'objet

Ajouter un destructeur qui assurera  
le nettoyage automatique de l'objet

Ajouter la méthode d'interface  
Dispose qui assurera le nettoyage  
manuel de l'objet et désactivera le  
destructeur

Ajouter une méthode virtuelle  
Dispose qui par le paramètre  
booléen, effectuera le nettoyage  
manuel (toutes les ressources) ou  
automatique (seulement les  
ressources non managées)



## Gestion des ressources dans une application

- Implémentez l'interface *IDisposable* sur les classes qui gèrent des données que le CLR ne peut pas récupérer
- Beaucoup de classes du Framework son *IDisposable*
- Il est généralement conseillé d'utiliser ces classes avec la déclaration **using**
- Si le **using** n'est pas possible, utiliser un try/catch

```
DisposableObject disp = new  
    DisposableObject();  
try  
{  
    // Utiliser l'objet  
}  
finally  
{  
    disp.Dispose();  
}
```

## Déclaration using

- Ne fonctionne qu'avec des types *IDisposable*
- A ne pas confondre avec l'instruction **using** d'import des espaces de noms

```
using(DisposableObject disp = new DisposableObject())  
{  
    // Utiliser l'objet  
} // Dispose
```

```
DisposableObject disp2 = new DisposableObject();  
using(disp2)  
{  
    // Utiliser l'objet  
} // Dispose
```

```
Using DisposableObject disp = new DisposableObject();  
// Utiliser l'objet  
// Dispose quand la variable est hors de portée
```

# Atelier 01

**Activité**  
**20 min**



- Exercice 1: Implémentation de l'interface IDisposable



## A retenir

1. **Automatisation de la gestion de la mémoire** : En C#, le Garbage Collector (GC) gère automatiquement la libération de la mémoire qui n'est plus utilisée, ce qui élimine la nécessité pour le développeur de le faire manuellement.
2. **Principe du Garbage Collection** : Le GC fonctionne en traquant et libérant la mémoire des objets qui ne sont plus référencés dans l'application, assurant ainsi une utilisation efficace de la mémoire.
3. **Eviter les fuites de mémoire** : Même avec le GC, il est possible d'avoir des fuites de mémoire si les objets sont toujours référencés mais inutilisés. Il est crucial de surveiller et de déréférencer les objets lorsque nécessaire.
4. **L'importance de IDisposable** : L'interface IDisposable fournit le mécanisme pour libérer les ressources non gérées telles que les fichiers, les flux et les connexions. Lorsqu'une classe implémente cette interface, elle doit fournir une méthode Dispose pour la libération des ressources.
5. **Utilisation du mot-clé using** : En C#, le mot-clé using garantit que les objets IDisposable sont correctement disposés, ce qui est particulièrement utile pour s'assurer que les ressources sont libérées immédiatement après leur utilisation.
6. **Finaliseurs et destructeurs** : En C#, les destructeurs (ou finaliseurs) peuvent être utilisés pour libérer des ressources non gérées, mais ils ne devraient être utilisés que lorsque c'est absolument nécessaire car le GC gère cela automatiquement pour la plupart des cas.
7. **Gestion des ressources non gérées** : Toutes les ressources, comme les connexions à la base de données ou les fichiers, ne sont pas gérées par le GC. Il est crucial de comprendre quand et comment libérer ces ressources pour éviter des problèmes de performance ou d'accès.
8. **Performance et GC** : Bien que le GC soit conçu pour être efficace, une utilisation excessive de la mémoire peut entraîner des performances médiocres. Les développeurs doivent être conscients de l'impact de la création excessive d'objets et de la rétention de références inutiles.
9. **Les générations du GC** : Le GC utilise le concept de générations pour optimiser la collecte des objets. Comprendre comment les différentes générations fonctionnent peut aider à écrire un code plus performant.
10. **Responsabilité du développeur** : Bien que le GC soit puissant, le développeur doit toujours être attentif à la gestion de la mémoire et des ressources pour garantir des applications performantes et robustes.-

# **Chapitre 10 - Encapsulation avancée**

# Plan de la formation

- 1 - Introduction à C# et .Net
- 2 - Structure de programmation C#
- 3 - Déclaration et appel de méthodes
- 4 - Gestion d'exceptions
- 5 - Lire et écrire dans des fichiers
- 6 - Création de nouveaux types d'objets
- 7 - Encapsulation de données et de méthodes
- 8 - Héritage de classes et implémentation d'interfaces
- 9 - Gestion de la durée de vie des objets et contrôle des ressources
- 10 - **Encapsulation avancée**
- 11 - Découplage de méthodes et gestion d'évènements
- 12 - Utilisation des collections et construction de types génériques
- 13 - Programmation asynchrone et personnalisation du code
- 14 - Utilisation de LINQ pour interroger des données
- 15 - Développement dirigé par les Tests

## Objectifs :

- Approfondir la compréhension de l'encapsulation
- Appliquer l'encapsulation dans des scénarios complexes
- Promouvoir la maintenance et l'évolutivité du code



# Commençons par échanger

Pourquoi pensez-vous que l'encapsulation est considérée comme un pilier fondamental de la programmation orientée objet ?

Quels avantages voyez-vous à limiter l'accès direct aux données d'une classe ?

Avez-vous déjà rencontré des situations où un changement dans une partie du code a eu des effets inattendus sur d'autres parties du système ? Comment l'encapsulation pourrait-elle aider à prévenir de tels problèmes ?

Quelle est votre expérience avec les propriétés dans d'autres langages de programmation ou frameworks ? Comment les utilisez-vous généralement ?

Est-ce que l'idée de "types imbriqués" vous évoque quelque chose ? Dans quels contextes pourriez-vous imaginer que cela soit utile ?



- Création et utilisation des propriétés
- Création et utilisation des indexeurs
- Surcharge d'opérateurs



# Leçon 1: Création et utilisation des propriétés

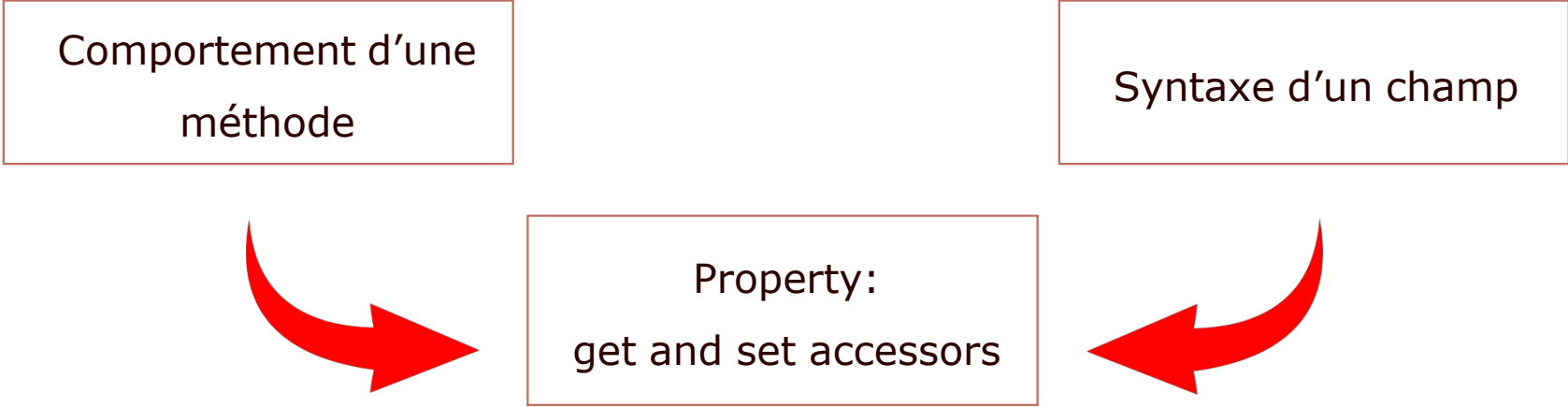
---

- Définition des propriétés
- Création d'une propriété
- Propriétés automatiques
- Instanciation d'objet avec propriétés
- Propriétés en initialisation uniquement
- Propriétés et Interface
- Bonnes pratiques

# Définition des propriétés

Comportement d'une  
méthode

Syntaxe d'un champ

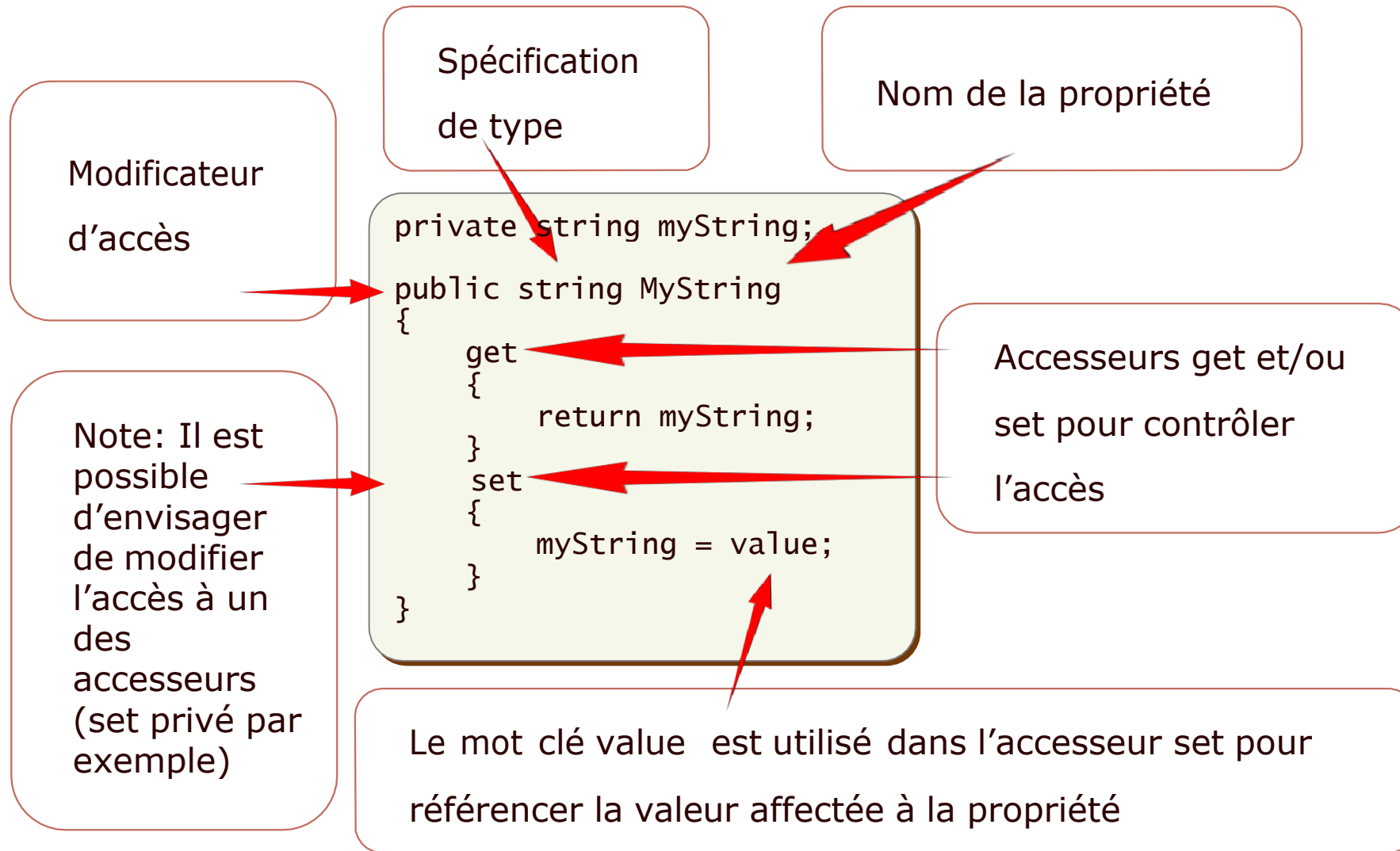


Property:  
get and set accessors

Les propriétés permettent:

- Le contrôle de l'accès aux données
- La validation
- Le contrôle Lecture/Ecriture

# Création d'une propriété



# Propriétés automatiques

```
public string Name { get; set; }
```

A la compilation, la propriété automatique est remplacée par le code suivant

```
private string _name;  
public string Name  
{  
    get  
    {  
        return _name;  
    }  
    set  
    {  
        this._name = value;  
    }  
}
```

- Permet d'assurer l'encapsulation des données rapidement et simplement
- Permet l'évolutivité du code tout en assurant la compatibilité descendante
- Peut être défini par get/set ou get uniquement
- Peut être initialisé à la déclaration

```
public string Name { get; set; } = "John";
```



# Instanciation d'objets avec propriétés

```
Employee john = new Employee { Name = "John" };  
Employee louisa = new Employee() { Department = "Technical" };  
Employee mike = new Employee  
{  
    Name = "Mike",  
    Department = "Technical"  
};
```

- Les initialiseurs d'objets évitent le problème de création d'un grand nombre de constructeurs
- Un constructeur par défaut devrait assurer les initialisations aux valeurs par défaut
- Le constructeur est toujours appelé d'abord, les propriétés ensuite
- Il est possible d'invoquer un constructeur vide (avec ou sans parenthèses) ou bien un constructeur avec paramètres (bien évidemment avec les valeurs appropriées)

## Propriétés en initialisation uniquement

- L'utilisation des initialiseurs d'objets nécessite une propriété disposant de l'accessneur *set*
  - Il est donc envisageable de modifier cette valeur ensuite
- En remplaçant le mot-clé *set* par le mot-clé *init*, la propriété n'autorise plus la modification sauf:
  - Dans un initialiseur d'objet
  - Dans un constructeur
  - Dans l'accessneur *init*

```
public string Name { get; init; }
```

```
private string _name;  
public string Name  
{  
    get  
    {  
        return _name;  
    }  
    init  
    {  
        this._name = value;  
    }  
}
```

# Propriétés et Interface

Rappel: une interface est un contrat entre un type et une application.  
Elle ne contient pas de détails d'implémentation (sauf méthode par défaut).

- Les champs sont considérés comme détail d'implémentation et sont donc interdits dans une déclaration d'interface.
- Les propriétés sont avant tout des méthodes d'accès à des données, et sont donc autorisées dans les interface (il en est de même avec les indexeurs)

```
interface IPerson
{
    string Name { get; set; }
    int Age { get; }
    DateTime DateOfBirth { set; }
}
```

Syntaxe identique à celle des propriétés automatiques, mais s'agissant de prototypes, il est possible de choisir les accesseurs

# Bonnes pratiques

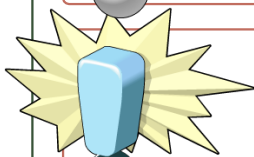
N'utilisez des propriétés uniquement que lorsque cela a un sens

BankAccount

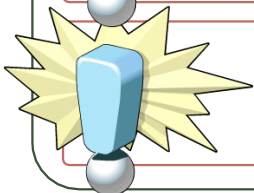
- ☐ double Balance (get, ~~set~~)
- ☒ WithdrawMoney(double Amount)
- ☒ DepositMoney (double Amount)



Une application peut-elle directement définir le solde?



Le code de l'accessor get ne doit pas modifier de données



Attention aux problèmes de récursivité dans les propriétés



## Leçon 2: Création et utilisation des Indexeurs

---

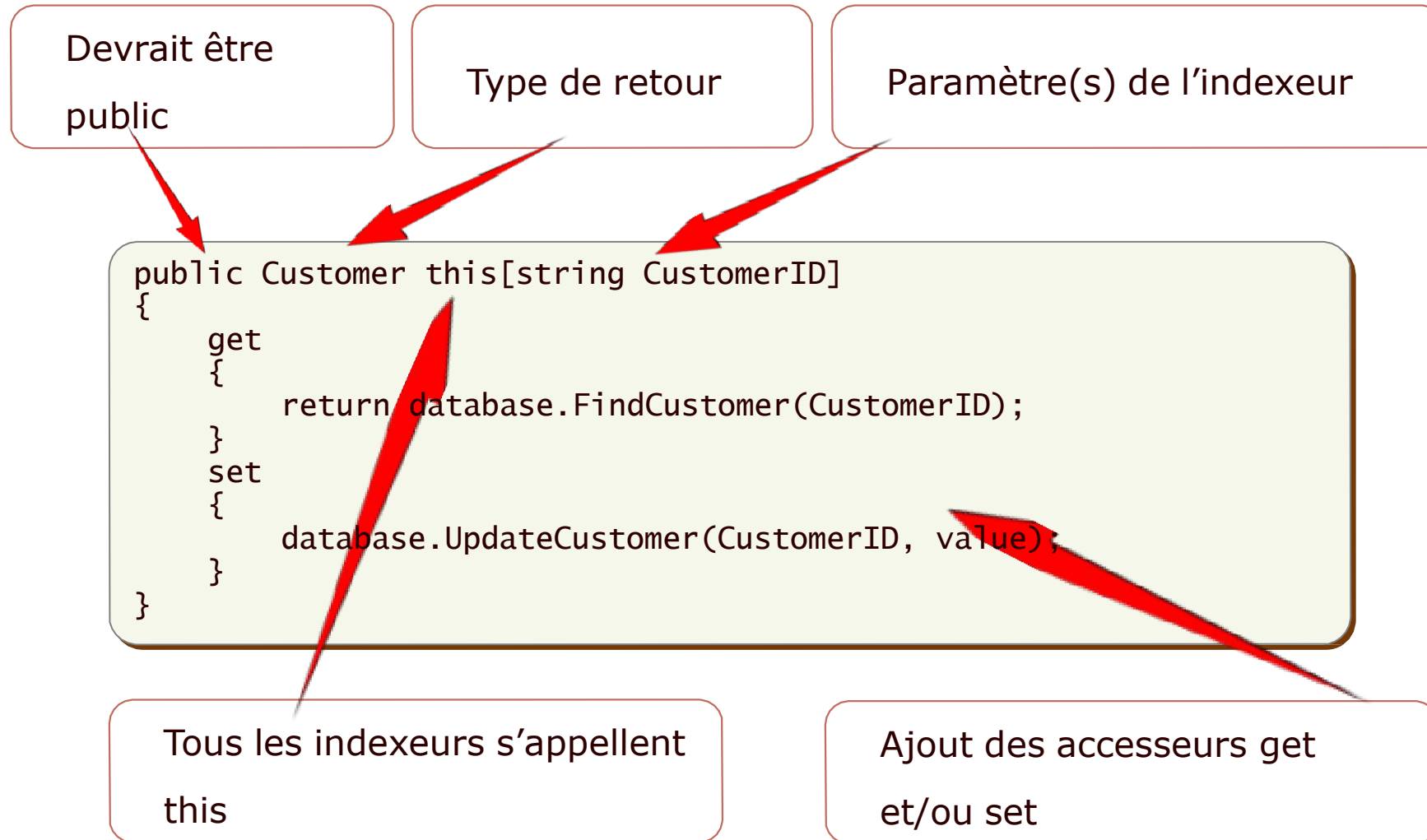
- Définition
- Création
- Comparaison entre Indexeur et Tableau
- Initialisation d'indexeur

# Définition

- Fournit une syntaxe type-tableau pour accéder aux membres d'un ensemble
- Peut utiliser n'importe quel type de données pour accéder à un élément
- Peut être surchargé

```
CustomerAddressBook addressBook = ...;  
Address customerAddress = addressBook["a2332"];  
...  
customerAddress = addressBook[99];
```

# Création



# Comparaison Indexeur et Tableau

|                                          | Tableaux                                               | Indexeurs                                                               |
|------------------------------------------|--------------------------------------------------------|-------------------------------------------------------------------------|
| Indexation                               | Uniquement numérique entier                            | Numérique, Chaîne de caractères, voire multiple                         |
| Surcharge                                | Pas de surcharge possible                              | Possibilité de surcharge                                                |
| Passage en tant que paramètre de méthode | Peut être utilisé avec ou sans <b>ref</b> / <b>out</b> | Ne peut pas être utilisé en tant que paramètre <b>ref</b> ou <b>out</b> |

# Initialisation d'indexeur

- Basée sur les initialiseurs d'objet
  - Initialise chaque membre de l'ensemble sous-jacent
- 
- Egaleme<sup>nt</sup> disponible sur les collections

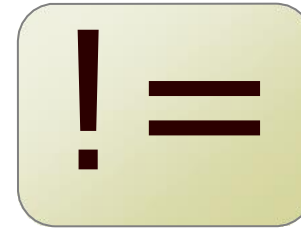
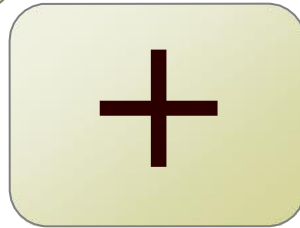
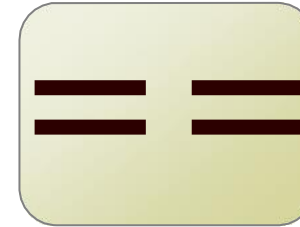
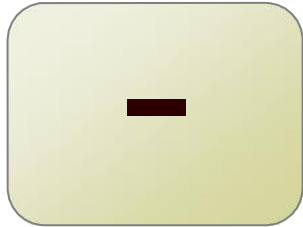
```
CustomerAddressBook addressBook =  
    new CustomerAddressBook {  
        ["A1001"] = new Address { ... },  
        ["A1002"] = new Address { ... },  
        ...  
    };
```

## Leçon 3: Surcharge d'opérateurs

---

- Définition
- Surcharge d'un opérateur
- Restrictions
- Bonnes pratiques
- Opérateurs de Conversion

# Définition



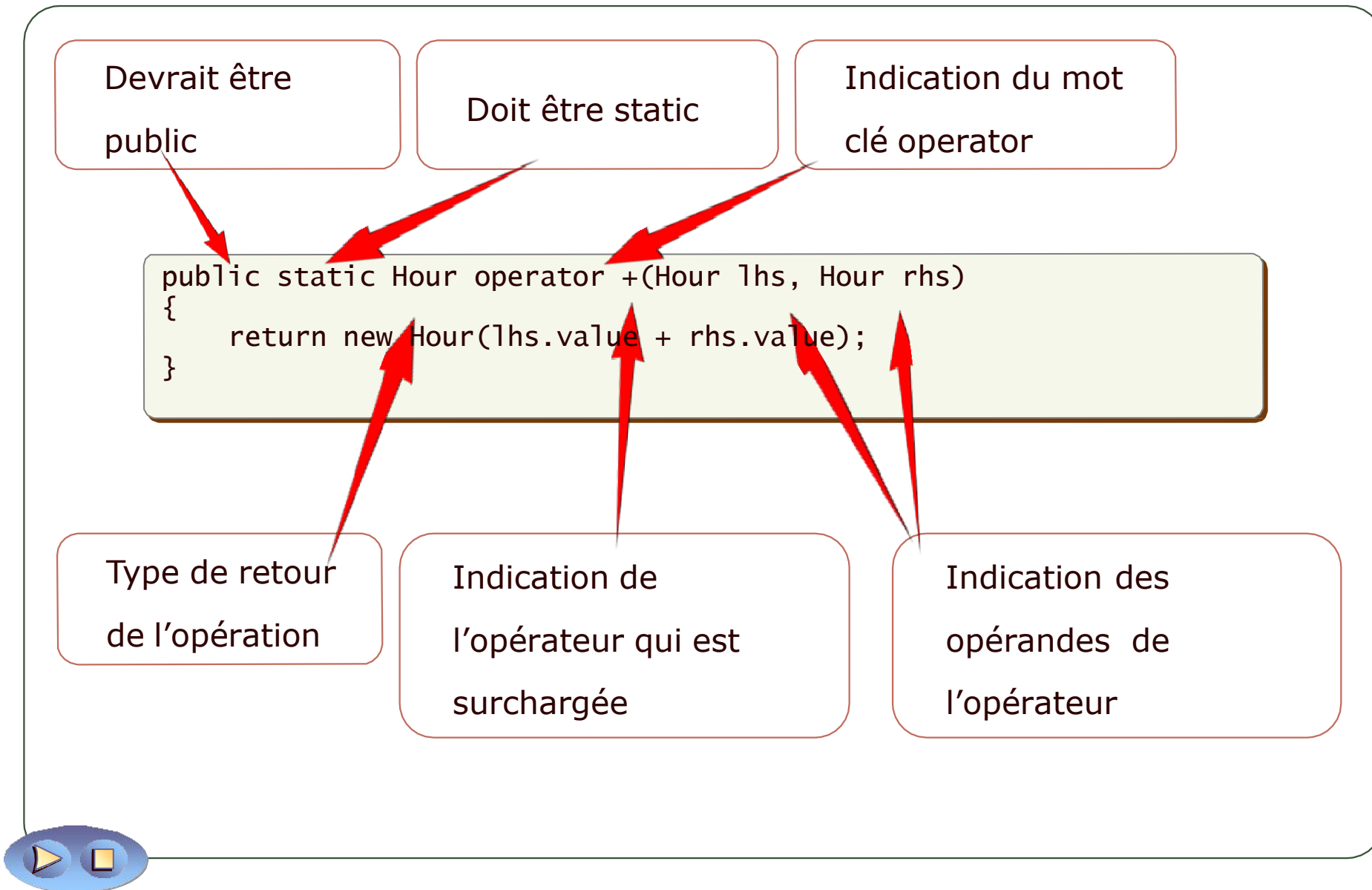
```
Class MyType { ... }
```

```
...  
MyType var1 = ...;  
MyType var2 = ...;  
...
```

```
? = var1 + var2;
```



# Surcharge d'un opérateur





# Restrictions

1. Il est impossible de changer précedence et associativité d'un opérateur
2. Il est impossible de changer la multiplicité d'un opérateur
3. Il est impossible d'inventer de nouveaux opérateurs (rôle des méthodes)
4. Il est impossible de modifier les opérateurs pour les types intégrés
5. Tous les opérateurs ne sont pas surchargeables, par exemple le point (.)
6. Il est obligatoire de définir certains opérateurs par pair (==, !=, etc)

# Bonnes pratiques

- Définissez les opérateurs de manière symétrique



```
public static Salary operator +(Salary salary, double number)
{
    return new Salary(salary.amount + number);
}
public static Salary operator +(double number, Salary salary)
{
    return salary + number;
}
```



- Ne modifiez pas les opérandes

- Ne définissez des opérateurs que lorsque cela a un sens



```
public static BankAccount operator +(BankAccount account, decimal amount)
{ ... }
```



```
public static BankAccount operator -(BankAccount account, decimal amount)
{ ... }
```

# Opérateurs de Conversion

## Conversion

implicite:

Pas de perte  
de données

```
public static implicit operator int (Hour from)
{
    return from.value;
}
```

```
public static void MyMethod(int parameter) { ... }
public static void Main()
{
    Hour lunch = new Hour(12);
    Example.MyMethod(lunch); // implicite
}
```

## Conversion explicite:

Risque de  
perte de  
données,  
nécessite un  
cast

```
public static explicit operator Hour (int from)
{
    return new Hour(from);
}
```

```
public static void MyOtherMethod(Hour parameter) { ... }
public static void Main()
{
    int lunch = 12;
    Example.MyOtherMethod((Hour)lunch); // explicite
}
```

# Atelier 01

- Exercice 1: Création de propriétés
- Exercice 2: Création d'une propriété Indexeur
- Exercice 3: Surcharge d'opérateur (facultatif)

**Activité**  
**20 min**





## A retenir

1. **Propriétés** : Les propriétés en C# permettent d'encapsuler les données d'un objet en offrant une méthode élégante pour lire, écrire ou calculer la valeur d'un champ privé.
2. **Sécurité des données** : Grâce aux propriétés, il est possible de valider les données avant de les affecter à un champ, ce qui garantit l'intégrité et la sécurité des données.
3. **Indexeurs** : Les indexeurs permettent à une classe d'être indexée comme un tableau, offrant une manière intuitive d'accéder à ses éléments.
4. **Flexibilité avec les indexeurs** : Vous pouvez définir plusieurs indexeurs pour une seule classe, chacun ayant un type d'index différent.
5. **Surcharge d'opérateurs** : En C#, vous pouvez redéfinir le comportement des opérateurs existants pour vos propres types, offrant une expression naturelle pour vos classes.
6. **Opérations intuitives** : La surcharge d'opérateurs permet de rendre certaines opérations plus intuitives, par exemple, en ajoutant deux objets ensemble avec l'opérateur +.
7. **Respect des conventions** : Bien que la surcharge d'opérateurs offre une grande flexibilité, il est essentiel de s'assurer que le comportement redéfini reste intuitif et ne surprend pas les utilisateurs de la classe.
8. **Encapsulation avancée** : L'encapsulation ne concerne pas uniquement la protection des données. Elle englobe également la manière dont les utilisateurs de la classe interagissent avec elle, rendant les interactions plus naturelles et intuitives.
9. **Cohérence** : L'utilisation de propriétés, d'indexeurs et de surcharge d'opérateurs doit être cohérente pour éviter la confusion.
10. **Meilleures pratiques** : Tout en exploitant ces fonctionnalités avancées, il est essentiel de se rappeler des meilleures pratiques en matière de conception d'objet pour garantir que la classe reste maintenable et compréhensible.

# **Chapitre 11 - Découplage de méthodes et gestion d'évènements**

# Plan de la formation

- 1 - Introduction à C# et .Net
- 2 - Structure de programmation C#
- 3 - Déclaration et appel de méthodes
- 4 - Gestion d'exceptions
- 5 - Lire et écrire dans des fichiers
- 6 - Création de nouveaux types de données
  - Encapsulation de données et de méthodes
- 8 - Héritage de classes et implémentation d'interfaces
- 9 - Gestion de la durée de vie des objets et contrôle des ressources
- 10 - Encapsulation avancée
- 11 - **Découplage de méthodes et gestion d'évènements**
- 12 - Utilisation des collections et construction de types génériques
- 13 - Programmation asynchrone et persistance
- 14 - Utilisation de LINQ pour interroger des données
- 15 - Développement dirigé par les Tests

## Objectifs :

- Comprendre le besoin de découplage
- Maîtriser les mécanismes de découplage
- Gérer les événements efficacement



# Commençons par échanger

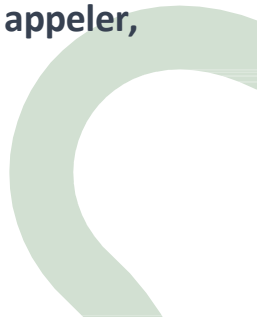
"Pouvez-vous me donner un exemple d'une situation où deux composants d'une application doivent interagir entre eux, mais sans dépendre fortement l'un de l'autre ?"

"Dans un contexte de développement logiciel, pourquoi pensez-vous qu'il est important que les différents composants d'une application soient indépendants et modulaires ?"

"Avez-vous déjà utilisé une application ou un service qui vous notifie lorsqu'un événement spécifique se produit ? Comment imaginez-vous que cela fonctionne techniquement en arrière-plan ?"

"Pourquoi est-il essentiel que chaque composant ou classe d'une application ait sa propre responsabilité claire ? Comment cela pourrait-il influencer la qualité du code et la maintenance ?"

"Dans quelles situations pourriez-vous avoir besoin de décider à la volée quelle fonction ou méthode appeler, plutôt que de l'avoir codée en dur ?"





- Déclaration et utilisation de délégués
- Utilisation des expressions Lambda
- Gestion d'événements

## Leçon 1: Déclaration et utilisation des délégués

---

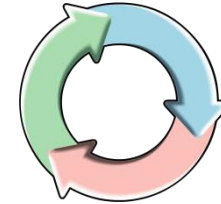
- Découplage opération/méthode
- Définition d'un délégué
- Appel d'un délégué
- Méthodes anonymes

# Découplage opération/méthode

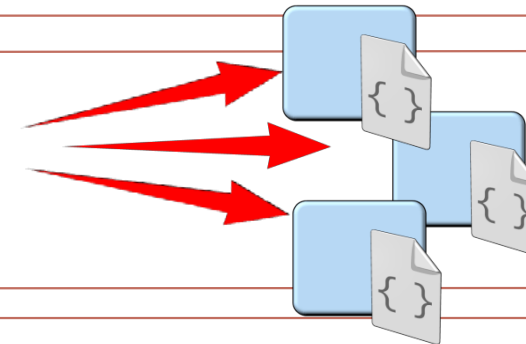
- Choisir la méthode au moment de l'exécution
  - Méthode A ou méthode B ou méthode C



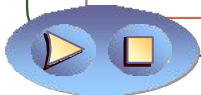
- Méthodes de rappel (Callback)
  - Fournir une méthode à exécuter lorsqu'un appel asynchrone se termine



- Opérations à multidiffusion (Multicast)
  - Méthode A et méthode B et méthode C



Implémentés par les délégués, que l'on peut considérer comme des références vers des méthodes



# Définition d'un délégué

## Développeur de Classe

- Déclaration de la signature du délégué à l'aide du mot clé delegate

```
public delegate bool isValidDelegate();
```

- Création de l'instance qui stockera les références de méthodes

```
public isValidDelegate isValid = null;
```

## Développeur Applicatif

- Ajout des références par l'opérateur +=

```
isValid += CheckStateValid;  
isValid += new isValidDelegate(CheckControl);
```

- Suppression de références par l'opérateur -=

```
isValid -= CheckStateValid;
```



# Appel d'un délégué

Toujours vérifier que le délégué soit non null avant de l'invoquer!

- Appel d'un délégué de façon synchrone

```
if (isValid != null)
{
    isValid();
}
```

- Appel d'un délégué de façon asynchrone à l'aide des méthodes BeginInvoke et EndInvoke

```
if (isValid != null)
{
    isValid.BeginInvoke(...);
}
```

# Méthodes anonymes

Ajout de la méthode anonyme aux références d'une instance de délégué

Utilisation du mot clé delegate

```
myDelegateInstance += new  
myDelegate  
(delegate (int parameter)  
{  
    // Perform operation.  
    return 10;  
});
```

Indiquer si besoin les paramètres nécessaires

Mettre en oeuvre le corps de la méthode

Il n'est pas nécessaire d'indiquer le type de retour car il sera déterminé par le contexte, à savoir par la signature du délégué



## Leçon 2: Utilisation des expressions Lambda

- Définition
- Syntaxe
- Portée des variables dans les expressions Lambda
- Réécriture de méthodes et de propriétés
- Alternative par les fonctions locales

# Définition

- Une expression Lambda est une expression qui retourne une méthode
- Les expressions Lambda sont définies en utilisant l'opérateur  $=>$
- Les expressions Lambda utilisent une syntaxe naturelle et concise
- Les types des paramètres et valeur de sortie sont déterminés par le contexte

$$X => X * X$$

Cet exemple peut être lu ainsi: Pour x, calculer  $x * x$



Expression simple où le type du paramètre  $x$  est déterminé par le contexte

```
x => x * x
```

Expression mettant en oeuvre un bloc de code C#

```
x => { return x * x ; }
```

Expression sans paramètre d'entrée et invoquant une méthode

```
() => myObject.MyMethod(0)
```

Expression avec typage explicite du paramètre

```
(int x) => x / 2
```

Expression avec paramètres multiples, dont un passé par référence

```
(ref int x,int y) => {x++; return x/y;}
```



# Portée des variables dans les expressions Lambda

- Une expression Lambda peut utiliser toutes les variables accessibles dans la portée où elle est déclarée
- Une expression Lambda peut définir ses propres variables

```
string name;           // Champs de niveau classe
MyDelegate del = null;
...
void myMethod() {
    int count = 0;
    del += new MyDelegate(() =>
    {
        int temp = 10;
        CheckName(name);
        count++; // Résultat imprévisible
    });
}
```

- Utiliser des variables externes dans une expression Lambda peut étendre leur durée de vie: A utiliser avec précaution!
- Il est possible d'interdire ce comportement en préfixant la lambda par *static*

# Réécriture de méthodes et de propriétés

Il est possible de simplifier l'écriture des méthodes et des propriétés en réutilisant la syntaxe des expressions Lambda

```
public string GetData(int index){  
    return String.Format("Text: {0}",data[index]);  
}
```



```
public string GetData(int index) => return String.Format("Text: {0}",data[index]);
```

```
// Constructeur  
public ExpressionMembersExample(string label) => this.Label = label;  
// Destructeur  
~ExpressionMembersExample() => Console.Error.WriteLine("Finalized!");  
  
private string label;  
// Propriétés  
public string Label  
{  
    get => label;  
    set => this.label = value ?? "Default label";  
}
```

## Alternative par les fonctions locales

- Utile pour la gestion de la récursivité
- Plus performant que les expressions Lambda pour des fonctions appelées uniquement dans le contexte d'une méthode

```
public static int LambdaFactorial(int n)
{
    MonDelegate nthFactorial = (number) => (number < 2) ?
        1 : number * nthFactorial(number - 1);

    return nthFactorial(n);
}
```

```
public static int LocalFunctionFactorial(int n)
{
    return nthFactorial(n);

    int nthFactorial(int number) => (number < 2) ?
        1 : number * nthFactorial(number - 1);
}
```

## Leçon 3: Gestion d'événements

---

- Définition
- Syntaxe
- Utilisation
- Bonnes pratiques

# Définition

- Basés sur les délégués, les événements permettent à un composant (fournisseur) de signaler à une application (consommateur) que quelque chose s'est produit
- Contrairement aux délégués, les événements ne peuvent être déclenchés que dans la classe qui l'a défini
- L'application consommatrice ajoute les références de méthodes qu'elle veut voir exécuter lors du déclenchement de l'événement. A noter que seuls les opérateurs += et -= sont autorisés
- Les événements sont beaucoup utilisés en .NET et en particulier dans les applications graphiques

Définir le délégué sur lequel sera basé l'événement; le délégué doit être au moins aussi visible que l'événement

```
public delegate void MyEventDelegate(object sender, EventArgs e);  
  
public event MyEventDelegate MyEvent = null;
```

Utiliser le mot  
clé event

Spécifier le type du  
délégué

Donner un nom à  
l'événement



Il est possible de définir un événement  
dans une interface



# Utilisation

Inscription à un événement par l'opérateur += comme cela aurait été le cas avec un simple délégué

```
MyEvent += new MyEventDelegate(myHandlingMethod);
```

Utilisation de l'opérateur -= pour se désinscrire

```
MyEvent -= myHandlingMethod;
```

Pour soulever/déclencher l'événement, il suffit de l'invoquer comme s'il s'agissait d'une méthode (en fait, nous invoquons les méthodes référencées)

```
if (MyEvent != null)
{
    EventArgs args = new EventArgs();
    MyEvent(this, args);
}
```

Toujours vérifier que l'événement est différent de null avant de le déclencher!





# Bonnes pratiques



Utiliser la signature standard

```
void MyEvent(object sender, EventArgs e)
```



Utiliser une méthode *protected virtual* pour lancer l'événement



Ne jamais passer de valeurs *null* à l'événement

Il devient rare de créer soi-même ses propres déclarations de délégués.

En effet, il existe en .NET des délégués prédéfinis: les *délégués génériques*

- Action, Action<T>, Action<T1,T2>,...
- Func<TResult>, Func<T,Tresult>, Func<T1,T2,Tresult>,...

Les types T, T1, T2, ... sont définis au moment où le code est écrit.

Nous en reparlerons dans le module 12.

# Atelier 01

- Exercice 1: Application graphique et événement
- Exercice 2: Mise en œuvre d'un délégué (facultatif)
- Exercice 3: Création et gestion d'un événement (facultatif)

**Activité**  
**20 min**





## A retenir

1. **Délégués comme Types** : Les délégués sont des types qui représentent des références à des méthodes avec un type de paramètre particulier et un type de retour. Ils peuvent référencer à la fois des méthodes statiques et des méthodes d'instance.
2. **Flexibilité des Délégués** : Grâce aux délégués, vous pouvez définir des opérations comme des paramètres et créer des méthodes plus flexibles et réutilisables.
3. **Multicast Délégués** : Un délégué peut référencer plusieurs méthodes; on parle alors de délégué multicast. Cela permet de déclencher plusieurs méthodes via une seule invocation de délégué.
4. **Expressions Lambda** : Les expressions lambda offrent une syntaxe concise pour écrire des méthodes anonymes. Elles sont particulièrement utiles pour écrire des fonctions courtes qui sont passées à des méthodes d'ordre supérieur.
5. **Syntaxe des Lambdas** : La syntaxe générale d'une expression lambda est (paramètres) => expression ou (paramètres) => { instructions; }.
6. **Les événements comme Communication** : Les événements sont un mécanisme par lequel un objet peut notifier d'autres objets que quelque chose s'est produit. Ils permettent une communication propre et découpée entre composants.
7. **Mécanisme Sous-Jacent** : Les événements utilisent en réalité des délégués en arrière-plan. Chaque événement est basé sur un délégué qui définit la signature des méthodes de rappel (handlers).
8. **Inscription et Désinscription** : Avec les opérateurs += et -=, les méthodes peuvent s'inscrire ou se désinscrire d'un événement et être appelées lorsque l'événement est déclenché.
9. **Principe de Sécurité** : Grâce aux événements, vous pouvez empêcher d'autres classes d'émettre l'événement de votre classe, garantissant ainsi que seul l'objet qui a défini l'événement peut déclencher ses handlers.
10. **Utilisation Judicieuse** : Bien que puissants, les délégués, les lambdas et les événements doivent être utilisés judicieusement pour garantir la lisibilité du code et éviter des complications inutiles.

# **Chapitre 12 - Utilisation des collections et construction de types génériques**

# Plan de la formation

- 1 - Introduction à C# et .Net
- 2 - Structure de programmation C#
- 3 - Déclaration et appel de méthodes
- 4 - Gestion d'exceptions
- 5 - Lire et écrire dans des fichiers
- 6 - Création de nouveaux types de données
  - Encapsulation de données et de méthodes
- 8 - Héritage de classes et implémentation
- 9 - Gestion de la durée de vie des objets
- 10 - Encapsulation avancée
- 11 - Découplage de méthodes et gestion d'événements
- 12 - Utilisation des collections et construction de types génériques**
- 13 - Programmation asynchrone et personnalisation du code
- 14 - Utilisation de LINQ pour interroger des données
- 15 - Développement dirigé par les Tests

## Objectifs :

- Comprendre les bases des collections
- S'initier à la programmation générique
- Appliquer des pratiques de codage avancées



# Commençons par échanger

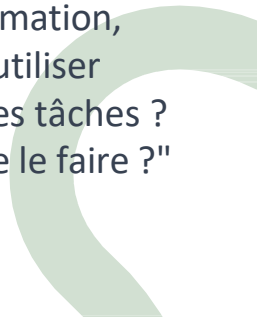
"Pouvez-vous me donner un exemple d'une situation où vous auriez besoin de stocker et d'organiser plusieurs éléments ou données dans votre code ? Comment l'avez-vous géré jusqu'à présent ?"

"Avez-vous déjà rencontré une situation où vous aviez à réutiliser une certaine fonctionnalité pour différents types de données ? Comment avez-vous abordé ce défi ?"

"Quels sont, selon vous, les défis associés à la gestion de grandes quantités de données dans une application ? Comment pensez-vous que des structures de données bien conçues pourraient aider à améliorer les performances ?"

"Pouvez-vous expliquer pourquoi il est important d'avoir un contrôle strict sur les types de données avec lesquels vous travaillez dans une application ? Avez-vous déjà rencontré des problèmes dus à un mélange involontaire de types ?"

"Dans vos expériences précédentes de programmation, combien de fois avez-vous senti le besoin de réutiliser certaines portions de votre code pour différentes tâches ? Quels défis avez-vous rencontrés en essayant de le faire ?"



- Utilisation des collections
- Création et utilisation des types génériques
- Définir des interfaces génériques et comprendre la variance
- Utilisation de méthodes génériques et des délégués

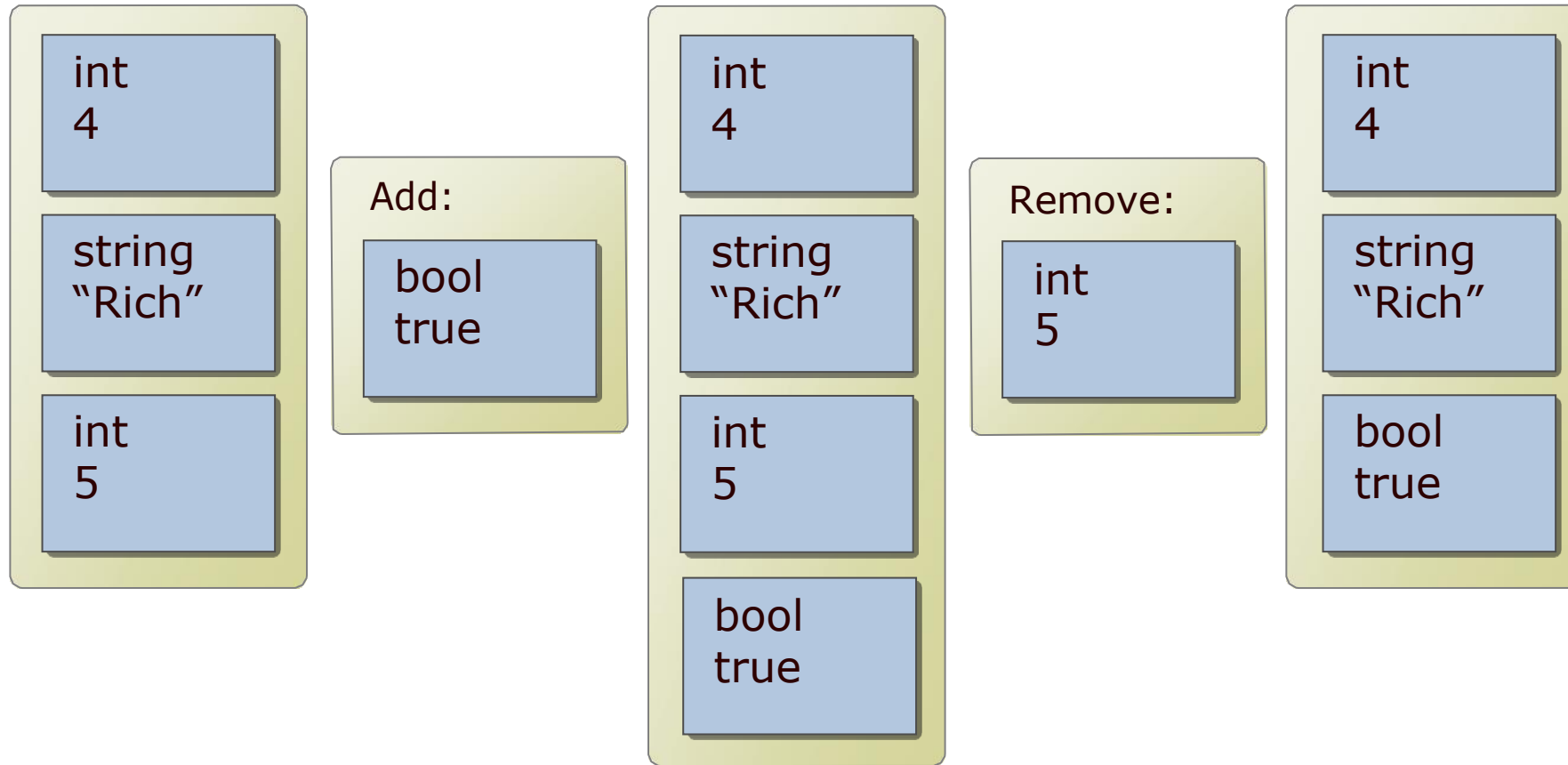


# Leçon 1: Utilisation des collections

---

- Définition
- Utilisation
- Itération
- Collections courantes
- Initialiseurs de collections

# Définition



- Les éléments d'une collection sont référencés à travers le type `System.Object`
- Les collections gèrent l'espace automatiquement

Interface ICollection:

CopyTo

GetEnumerator

Count

Implementée par toutes classes de collection

Interface IList:

Add

Remove

Implémentée par certaines collections

Les éléments sont stockés en tant qu'objet, nécessitant un cast pour les récupérer.

Add et Remove peuvent être remplacés par d'autres méthodes, voire accompagnées d'autres solutions

```
ArrayList list = new ArrayList();  
list.Add(6);  
list.Remove(6);  
list.RemoveAt(1);  
int temp = (int)list[0];
```

# Itération

La boucle foreach récupère chaque élément les uns après les autres

```
foreach(<type> <control_variable> in <collection>)  
{  
    <foreach_statement_body>  
}
```

L'utilisation de la boucle foreach n'est possible que sur les classes implémentant la méthode GetEnumerator de l'interface IEnumerable

Il est également possible d'implémenter GetEnumerator via une méthode d'extension pour les classes dont le code n'est pas modifiable

```
ArrayList list = new ArrayList();  
list.Add(99);  
list.Add(10001);  
list.Add(25);  
...  
foreach (int i in list)  
{  
    Console.WriteLine(i);  
}  
// Sortie : 99  
//           10001  
//           25
```

# Collections courantes

ArrayList

Collection non ordonnée, similaire à un tableau. Accès par indice

Queue

Collection FIFO avec méthodes Enqueue et Dequeue

Stack

Collection FILO avec méthodes Push et Pop

Hashtable

Collection de paires clé-valeur

SortedList

Collection de paires clé-valeur, triée sur la clés

# Initialiseurs de collections

Il est possible d'utiliser la méthode Add pour ajouter des éléments...

```
ArrayList a1 = new ArrayList();  
a1.Add("Value");  
a1.Add("Another Value");
```

...ou encore utiliser les initialiseurs de collection, similaires aux initialiseurs d'objets

```
// person1 contient une référence de Person  
ArrayList a12 = new ArrayList()  
{  
    person1,  
    new Person() {Name="Tom", Age =31}  
};
```

Il est également possible de tirer partie des initialiseurs d'indexeur

```
Hashtable data = new Hashtable() {  
    ["A01"]=person1,  
    ["A02"]=new Person() {Name="Tom", Age =31}  
};
```



## Leçon 2: Création et utilisation des types génériques

- Définition
- Compilation
- Définir un type générique
- Contraintes de généricité
- Collections génériques

# Définition

- Un type générique est un type qui déclare un ou plusieurs paramètres de type
- Un paramètre de type est un paramètre comme les autres, à la différence qu'il permet d'indiquer un type et non une instance d'un type
- Les paramètres de type sont indiqués entre inférieur et supérieur

```
public class List<T>
```

Cet exemple présente la définition d'une classe nommée List acceptant un paramètre de type T. T est un type que l'on pourra utiliser dans la définition de List comme tout autre type



# Compilation

List<T> est ici utilisé plusieurs fois avec des types différents

```
List<string> names = new List<string>();  
names.Add("John");  
...  
string name = names[0];  
  
List<List<string>> listOfLists = new List<List<string>>();  
listOfLists.Add(names);  
...  
List<string> data = listOfLists[0];
```

Le compilateur génère une version fortement typée de la classe générique, en fonction des types passés en paramètre

```
public void Add(string item)  
...  
public void Add(List<string> item)
```

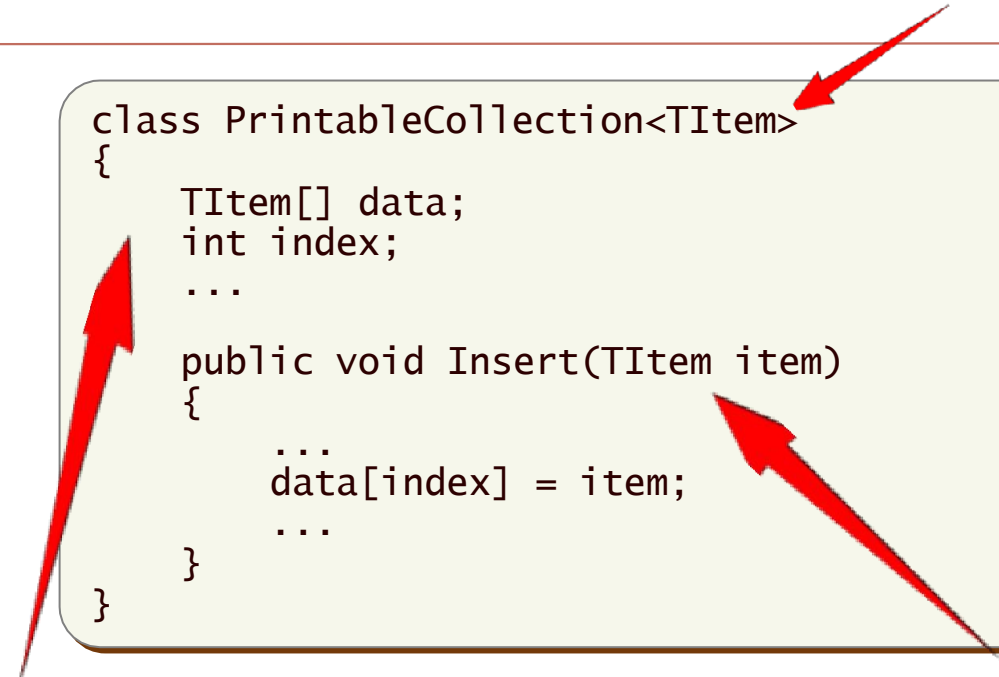
Ces classes et méthodes sont générés à la compilation et donc, vous ne pouvez les accéder directement dans votre code

# Définir un type générique

Définir la classe en ajoutant derrière son nom les paramètres de type

```
class PrintableCollection<TItem>
{
    TItem[] data;
    int index;
    ...

    public void Insert(TItem item)
    {
        ...
        data[index] = item;
        ...
    }
}
```



Le paramètre de type est utilisé tel un alias dans tout le code, que ce soit pour des champs, des propriétés, des paramètres de méthode...



# Contraintes de généricité

| Contraintes                             | Description                                           |
|-----------------------------------------|-------------------------------------------------------|
| <b>where T: struct</b>                  | <b>T</b> doit être un type valeur                     |
| <b>where T : class</b>                  | <b>T</b> doit être un type référence                  |
| <b>where T : new()</b>                  | <b>T</b> doit avoir un constructeur public par défaut |
| <b>where T : &lt;classe de base&gt;</b> | <b>T</b> doit hériter de la classe indiquée           |
| <b>where T : &lt;interface&gt;</b>      | <b>T</b> doit implémenter l'interface indiquée        |
| <b>where T : U</b>                      | <b>T</b> doit hériter de <b>U</b>                     |

# Collections génériques

List<T>

Equivalent de ArrayList

Queue<T>

Equivalent de Queue

Stack<T>

Equivalent de Stack

Dictionary<Tkey,Tvalue>

Equivalent de Hashtable

Mais aussi: LinkedList<T>, SortedList<Tkey,Tvalue>,  
SortedDictionary<Tkey,Tvalue>, ...

## Leçon 3: Définir des interfaces génériques et comprendre la variance

---

- Interfaces génériques
- Problème de l'invariance
- Covariance et contravariance

- Une interface générique peut être implémentée

- Par une classe typée

```
interface IPrinter<T> {  
    ...  
}  
  
class Printer: IPrinter<WordDocument> {  
    ...  
}
```

- Par une classe générique

```
interface IPrinter<T> {  
    ...  
}  
  
class Printer<T>: IPrinter<T> {  
    ...  
}
```

# Définition de l'invariance

- L'invariance est une sécurité empêchant de caster des paramètres de type vers d'autres types de la même hiérarchie d'héritage
- Les classes génériques sont toujours invariantes
- Les interfaces génériques sont invariantes par défaut
- Covariance et contravariance permettent de changer le comportement des interfaces génériques

# Définition de l'invariance (*suite*)

```
string myString = "Hello";  
object myObject = myString;
```

```
interface IData<T>  
{  
    void SetData(T data);  
    T GetData();  
}  
  
class Truc<T> : IData <T>  
{  
    private T storedData;  
  
    void IData<T>.SetData(T data)  
    {  
        this.storedData = data;  
    }  
  
    T IData<T>.GetData()  
    {  
        return this.storedData;  
    }  
}
```

```
Truc<string> strTruc = new Truc<string>();  
IData<string> dataStr = strTruc;
```

```
dataStr.SetData("Hello");  
Console.WriteLine("Stored value is {0}",  
    dataStr.GetData());
```

```
// Ne compile pas  
IData<object> dataObj = strTruc;
```

```
// Compile mais Erreur d'exécution  
IData<object> dataObj =  
    (IData<object>) strTruc;
```

```
// Ce qu'empêche l'invariance  
IData<object> dataObj =  
    (IData<object>) strTruc;  
Window myWindow = new Window();  
objData.SetData(myWindow);
```



- Mécanismes permettant d'autoriser les conversions entre interface générique et classe générique
- Utilisation des mots-clés *out* et *in* dans les déclarations d'interface

Bonne pratique:

Implémenter les interfaces génériques en tant que classe concrète!

## Leçon 4: Utilisation de méthodes génériques et des délégués

---

- Définir une méthode générique
- Utiliser une méthode générique
- Utilisation des délégués génériques

# Définir une méthode générique

Définir une méthode et spécifier le ou les types à utiliser après le nom de la méthode entre <>

```
ResultType MyMethod<Parameter1Type, ResultType>(Parameter1Type  
param1)  
{  
    where ResultType : new()  
    ResultType result = new ResultType();  
    return result;  
}
```

Si besoin, indiquer les contraintes à appliquer aux paramètres de type

Utiliser les paramètres de type là où cela est nécessaire

Créer une méthode générique permet d'éviter la multiplication des surcharges pour prendre en compte différents types



# Utiliser une méthode générique

Lors de l'appel de la méthode, spécifiez le type à utiliser

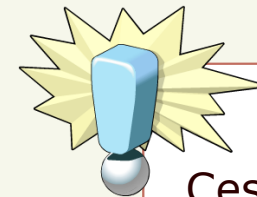
```
T PerformUpdate<T>(T input)
{
    T output = ...;
    return output;
}
...

string result = PerformUpdate<string>("Test");
int result2 = PerformUpdate<int>(1);
```

Le compilateur génère une version de la méthode pour chaque type de paramètre

```
string PerformUpdate(string input)
{
    string output = ...;
    return output;
}

int PerformUpdate(int input)
{
    int output = ...;
    return output;
}
```



Ces méthodes ne sont pas accessibles dans votre code

# Utilisation des délégués génériques

**Action<T>**

Le délégué générique Action permet de ne pas avoir à déclarer la signature d'un délégué sans valeur de retour

**Func<T, TResult>**

Le délégué générique Func<Tresult> permet de ne pas avoir à déclarer la signature d'un délégué à retour de valeur, le paramètre de type TResult représentant la type de cette valeur

.NET inclut des surcharges des délégués Action et Func jusqu'à 16 paramètres

# Atelier 01

**Activité**  
**20 min**



- Exercice 1: Utilisation d'une collection



## A retenir

1. **Les Collections** : Les collections sont essentielles pour stocker et gérer des groupes d'objets. Elles offrent une manière flexible et efficace de travailler avec des ensembles de données dans des applications.
2. **L'importance des types génériques** : Les types génériques permettent de créer des structures de données et des méthodes flexibles tout en conservant la sécurité du typage, réduisant ainsi les risques d'erreurs.
3. **La puissance de la réutilisabilité** : Grâce aux types génériques, vous pouvez concevoir des classes, interfaces et méthodes qui peuvent être réutilisées avec n'importe quel type de données, offrant une adaptabilité énorme.
4. **Sécurité du typage avec les génériques** : Utiliser des génériques garantit que vous travaillez toujours avec le bon type de données, ce qui minimise les erreurs de typage et renforce la robustesse de votre code.
5. **Interfaces génériques** : Les interfaces génériques définissent des contrats pour des classes qui travailleront avec des types spécifiés au moment de l'implémentation, ajoutant une couche supplémentaire d'abstraction et de flexibilité.
6. **Comprendre la variance** : En .NET, la variance permet d'assigner un type générique moins dérivé (par exemple, une liste de base) à un type générique plus dérivé (par exemple, une liste dérivée). Cela offre plus de flexibilité tout en conservant la sécurité du typage.
7. **Méthodes génériques et délégués** : Les méthodes génériques vous permettent de définir une méthode une fois et de l'utiliser pour une variété de types de données. Les délégués génériques étendent cette capacité aux méthodes de rappel.
8. **Performance des génériques** : Les génériques offrent souvent de meilleures performances que les structures non génériques car ils éliminent le besoin de conversions de type coûteuses.
9. **Extensibilité des collections et des génériques** : En comprenant et en maîtrisant les collections et les génériques, vous pouvez étendre leurs fonctionnalités pour répondre aux besoins spécifiques de votre application.
10. **Les génériques renforcent les bonnes pratiques** : En encourageant la réutilisabilité, la sécurité du typage, et l'abstraction, les génériques favorisent les bonnes pratiques de programmation et aident à créer un code propre et maintenable.

# **Chapitre 13 - Programmation asynchrone et personnalisation du code**



# Plan de la formation

- 1 - Introduction à C# et .Net
- 2 - Structure de programmation C#
- 3 - Déclaration et appel de méthodes
- 4 - Gestion d'exceptions
- 5 - Lire et écrire dans des fichiers
- 6 - Création de nouveaux types de données
- 7 - Encapsulation de données et de méthodes
- 8 - Héritage de classes et implémentation
- 9 - Gestion de la durée de vie des objets
- 10 - Encapsulation avancée
- 11 - Découplage de méthodes et gestion
- 12 - Utilisation des collections et constantes
- 13 - Programmation asynchrone et personnalisation du code**
- 14 - Utilisation de LINQ pour interroger des données
- 15 - Développement dirigé par les Tests

## Objectifs :

- Comprendre la Programmation Asynchrone :
- Maîtriser les Mécanismes Asynchrones
- Personnaliser le Comportement des Applications
- Gérer les Erreurs en Asynchrone
- Optimiser les Performances des Applications



# Commençons par échanger

"Avez-vous déjà rencontré des situations, dans votre utilisation quotidienne des applications, où certaines tâches semblaient s'exécuter en arrière-plan sans interrompre votre travail ? Pouvez-vous donner des exemples ?"

"Pensez à vos applications préférées. Quels sont les aspects que vous avez pu personnaliser pour répondre à vos besoins spécifiques ? Estimez-vous que cette personnalisation améliore votre expérience utilisateur ?"

"Avez-vous déjà abandonné l'utilisation d'une application ou d'un site web parce qu'elle était trop lente ou ne répondait pas ? Comment pensez-vous qu'une meilleure gestion de tâches concurrentes aurait pu améliorer cette expérience ?"

"Savez-vous quelle est la différence entre une opération qui est exécutée de manière concurrente et une opération qui est exécutée en parallèle ? Pourquoi est-ce pertinent pour le développement d'applications modernes ?"

"Selon vous, est-il plus complexe de tester et de déboguer un code qui s'exécute de manière asynchrone par rapport à un code synchrone ? Pourquoi ou pourquoi pas ?"



- Programmation asynchrone
- Création d'une classe de collection personnalisée
- Enregistrements
- Simplification du code

# Leçon 1: Programmation asynchrone

---

- Bénéfice
- Modèles de programmation asynchrone
- Utilisation de `async` et `await`

- En programmation traditionnel, une tâche longue bloque l'exécution (modèle synchrone)
- En programmation asynchrone, les tâches longues sont exécutées sur un thread parallèle et ne bloquent pas l'exécution
- Evite que l'application apparaisse « plantée » pour l'utilisateur

- Task-based Asynchronous Pattern (TAP)
  - Une méthode unique qui retourne une instance de *Task* ou *Task<T>*
  - Utilisation des mots-clés *async* et *await*
- Event-based Asynchronous Pattern (EAP)
  - Une méthode (*xxxAsync*) et un ou plusieurs événements
- Asynchronous Programming Model (APM)
  - Une méthode d'initialisation *Beginxxx*
  - Une méthode de récupération *Endxxx*

Seule la TAP est à présent recommandée

## Utilisation de async et await

- Une méthode asynchrone (*awaitable*) est une méthode marquée par le mot-clé **async** et pour laquelle le type de retour est transformé : *void* devient *Task*, *T* devient *Task<T>*
- Une méthode asynchrone a pour but d'appeler une autre méthode asynchrone avec le mot-clé **await**
- L'appel **await** met « en pause » la méthode principale, qui se terminera au retour de la méthode asynchrone

```
private readonly HttpClient _httpClient = new HttpClient();

[HttpGet, Route("DotNetCount")]
public async Task<int> GetDotNetCount()
{
    var html = await _httpClient.GetStringAsync("https://monsite.org");
    return Regex.Matches(html, @"\.NET").Count;
}
```

## Leçon 2: Création d'une classe de collection personnalisée

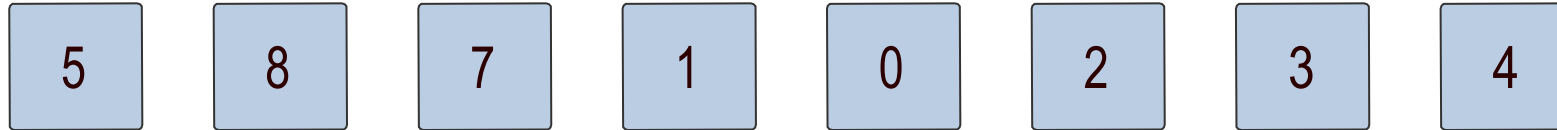
---

- Pourquoi créer sa propre collection?
- Interfaces de collection génériques de .NET
- Exemple de collection : File à double entrée
- Énumération

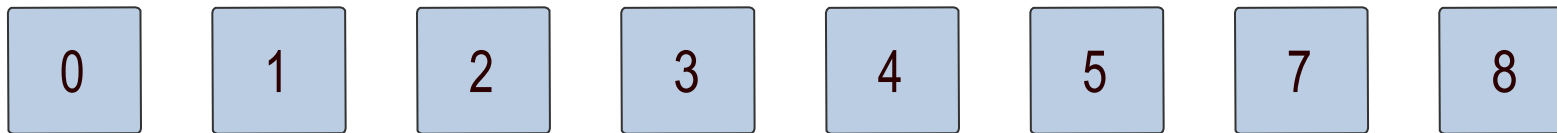


# Pourquoi créer sa propre collection?

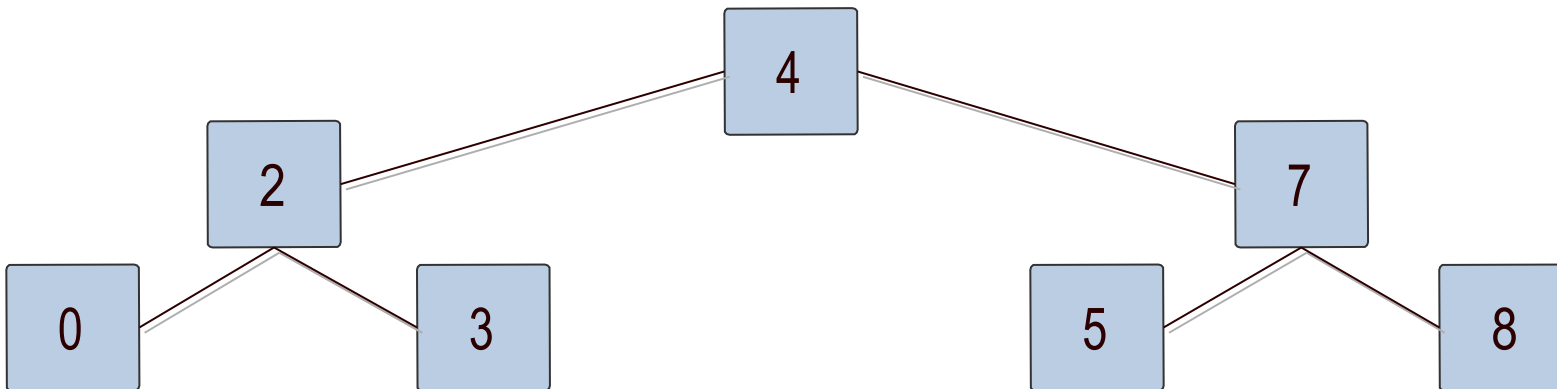
Liste séquentielle -> List<T>



Liste triée -> SortedList<T>



Arbre binaire -> ? -> Il faut créer une nouvelle collection



# Interfaces de collection génériques de .NET

ICollection<T> :  
IEnumerable<T>

Méthodes :

- Add
- Clear
- Contains
- CopyTo
- Remove

Propriétés :

- Count
- IsReadOnly

ICollection<T> :  
ICollection<T>

Méthodes :

- IndexOf
- Insert
- RemoveAt

Propriétés :

- Item

IDictionary<TKey, TValue> :  
ICollection<KeyValuePair  
<TKey, TValue>>

Méthodes :

- Add
- ContainsKey
- GetEnumerator
- Remove
- TryGetValue

Propriétés :

- Item
- Keys
- Values

# Exemple de collection: File à double entrée

```
class DoubleEndedQueue<T> :  
ICollection<T>, IList<T>  
{  
    private List<T> items;  
  
    // Type specific methods.  
    public DoubleEndedQueue() {...}  
  
    public EnqueueItemAtStart() {...}  
  
    public DequeueItemFromStart()  
    {...}  
  
    public EnqueueItemAtEnd() {...}  
  
    public DequeueItemFromEnd() {...}  
  
    // Interface methods.  
    public void Add(T item) {...}  
  
    public void Clear() {...}  
  
    public bool Contains(T item) {...}  
  
    public void CopyTo(T[] array,  
        int arrayIndex) {...}
```

```
    public int Count  
    { get {...} }  
  
    public bool IsReadOnly  
    { get {...} }  
  
    public bool Remove(T item) {...}  
  
    public IEnumerator<T>  
        GetEnumerator() {...}  
  
    IEnumerator  
        IEnumerable.GetEnumerator() {...}  
  
    public int IndexOf(T item) {...}  
  
    public void Insert  
        (int index, T item) {...}  
  
    public void RemoveAt(int index)  
    {...}  
  
    public T this[int index]  
    {  
        get {...}  
        set {...}  
    }  
}
```

Le code complet est fourni en annexe de votre support.

- L'utilisation de **foreach** nécessite la méthode **GetEnumerator** (interface **IEnumerable<T>**)
  - Cette méthode retourne une instance d'une classe **IEnumerator<T>**
- Il est souvent plus simple de s'appuyer sur un objet interne qui implémente déjà **IEnumerable**

```
private List<T> data;  
  
public IEnumerator<T> GetEnumerator()  
{  
    return data.GetEnumerator();  
}
```

Il est également possible de produire ses propres fonctions ou propriétés **IEnumerable** grâce au mot-clé **yield**

```
// data est une collection quelconque
public IEnumerable<T> Backwards
{
    get
    {
        for (int i = data.Count - 1; i >= 0; i--)
        {
            yield return data[i];
        }
    }
}
```

```
foreach (var item in moObjet.Backwards){
    ...
}
```

## Leçon 3: Enregistrements

---

- Définition
- Syntaxe
- Utilisation

# Définition

- Type référence, optimisé pour les tests d'égalité
- Facilite la mise en œuvre de type référence immuable
- Deux enregistrements sont égaux
  - s'ils sont du même type
  - si toutes leurs propriétés sont égales
- Le compilateur génère:
  - Les méthodes de comparaison basées sur les propriétés (Equals, ==, !=)
  - La surcharge de GetHashCode()
  - La surcharge de ToString()

- Syntaxe abrégée

```
public record Ami(string Nom, string Prenom);
```

- Syntaxe complète

```
public record Ami
{
    public string Nom{ get; init;}
    public string Prenom{ get; init;}

    public Ami(string nom, string prenom) =>
        (Nom, Prenom) = (nom, prenom);
}
```

Rien n'empêche de remplacer l'accesseur *init* par l'accesseur *set*, mais l'enregistrement n'est plus dans ce cas immuable.



## Instanciation d'enregistrement

```
Ami a = new Ami("Doe", "John");  
Ami b = new("Doe", "Jane");
```

## Comparaison d'enregistrement

```
Console.WriteLine( a.Equals(b) );  
Console.WriteLine( a==b );
```

## Recopie d'enregistrement

```
Ami c = a with { Nom = "Toto" };  
Console.WriteLine(c); // Ami { Nom = Toto, Prenom = John }
```

## Leçon 4: Simplification du code

---

- Instruction de niveau supérieur
- Omission de type
- Abandon de paramètres

Un seul fichier de l'application peut utiliser cette simplification qui élimine le code inutile

```
using System;  
  
namespace HelloWorld  
{  
    class Program  
    {  
        static void Main(string[] args)  
        {  
            Console.WriteLine("Hello World!");  
        }  
    }  
}
```



```
using System;  
  
Console.WriteLine("Hello World!");
```

# Omission de type

- Consiste à omettre le nom du type utilisé dans une expression *new*
- Le nom du type est déterminé par le contexte

```
private List<WeatherObservation> _observations = new();
```

```
public WeatherForecast ForecastFor(DateTime forecastDate,  
    WeatherForecastOptions options){ ... }  
  
...  
  
var forecast = station.ForecastFor(DateTime.Now.AddDays(2), new());
```

```
WeatherStation station = new() { Location = "Seattle, WA" };
```

# Abandon de paramètres

- Ignorer deux paramètres ou plus passés à une expression Lambda
- Utile pour les gestionnaire d'événements

```
Func<int, int, int> constant = (_, _) => 42;
```

À des fins de compatibilité descendante, si un seul paramètre d'entrée est nommé `_` dans l'expression Lambda, alors, `_` est traité comme le nom de ce paramètre.

# Atelier 01

**Activité**  
**20 min**



- Exercice 1: Programmation asynchrone
- Exercice 2: Implémentation de l'interface IEnumerable
- Exercice 3: Utilisation de la syntaxe de niveau supérieur



## A retenir

1. **Nature Asynchrone** : "La programmation asynchrone permet d'exécuter des tâches de manière non séquentielle, offrant ainsi une meilleure réactivité et performance pour les utilisateurs et les systèmes."
2. **Avantages Asynchrones** : "En utilisant l'asynchronicité, les applications peuvent traiter plusieurs tâches simultanément sans se bloquer, améliorant ainsi l'expérience utilisateur et l'utilisation des ressources."
3. **Collections Personnalisées** : "Créer une classe de collection personnalisée permet d'adapter et d'optimiser le stockage et l'accès aux données selon les besoins spécifiques de votre application."
4. **Un Enregistrement, ce n'est pas qu'une Classe** : "Les enregistrements offrent une manière de représenter des données immuables sans toute la lourdeur associée aux classes. Ils sont parfaits pour représenter des objets de données légers."
5. **Simplicité et Expressivité** : "Simplifier le code ne signifie pas seulement le rendre plus court, mais aussi le rendre plus expressif, lisible et maintenable."
6. **Réduire la Complexité** : "Une des clés de la maintenance du code est de réduire la complexité. Les techniques de simplification du code aident à atteindre cet objectif."
7. **Testabilité** : "Un code simplifié est souvent plus facile à tester, car il a tendance à avoir moins de cas limites et de dépendances."
8. **Structure des Enregistrements** : "Contrairement aux classes, les enregistrements sont conçus pour être immuables et offrent une sémantique de valeur, ce qui les rend idéaux pour représenter des données qui ne changent pas."
9. **Evolution des Collections** : "Même si .NET offre une panoplie de collections, parfois les besoins spécifiques exigent la création d'une collection personnalisée pour optimiser les performances ou le comportement."
10. **L'Asynchrone est Partout** : "À l'ère des applications modernes, la programmation asynchrone n'est plus une option. Que ce soit pour les interfaces utilisateur, les appels réseau ou l'accès aux bases de données, l'asynchronicité est essentielle pour des applications réactives."

# **Chapitre 14 - Utilisation de LINQ pour interroger des données**



# Plan de la formation

- 1 - Introduction à C# et .Net
- 2 - Structure de programmation C#
- 3 - Déclaration et appel de méthodes
- 4 - Gestion d'exceptions
- 5 - Lire et écrire dans des fichiers
- 6 - Création de nouveaux types de données
- 7 - Encapsulation de données et de méthodes
- 8 - Héritage de classes et implémentation d'interfaces
- 9 - Gestion de la durée de vie des objets et contrôle des ressources
- 10 - Encapsulation avancée
- 11 - Découplage de méthodes et gestion des données
- 12 - Utilisation des collections et collections génériques
- 13 - Programmation asynchrone et parallèle
- 14 - Utilisation de LINQ pour interroger des données**
- 15 - Développement dirigé par les Tests

## Objectifs :

- Maîtriser les bases de
- Savoir construire des requêtes
- Comprendre l'interaction de LINQ avec d'autres
- Optimiser les requêtes LINQ



# Commençons par échanger

"Pouvez-vous partager une expérience où vous avez dû filtrer ou trier des données dans une application ou un projet précédent ? Quels outils ou méthodes avez-vous utilisés ?"

"Quand vous travaillez avec des bases de données, comment procédez-vous généralement pour extraire des informations spécifiques ? Y a-t-il des défis auxquels vous êtes fréquemment confrontés ?"

"Dans quelles situations trouvez-vous qu'il serait plus efficace d'intégrer des requêtes directement dans le code plutôt que de les externaliser ?"

"Lors de la manipulation de listes ou de collections en C#, avez-vous déjà ressenti le besoin d'avoir des outils plus expressifs ou flexibles pour interroger ces données ? Pouvez-vous donner un exemple ?"

"Avez-vous déjà été confronté à des problèmes de performances lors de la récupération ou du traitement de grandes quantités de données ? Comment avez-vous abordé ces défis ?"



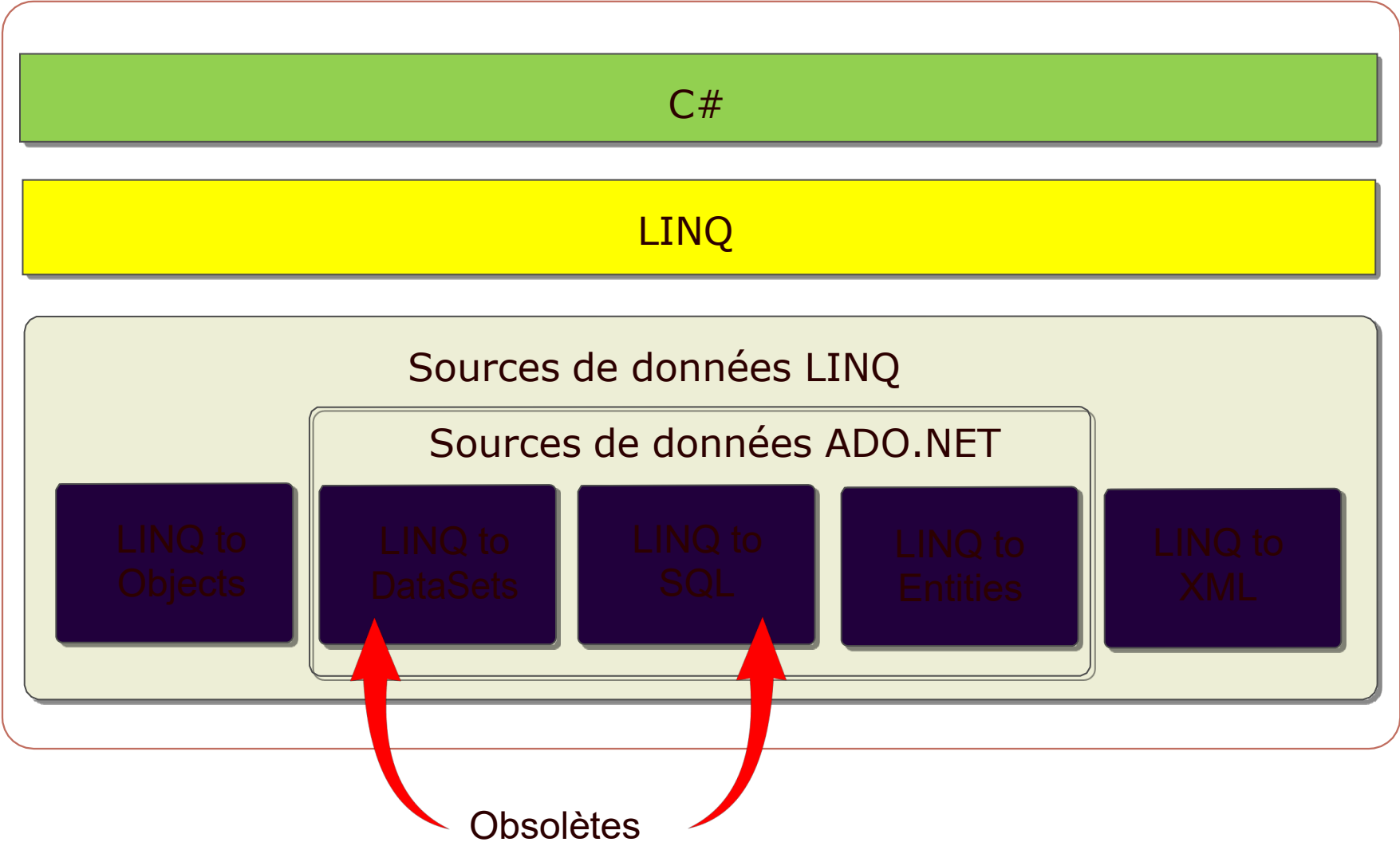
- Utilisation des méthodes d'extension LINQ et des opérateurs de requête
- Construction de requêtes et d'expressions LINQ dynamiques

# Leçon 1: Utilisation des méthodes d'extension LINQ et des opérateurs de requête

---

- Pourquoi LINQ?
- La face cachée de LINQ
- Interrogation de données
- Filtrage
- Tri
- Regroupement et agrégations
- Jointure
- Opérateurs de requête
- Modèles d'évaluation des requêtes

# Pourquoi LINQ?



# La face cachée de LINQ

LINQ est avant tout basé sur les méthodes d'extension et les expressions Lambda

LINQ tire partie également du typage implicite...

```
var myString = "Hello World!"; // myString est typé String  
var data = 5;                  // data est typé int
```

...et des classes anonymes

```
var obj = new {Name="Doe",FirstName="John",Age=30};  
// obj est une instance d'un type anonyme  
// qui expose 3 propriétés: Name, FirstName et Age
```

# Interrogation de données

Manipuler des données nécessite de pouvoir choisir les données

```
IEnumerable<Customer> customers = new[]  
{  
    new Customer{ FirstName = "...", LastName="...", Age = 41},  
    ...  
};
```

```
List<string> customerLastNames = new List<string>();  
foreach (Customer customer in customers)  
{  
    customerLastNames.Add(customer.LastName);  
}
```

Approche traditionnelle à l'aide d'un foreach

```
...  
IEnumerable<string> customerLastNames =  
    customers.Select(cust => cust.LastName);
```

Approche LINQ avec la méthode d'extension Select

La méthode Select est une méthode d'extension des types génériques IQueryable<T> et IEnumerable<T>

La méthode d'extension Where permet de filtrer les résultats

```
var customerLastNames =  
    customers.Where(cust => cust.Age > 25).  
    Select(cust => cust.LastName);
```

Tous les noms de  
clients d'âge  
supérieur à 25

La méthode Where filtre et la méthode Select projette comme le font les mots clés SELECT et WHERE du langage SQL



Attention:  
L'ordre des méthodes d'extension est important



Le tri des données est assuré par les méthodes `OrderBy`, `OrderByDescending`, `ThenBy` et `ThenByDescending`

- `OrderBy` et `OrderByDescending` assurent les tris simples

```
var sortedCustomers =  
    customers.OrderBy(cust => cust.FirstName);
```

Tri par prénom  
croissant

- `ThenBy` et `ThenByDescending` assurent les tris complexes

```
var sortedCustomers =  
    customers.OrderBy(cust => cust.FirstName).  
    ThenBy(cust => cust.Age);
```

Tri par prénom puis  
par âge

# Regroupement et agrégations

Des calculs d'agrégations sont possibles sur une collection

```
Console.WriteLine(  
    "Count:{0}\t\tAverage age:{1}\t\tLowest:{2}\t\tHighest:{3}",  
    customers.Count(), customers.Sum(cust => cust.Age),  
    customers.Average(cust => cust.Age),  
    customers.Min(cust => cust.Age),  
    customers.Max(cust => cust.Age));
```

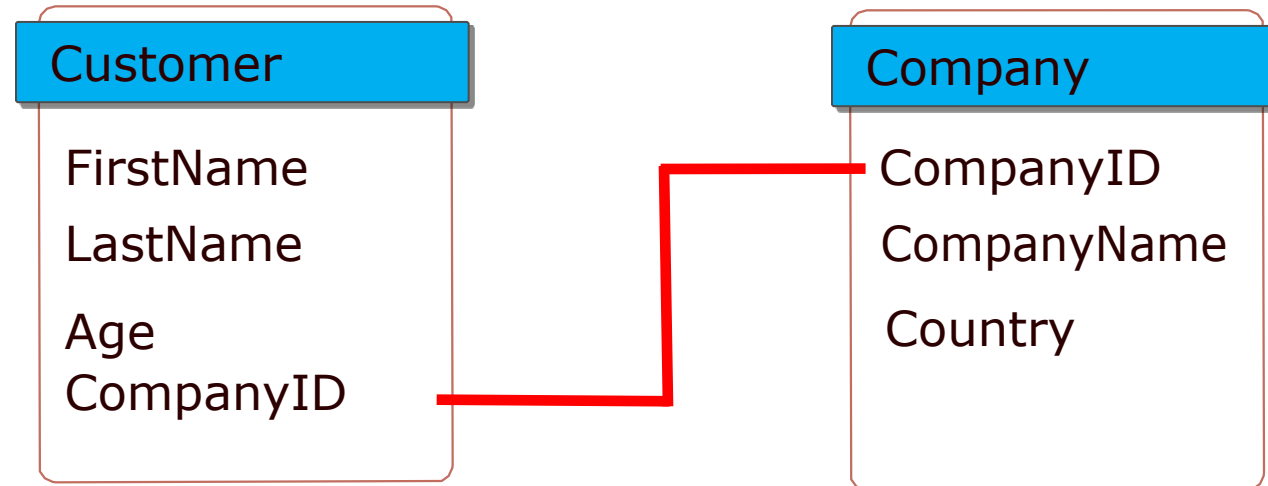
La génération de groupe est possible par la méthode GroupBy

```
var customersGroupedByAgeRange = customers.GroupBy( ... );
```

Il est également possible d'éliminer les doublons

```
Console.WriteLine("{0}",  
    customers.Select(cust => cust.Age).Distinct().Count());
```

LINQ fournit ma méthode d'extension Join afin de joindre des données de plusieurs sources distinctes en une seule requête



```
var customersAndCompanies = customers.Join(  
    companies,  
    cust => cust.CompanyID,  
    comp => comp.CompanyID,  
    (cust, comp) =>  
        new { cust.FirstName, cust.LastName, comp.Country});
```

# Opérateurs de requête

Les opérateurs de requête LINQ fournissent une syntaxe abrégée très proche de celle du langage SQL

```
IEnumerable<string> customerLastNames =  
    customers.Select(cust => cust.LastName);
```

*Approche par méthode  
d'extension*

```
IEnumerable<string> customerLastNames =  
    from cust in customers  
    select cust.LastName;
```

*Approche par  
opérateur de requête*

Il existe un opérateur de requête pour chaque méthode d'extension

```
var customersOver25 = from cust in customers  
    where cust.Age > 25 select cust;  
...  
var sortedCustomers = from cust in customers  
    orderby cust.FirstName select cust;  
...  
var customersGroupedByAge = from cust in customers  
    group cust by cust.Age;
```

# Modèles d'évaluation des requêtes

- La collection définie par votre requête LINQ n'est pas créée au moment où celle-ci est définie
- Cette stratégie est appelée *évaluation différée*

```
var usCompanies = from a in companies
                  where String.Equals(a.Country, "United States")
                  select a.CompanyName;
```

----- Données non récupérées

```
foreach (string name in usCompanies)
{
    Console.WriteLine(name);
}
```

----- Données récupérées

- Il est possible de forcer l'évaluation par l'utilisation de méthodes telles que ToList ou ToArray. On parle alors de *matérialisation* des données, car une copie des données est créée dans le programme

## Leçon 2: Construction de requêtes et d'expressions LINQ dynamiques

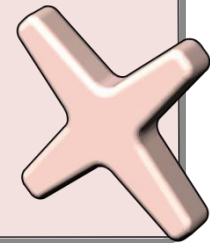
- Requêtes statiques vs Requêtes dynamiques
- Réflexion
- Arbre d'expression
- Expression
- Compilation et exécution

# Requêtes statiques vs Requêtes dynamiques

Les requêtes statiques sont puissantes mais nécessitent de connaître la requête au moment de la compilation

Les requêtes statiques:

- Vous oblige à écrire tous les cas possibles
- Nécessite parfois d'écrire du code inutile qui ne sera jamais exécuté
- Masque parfois des erreurs en utilisant des instructions conditionnelles complexes



Les requêtes dynamiques sont encore plus puissantes car elles sont construites au moment de l'exécution

Les requêtes dynamiques permettent:

- D'écrire une seule requête basée sur des paramètres fournis
- De produire une requête en fonction de l'utilisateur
- De n'écrire que le code nécessaire
- De réduire les erreurs potentielles



Ensemble de fonctionnalités pour analyser du code par le code

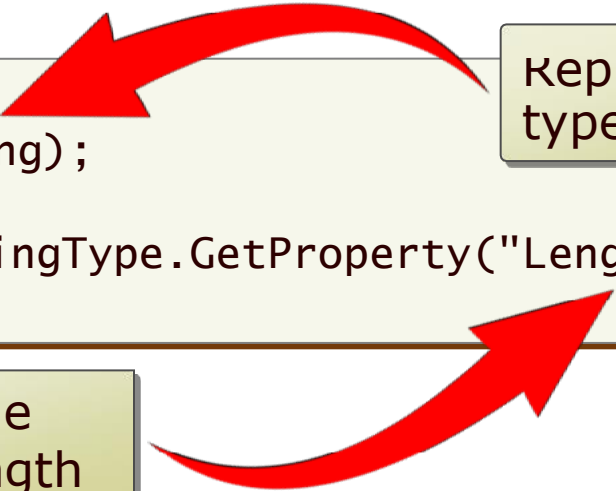
Nécessaire pour accéder aux membres des objets de manière dynamique

- La classe Type représente le type d'un objet
- La classe MemberInfo représente un membre

```
Type stringType = typeof(string);
```

```
MemberInfo stringLength = stringType.GetProperty("Length");
```

Représente le  
type string



```
graph TD
    A[Représente le type string] --> B[typeof(string)]
    C[Représente le membre Length] --> D[GetProperty("Length")]
```

Représente le  
membre Length

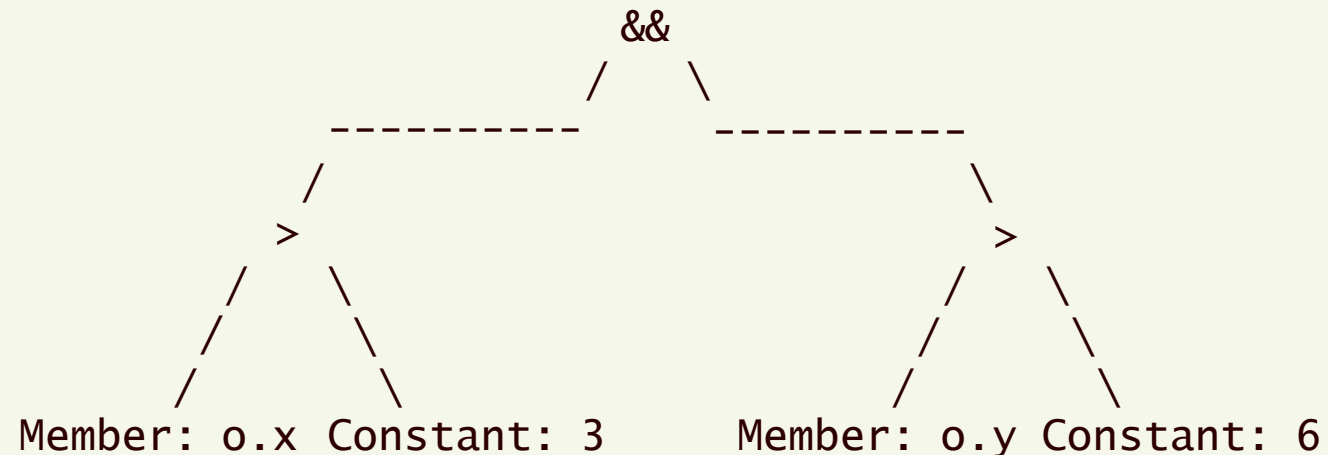
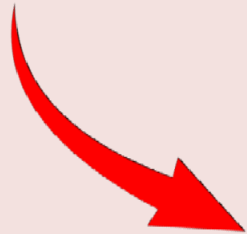


# Arbre d'expression

- Les arbres d'expression contiennent plusieurs expressions qui une fois évaluées, formeront la requête dynamique
- Les arbres d'expression fournissent une syntaxe flexible pour développer des expressions Lambda

Pendant la construction de l'arbre, chaque expression est décomposée en éléments individuels

< > 3 && y > 6



# Expression

## La classe Expression

Fournit des méthodes statiques pour créer les autres objets d'expression

## Les classes utiles

BinaryExpression, ConstantExpression, MemberExpression,  
UnaryExpression...  
Expression<TDelegate> qui représente une expression Lambda

```
x => x > 2 // expression Lambda à générer
```

```
Expression<Func<int, bool>> lambda = null;  
ParameterExpression param = Expression.Parameter(typeof(int), "x");  
ConstantExpression two = Expression.Constant(2, typeof(int));  
BinaryExpression body = Expression.GreaterThan(param, two);  
lambda = Expression.Lambda<Func<int, bool>>(body, param);  
Console.WriteLine(lambda.ToString());
```

# Compilation et exécution

La méthode d'instance `Compile` de la classe `Expression<TDelegate>` assure la compilation de l'arbre d'expression Lambda en expression Lambda exécutable

```
Expression<Func<int, bool>> expressionTree =  
    Expression.Lambda<Func<int, bool>>(expression, parameter);  
  
Func<int, bool> myDelegate = null;  
myDelegate += expressionTree.Compile();
```

Le résultat de la méthode `Compile` peut être passé directement en argument à une méthode d'extension LINQ

```
var bankAccounts = accounts.Where(expressionTree.Compile());
```

Il est également possible de tester l'invocation de l'expression Lambda par la méthode `DynamicInvoke` de l'objet `TDelegate` retourné par la méthode `Compile`

```
bool response = expressionTree.Compile().DynamicInvoke(1);
```

# Atelier 01

**Activité**  
**20 min**



- Exercice 1: Utilisation de requêtes LINQ



## A retenir

1. **Universalité de LINQ** : LINQ (Language Integrated Query) est une fonctionnalité puissante de C# qui permet d'interroger tout type de données, des collections aux bases de données, en passant par XML.
2. **Méthodes d'extension** : Les méthodes d'extension LINQ comme `Where()`, `Select()`, `GroupBy()`, etc., offrent une manière fluide et intuitive de construire des requêtes sur des collections.
3. **Syntaxe** : Avec LINQ, on peut utiliser une syntaxe proche du SQL directement en C#, ce qui rend les requêtes plus lisibles et expressives.
4. **Types Anonymes** : LINQ permet de créer des types anonymes pour structurer des résultats temporaires sans avoir besoin de définir des classes dédiées.
5. **Requêtes Différées** : Les requêtes LINQ sont évaluées de manière "différée", ce qui signifie qu'elles ne sont exécutées que lorsqu'on accède réellement aux données. Cela optimise les performances en évitant des opérations inutiles.
6. **Requêtes Dynamiques** : LINQ offre la possibilité de construire des requêtes dynamiquement, permettant une flexibilité accrue lors de l'interrogation des données.
7. **Sûreté de type** : Grâce à LINQ, on bénéficie de la vérification de type au moment de la compilation, réduisant les erreurs potentielles lors de l'exécution.
8. **Intégration avec d'autres sources de données** : Au-delà des collections en mémoire, LINQ peut être utilisé avec des bases de données (comme avec Entity Framework) ou des documents XML.
9. **Expressivité** : LINQ simplifie considérablement la construction d'opérations complexes sur les données, comme les jointures, les groupements ou les agrégations.
10. **Optimisation** : Bien que LINQ facilite l'interrogation des données, il est crucial de comprendre les implications en termes de performances et d'optimiser les requêtes lorsque cela est nécessaire.

# **Chapitre 15 - Utilisation de LINQ pour interroger des données**

# Plan de la formation

- 1 - Introduction à C# et .Net
- 2 - Structure de programmation C#
- 3 - Déclaration et appel de méthodes
- 4 - Gestion d'exceptions
- 5 - Lire et écrire dans des fichiers
- 6 - Création de nouveaux types de données
- 7 - Encapsulation de données et de méthodes
- 8 - Héritage de classes et implémentation d'interfaces
- 9 - Gestion de la durée de vie des objets et contrôle des ressources
- 10 - Encapsulation avancée
- 11 - Découplage de méthodes et gestion des erreurs
- 12 - Utilisation des collections et des génériques
- 13 - Programmation asynchrone et parallèle
- 14 - Utilisation de LINQ pour interroger des données
- 15 - Développement dirigé par les Tests

## Objectifs :

- Maîtriser les bases de
- Savoir construire des requêtes
- Comprendre l'interaction de LINQ avec d'autres
- Optimiser les requêtes LINQ



# Commençons par échanger

"Avez-vous déjà vécu une situation où un petit changement dans le code a provoqué des bugs inattendus ailleurs dans votre application ? Comment avez-vous géré cela ?"

"Selon vous, quels sont les critères essentiels pour juger de la qualité d'un code ? Avez-vous des méthodes ou des pratiques que vous adoptez régulièrement pour assurer cette qualité ?"

"Avez-vous déjà écrit des tests pour votre code ? Si oui, comment les avez-vous intégrés dans votre processus de développement ? Les écrivez-vous avant, pendant ou après le développement de la fonctionnalité ?"

"Pouvez-vous citer des avantages de l'automatisation des tests dans un projet ? Comment pensez-vous que cela pourrait impacter la livraison et la maintenance d'une application ?"

"Comment définiriez-vous habituellement le cycle de développement d'une fonctionnalité dans vos projets actuels ? Quelle place pourraient avoir les tests dans ce cycle ?"





- La place des tests dans le développement
- Modèles de conception d'application : MVC, MVVM
- Tests Unitaires et Visual Studio 2022

# Leçon 1: La place des tests dans le développement

---

- Méthodologies de développement
- Tests en développement

- Approche Traditionnelle
  - En cascade ou en V
  - Processus relativement linéaire
  - Peu évolutif, les besoins doivent être parfaitement connus dès le départ

- Approche Agile
  - Basé sur des cycles courts
  - Processus répétitif et évolutif
  - Les méthodes les plus répandues: Adaptative Software Development, eXtreme Programming, Scrum...
  - Technique souvent couplée à l'approche: Développement Dirigé par les Tests (TDD)
    - Un test est écrit
    - Le test échoue
    - Le code est écrit pour que le test réussisse
    - Le code est nettoyé
    - Un nouveau test est écrit...

# Tests en développement

---

- Test unitaire:
  - tester une fonction du système de manière autonome
- Test d'intégration
  - tester un ensemble de fonctions conjointement
- Test système
  - tester le logiciel sous forme de boîte noire dans un environnement compatible
- Test d'acceptation
  - vérifier que le produit est conforme aux spécifications

## Leçon 2: Modèles de conception d'application

---

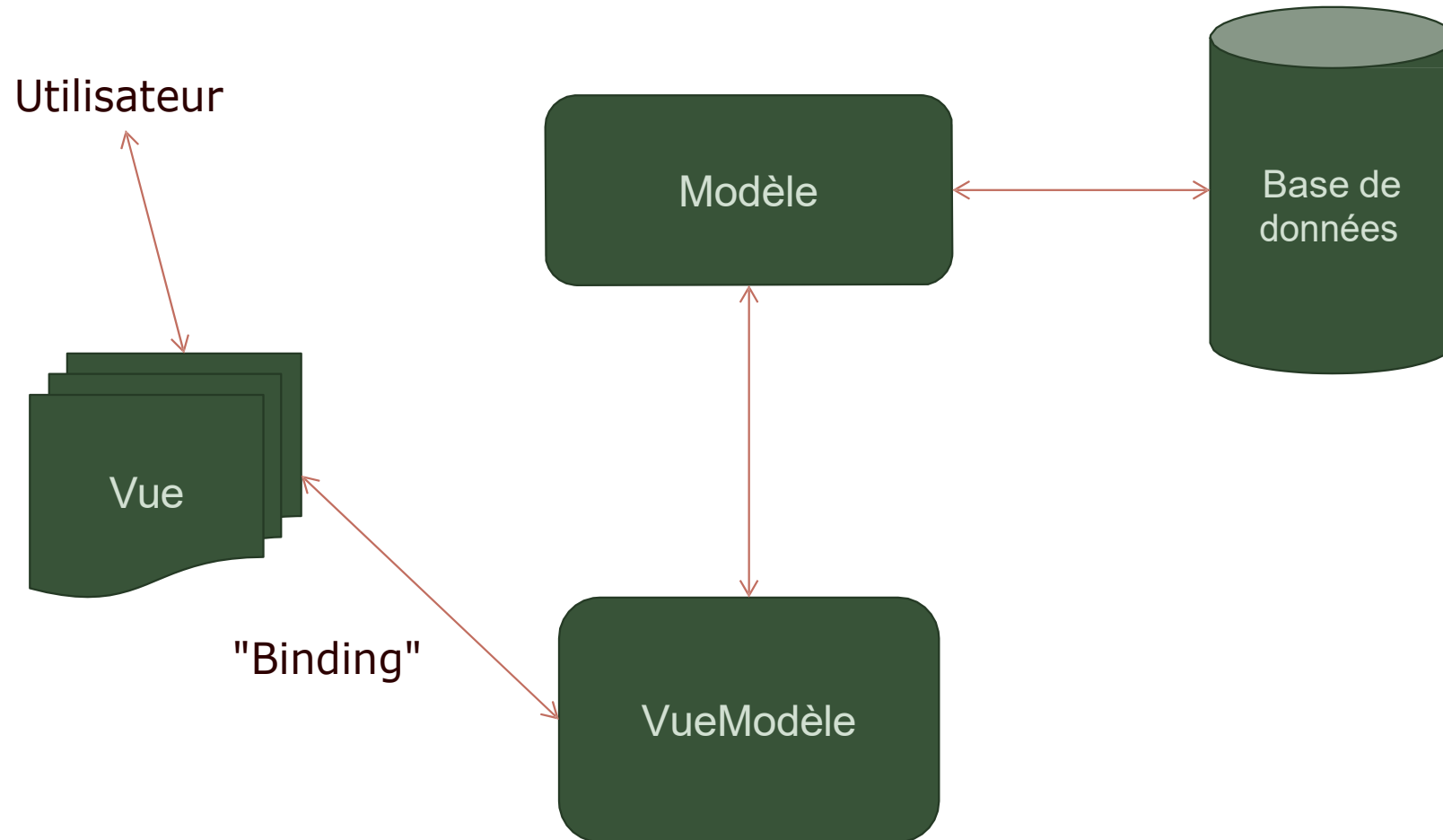
- Design Pattern: concepts
- MVVM
- MVC
- Bonnes pratiques du modèle MVC

# Design Pattern (Patrons de conception)

- Ensemble de caractéristiques de module
- Mise en œuvre de bonnes pratiques pour la résolution d'une problématique donnée
- Issus de l'expérience de concepteurs de logiciels
- Exemples:
  - Parmi les 23 patrons du *Gang Of Four*\*: Factory, Proxy, Singleton, MVC...
  - Repository, Disposable, MVVM ...

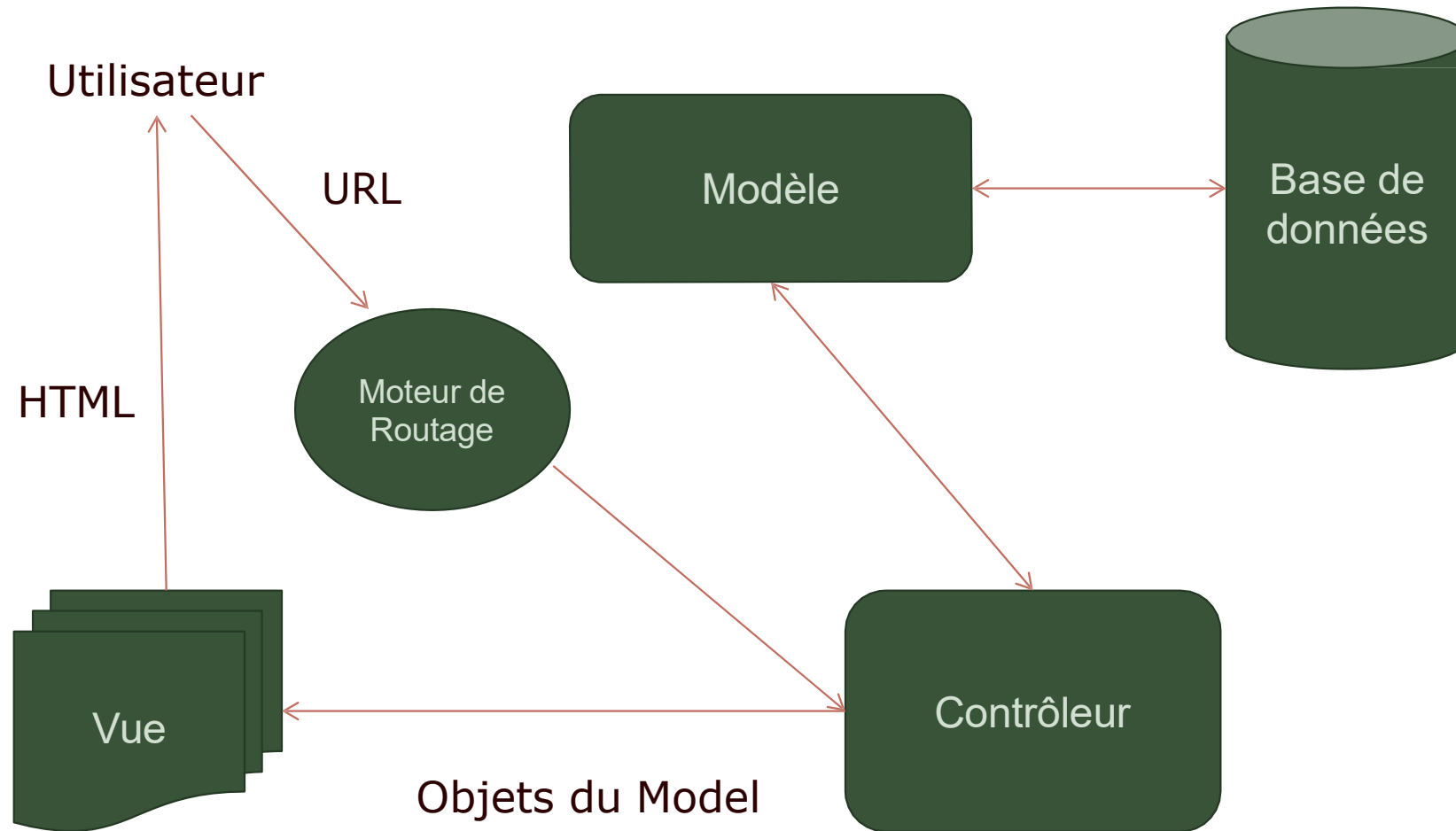
\* Design Patterns: Elements of Reusable Object-Oriented Software par Gamma E., Helm R., Johnson R. et Vlissides J.

## Modèle – Vue - VueModèle

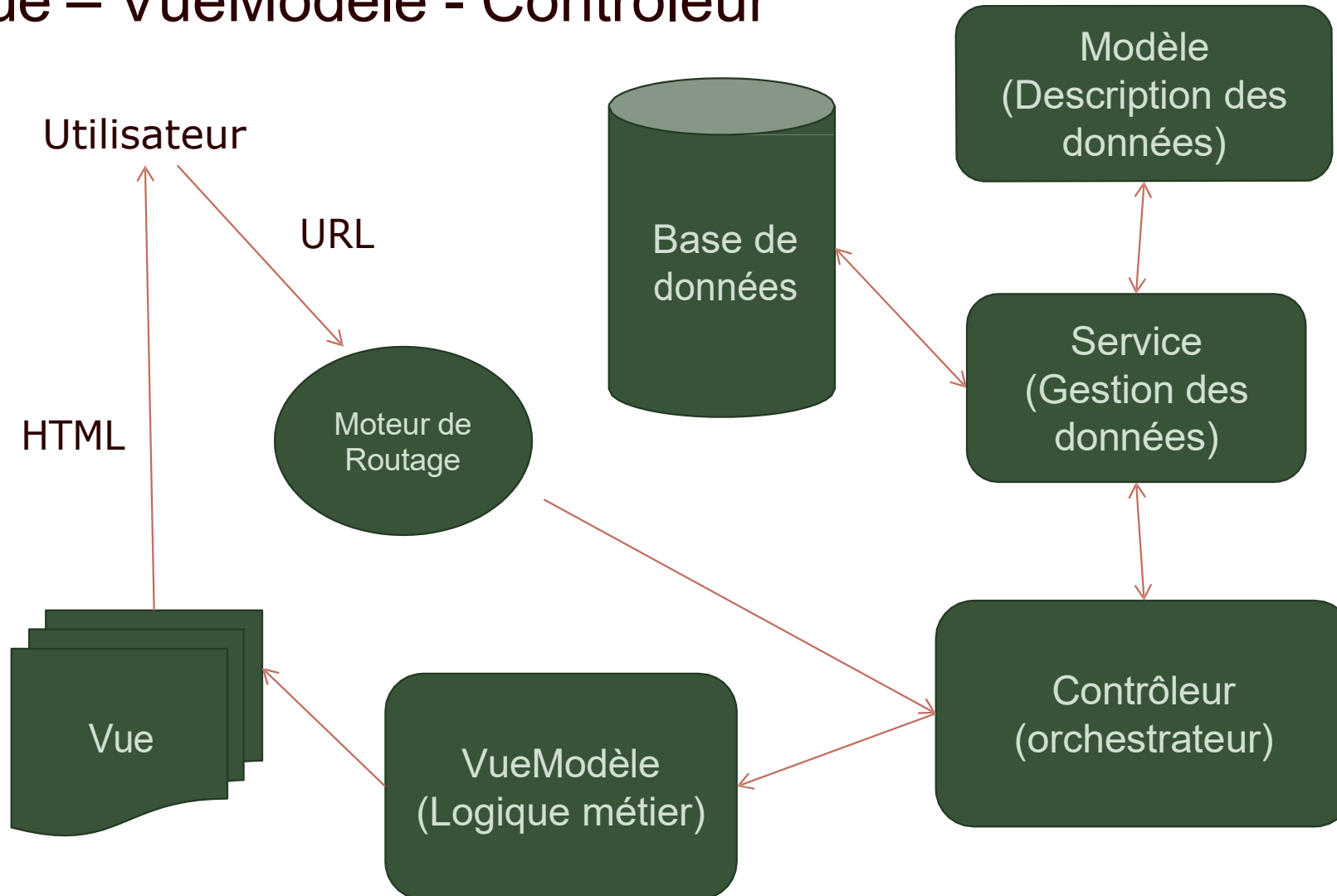




## Modèle – Vue – Contrôleur



## Modèle – Vue – VueModèle - Contrôleur



- Structure des Tests Unitaires
- Création de Tests dans Visual Studio
- Résultats des Tests dans Visual Studio
- Création de Tests avec .NET CLI
- Résultats des Tests avec .NET CLI
- Démonstration

- Arrange

- Préparer le test en initialisant les variables nécessaires

- Act

- Utiliser les variables pour invoquer la méthode et récupérer le résultat

- Assert

- Tester le résultat par rapport aux valeurs attendues et signaler la réussite ou l'échec du test

# Création de Tests dans Visual Studio

```
int Calculate(int operandOne, int operandTwo)
{
    int result = 0;

    // Un calcul.

    return result;
}
```

Ajout d'un projet  
de Test Unitaire



```
[TestMethod()]
public void CalculateTest()
{
    // Arrange
    Program target = new Program();
    int operandOne = 0, operandTwo = 0;
    int expected = 0;
    int actual;
    // Act
    actual = target.Calculate(operandOne, operandTwo);
    // Assert
    Assert.AreEqual(expected, actual);
}
```

Visual Studio 2017  
Enterprise Live  
Unit Testing:  
Validation des tests  
au fur et à mesure  
de l'écriture du  
code



# Résultats des Tests dans Visual Studio

Explorateur de tests

Rechercher

Exécuter tout

Exécuter... 1

Sélection : Tous les tests ,..

...||| Succès tests (4)

0

TestAddition< 1 ms

0

TestOpDroite< 1 ms

0

TestOpGauche11 ms

0

TestSoustraction< 1 ms

Résumé

Dernière série de tests Succès (Temps total d'exécution 0:0000,1604251)

0

4 tests Succès

Explorateur de tests

Rechercher

Exécuter tout

Exécuter... 1

Sélection : Tous les tests ,..

...||| Échec tests (1)

Ⓡ

TestAddition48 ms

...||| Succès tests (3)

0

TestOpDroite< 1 ms

0

TestOpGauche13 ms

0

TestSoustraction< 1 ms

TestAddition

Copier tout

Source : UnitTest1 ligne 26

Test Échec - TestAddition

Message :Échec de Assert.AreEqual. Attendu : <12>, Réel : <8>.

Temps écoulé : 0:00:00.0480134

...||| StackTrace :

UTClass1.TestAddition()

- Créer le projet de test

```
dotnet new mstest
```

- Référencer le projet à tester

```
dotnet add reference ..\MonProjet\MonProjet.csproj
```

- Ecrire le code nécessaire au test

- Lancer les tests

```
dotnet test
```

# Résultats des Tests avec .NET CLI

```
Démarrage de l'exécution de tests, patientez...  
Au total, 1 fichiers de test ont correspondu au modèle spécifié.
```

```
Réussi! - Failed:      0, Passed:      1, Skipped:      0, Total:      1, Duration: 11 ms- testprj.dll (net5.0)
```

```
Démarrage de l'exécution de tests, patientez...  
Au total, 1 fichiers de test ont correspondu au modèle spécifié.
```

```
Échoué TestMethod1 [46 ms]
```

```
Message d'erreur :
```

```
Assert.AreEqual failed. Expected:<150>. Actual:<15>.
```

```
Arborescence des appels de procédure :
```

```
at testprj.UnitTest1.TestMethod1() in C:\...\UnitTest1.cs:line 15
```

```
Échoué! - Failed:      1, Passed:      0, Skipped:      0, Total:      1, Duration: 46 ms- testprj.dll (net5.0)
```



- Création d'un projet de test unitaire

# Atelier 01

**Activité**  
**20 min**



- Exercice 1: Création de Tests Unitaires



## A retenir

1. **Primordial dans le Dev** : "Les tests ne sont pas une option ; ils sont essentiels pour garantir la robustesse et la fiabilité d'une application."
2. **Prévention avant la guérison** : "Les tests vous permettent de découvrir les erreurs bien avant que votre application n'atteigne l'utilisateur final, économisant ainsi du temps, de l'argent et de la réputation."
3. **TDD comme philosophie** : "Le TDD (Test Driven Development) n'est pas seulement une technique, c'est une philosophie : écrire des tests avant même le code garantit que le code répond spécifiquement aux besoins définis."
4. **MVC & MVVM** : "MVC et MVVM sont des modèles de conception qui permettent de séparer la logique métier de la présentation. Cela rend non seulement le code plus lisible, mais facilite également les tests."
5. **Autonomie des tests unitaires** : "Un test unitaire doit être autonome et indépendant des autres tests. Il doit tester une seule chose à la fois et le faire de manière cohérente."
6. **VS 2022 & Tests** : "Visual Studio 2022 offre des outils puissants pour écrire, exécuter et déboguer des tests unitaires, simplifiant ainsi l'adoption du TDD."
7. **Rétroaction rapide** : "L'une des plus grandes forces du TDD est la rétroaction rapide qu'il offre. Dès qu'une fonctionnalité est écrite, elle est testée, ce qui permet de corriger immédiatement les erreurs."
8. **Maintenabilité** : "Le code testé est plus facile à maintenir. Si une modification entraîne une régression, les tests le signaleront immédiatement."
9. **Confiance lors des refactorings** : "Avec une suite de tests robuste, refactoring et modification du code deviennent des opérations beaucoup moins risquées, car les tests assurent qu'aucune fonctionnalité n'est brisée."
10. **Document de référence** : "Les tests sont également une forme de documentation. Ils montrent comment le code doit se comporter et peuvent être utilisés pour comprendre les intentions originales du développeur."

# Synthèse, échanges



# Pour conclure

- L'Organisation : Savoir comment structurer vos équipes et canaux est essentiel pour une communication fluide.
- La Collaboration : Les outils, tels que les tableaux blancs et les sondages, peuvent enrichir vos réunions et discussions.
- La Personnalisation : Adapter Teams à vos besoins, que ce soit à travers les notifications ou les préférences de confidentialité, vous permettra de travailler plus efficacement.
- La Communication : L'utilisation efficace des @mentions et la structuration des conversations garantissent que l'information atteint les bonnes personnes.
- La Résolution de Problèmes : Être proactif et savoir où chercher les solutions est crucial pour minimiser les interruptions.

**Avant de se  
séparer**

- Retour sur vos attentes de cette formation



Tableau blanc Teams



**Merci pour  
votre  
attention**

